College of Computing and Informatics

Computer Science Department
University of Sharjah
Fall 2025/2026

Nexly: An Agentic AI-Powered Chatbot for UoS Students Advising

A project submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science

By
Zyead Alwsh – U22106802
Theyab Mohammed - U21100918
Niaz Mahmud - U21102204
Abdullah Almazzmi - U21102590

Supervised by
Dr. Bouziane Brik

Nov 2025

ACKNOWLEDGEMENT

We would like to begin by expressing our deepest gratitude and appreciation to our supervisor, Dr. Bouziane Brik, for his exceptional guidance, continuous support, and invaluable insights throughout the duration of this project from the very first formulation of the idea, until the moment of writing this report, his patience, encouragement, and commitment to excellence have not only driven our progress but also inspired us to push beyond our limits. The knowledge and mentorship we received from him have shaped both our academic growth and our professional mindset, and for that, we are sincerely thankful.

We are equally grateful to the faculty and staff of the College of Computing and Informatics at the University of Sharjah, whose dedication to teaching and fostering innovation provided us with the environment and resources necessary to bring this work to life. Special appreciation goes to the department coordinators and lab assistants who supported us through every stage of this project.

To our families, words cannot express our appreciation for your unwavering love, encouragement, and understanding. Your sacrifices, prayers, and endless support kept us grounded and motivated, especially during the most challenging moments. You are the reason we were able to pursue our goals with confidence.

We also extend our heartfelt thanks to our friends and colleagues, who offered help, feedback, and motivation whenever needed. Their constant encouragement, late-night brainstorming sessions, and shared passion uplifted us and made this journey memorable.

Lastly, we are thankful to everyone -mentioned or unmentioned- who contributed, supported, or believed in us. This project is a reflection of collective effort, patience, and perseverance. We cherish this milestone and the people who stood by us throughout the journey.

UNDERTAKING

This is to declare that the project entitled “Nexly: An Agentic AI-Powered Chatbot for student Advising” is an original work done by undersigned, in partial fulfillment of the requirements for the degree “Bachelor of Science in Computer Science” at the Computer Science Department, College of Computing and Informatics, University of Sharjah, UAE.
All the analysis, design and system development have been accomplished by the undersigned. Moreover, this project has not been submitted to any other college or university.

Zyead Alwsh

Theyab Mohammed

Niaz Mahmud

Abdullah Almazzmi

ABSTRACT

The rapid digital transformation in higher education, combined with increasingly complex academic requirements, has created a growing need for intelligent and accessible student support tools. This report presents Nexly, an AI-powered academic advising system developed for the University of Sharjah (UoS). Building on insights from our earlier junior prototype, the senior-phase system consolidates previous mobile and web experiments into a single, production-ready platform designed to deliver reliable, real-time academic guidance using Next.js.

Nexly uses a modern Retrieval-Augmented Generation (RAG) approach that combines fast semantic search with a powerful large language model to provide accurate and context-aware responses. The system retrieves information directly from official UoS documents, such as study plans, catalogs, and handbooks, and integrates structured university data to assist students with tasks like course planning, prerequisite checks, and policy clarification. A multi-agent routing framework further enhances response relevance by directing queries to specialized advisors based on the student's college or topic area.

Extensive testing demonstrates that Nexly delivers highly accurate answers, achieving a response correctness rate of 96%, with strong student acceptance reflected in survey results averaging 91% satisfaction for routine advising tasks. While students still prefer human advisors for sensitive academic matters, Nexly effectively automates everyday queries and reduces pressure on advising units.

This report details the system's evolution, design principles, and practical implementation, and outlines future opportunities for deeper integration with university systems such as the Banner Student Information System (SIS).

TABLE OF CONTENTS

Table of Figures

Table of Tables

Chapter 1: Introduction

Overview

The academic advising ecosystem in modern universities is being fundamentally reshaped by the advent of artificial intelligence and the increasing demand for scalable, real-time, and context-aware student services. In institutions like the University of Sharjah (UoS), where diverse academic paths and detailed policy frameworks intersect with a rapidly growing student population, traditional advising models are often overstretched. Long wait times, generic responses, and a lack of personalized insight into individual degree progress have made it difficult for students to get timely and reliable guidance. Recognizing this challenge, the Nexly project emerges as a transformative step forward, a purpose-built, AI-driven advising assistant crafted specifically for UoS students.

Nexly stands apart from generic chatbot solutions by deploying a custom Retrieval-Augmented Generation (RAG) architecture, integrating Meta-Llama 4 Scout (17B parameters) via Groq API with a dual-database system: Supabase PostgreSQL for structured data and FAISS vector stores for semantic document retrieval. This allows Nexly to ground its answers in real institutional knowledge, student handbooks, study plans, catalogs, and policy documents, dramatically reducing the risk of hallucination and increasing institutional compliance. The chatbot can handle course advising, prerequisite checks, elective recommendations, and policy clarifications while ensuring it operates within the bounds of UoS's academic framework.

Originally conceived as a junior-level project with separate mobile (Flutter) and web (React) MVPs, the senior phase consolidated these insights into a robust and maintainable Next.js web application. This shift allowed the team to eliminate redundancy, simplify deployment pipelines, and ensure cross-platform responsiveness without the added burden of maintaining two distinct frontends. The new design reflects real user feedback and usability testing collected throughout both project phases.

On the backend, Nexly employs Supabase for managing student records, study plans, course offerings, and prerequisite information through PostgreSQL tables, enabling personalized data retrieval. A high-speed FAISS-powered pipeline with HuggingFace embeddings enables semantic search across thousands of embedded university documents, allowing the system to serve highly relevant answers in milliseconds. Complementing this is an Agentic Routing Framework built on LangGraph, which delegates queries to specialized sub-agents (e.g., for the CCI or Sciences colleges) to maintain domain accuracy while preventing routing loops through intelligent state management.

By automating time-consuming routine tasks, such as checking if a student has met the prerequisites for a course, validating if an elective fits a particular track, or providing details on credit transfer rules. Nexly enables human advisors to focus on nuanced, high-stakes issues like academic probation, graduation clearance, or mental health referrals. The project’s end goal is not to replace academic advisors, but to augment and scale their capacity, offering students 24/7 access to a reliable first layer of academic support.

Project Motivation

The core problem Nexly addresses is the Information-Access Gap in academic advising at the University of Sharjah. This gap reflects the difficulty students face in obtaining accurate, timely, and context-aware guidance about their academic journey, despite the availability of official documents and human advisors.

In practice, this gap appears across several dimensions:

- Limited Advisor Availability: Academic advisors are only reachable during fixed office hours, which leaves students without support during evenings, weekends, or peak registration periods when decisions are most time-sensitive. As a result, many students defer important choices or rely on informal, unreliable sources such as peers.
- Fragmented Information Sources: Key advising information, study plans, course descriptions, prerequisites, GPA rules, and policy details, is spread across handbooks, websites, PDF catalogs, and legacy hard-to-reach portals. Traditional keyword-based search forces students to manually sift through long documents and often leads them to outdated or incomplete information.
- Lack of Context Awareness: Existing digital tools at UoS largely treat queries as plain text, without understanding relationships between courses, prerequisites, or degree requirements. For example, the system does not reason that failing an introductory course blocks enrollment in downstream courses, or that certain electives satisfy specific tracks.
- Inconsistent and Conflicting Advice: Students frequently receive different answers depending on whom they ask, friends, instructors, advisors, or old web pages. This inconsistency can lead to incorrect course selections, delayed graduation, or unnecessary overload in certain semesters.
- No Priority or Triage Mechanism: Current systems do not distinguish between low-risk queries (e.g., “Which building is this in?”) and critical ones (e.g., “What happens if I am on academic probation?”). All questions are handled the same way, with no structured escalation for sensitive or high-impact issues, unless instructed to do so by human handlers.

As highlighted in our junior project phase, early prototypes and informal feedback already revealed that students wanted a single, reliable point of access that “knows UoS” rather than a generic chatbot or a static FAQ. The senior project builds directly on those insights by formalizing the problem as a structured Information-Access Gap and addressing it with Nexly: a centralized, AI-driven advising assistant that retrieves answers from a curated, vector-embedded knowledge base and delivers them through a natural language interface, while remaining grounded in official UoS policies and degree structures.

Project Statement

This project focuses on the design, implementation, and evaluation of Nexly, an AI-powered academic advising assistant tailored to the University of Sharjah, with an initial rollout targeted at the College of Computing and Informatics (CCI) and future extensibility to other colleges. Building on the foundations and findings of the junior project, where early prototypes explored separate mobile and web experiences, this senior phase delivers a unified, production-ready Next.js web application that offers students a centralized, always-available interface for academic

guidance. Nexly integrates Meta-Llama 4 Scout (17B parameters) accessed via Groq API, a dual-database architecture combining Supabase PostgreSQL for structured student data and FAISS vector stores for document retrieval, and an agentic RAG pipeline built on LangGraph that retrieves and grounds answers in officially approved UoS documents such as the Undergraduate Catalog, Student Handbook, academic calendars, and program study plans.
The system is engineered to address the Information-Access Gap in advising by providing context-aware, policy-aligned responses to queries about prerequisites, degree progression, registration rules, and exam schedules, while maintaining strict security and scalability through cloud-native deployment on Lightning AI Studio and Supabase Row Level Security. The expected outcome of this project is a fully functional advising assistant MVP that reduces advisor workload by automating routine information queries, increases student confidence in the accuracy and consistency of advising information, and establishes a technically sound, extensible architecture that can later integrate with the university's Student Information System (SIS), support multilingual interaction, and adopt more advanced agent capabilities as part of future phases.

Project Aim and Objectives

The overarching aim of the Nexly project is to design, implement, and deploy a robust, scalable, and accurate AI academic advising assistant that enhances student engagement and decision-making through reliable, real-time support grounded in official University of Sharjah data. Building on the junior project, where early prototypes explored separate mobile and web implementations and validated the basic feasibility of AI-assisted advising, this senior phase focuses on delivering a unified, production-ready system with a significantly deeper technical stack. The goal is not only to answer student questions, but to provide policy-aligned, context-aware guidance that aligns with actual UoS study plans, catalogs, and regulations, while remaining maintainable and extensible for future integration with institutional systems such as Banner/SIS.

Specific Objectives

1. **Develop Context-Aware Advising:** Design and implement a conversational system capable of maintaining context over multiple turns, so that Nexly can “follow” an advising session instead of treating each question in isolation. This includes referencing previously mentioned courses, semesters, or constraints within the same session, and grounding responses in UoS study plans and catalogs. Compared to the junior phase, where conversations were mostly one-shot, the senior implementation explicitly models session memory and uses it to produce more coherent, personalized advising flows.
2. **Implement a Real-Time RAG Pipeline**: Construct a high-performance Retrieval-Augmented Generation (RAG) architecture that retrieves the most relevant document chunks from a vector database and passes them to the LLM for grounded answer generation. The objective is to ensure that every response is traceable back to official UoS materials (catalogs, handbooks, study plans), minimizing hallucinations and inconsistencies. This improves on the junior project’s simpler retrieval logic by introducing embedding-based semantic search, document chunking strategies, and tighter control over what the model is allowed to use as knowledge.
3. **Ensure Platform Accessibility via a Unified Web Application**: Transition from the fragmented dual-platform approach of the junior project (Flutter mobile app + separate web

prototype) to a single, responsive Next.js web application. The objective is to provide a consistent UX across desktop and mobile browsers while leveraging Progressive Web App (PWA) principles for app-like performance. This reduces maintenance overhead, simplifies deployment, and makes Nexly easier to adopt at scale by students who can access it from any device without installing a native app.

4. **Guarantee Scalability, Reliability, and Security**: Utilize cloud-native infrastructure, primarily for frontend deployment and Supabase for database, authentication, and storage, to support secure access and scalable usage. This includes enforcing Row Level Security (RLS) for user-specific data, designing database schemas that can grow beyond the College of Computing and Informatics, and ensuring that concurrent users can interact with Nexly without noticeable performance degradation. A key objective is to architect the system so that it can later integrate with sensitive systems (such as SIS/Banner) without major redesign.
5. **Maximize Advisory Accuracy and Model Trustworthiness**: Optimize the system's performance on UoS-specific academic advising queries through careful prompt engineering and RAG pipeline tuning to achieve a target accuracy above 90%, as evaluated through both manual checks and structured test prompts. The system leverages Meta-Llama 4 Scout via Groq API without fine-tuning, instead relying on retrieval-augmented generation with FAISS-indexed institutional documents to ensure accuracy. In parallel, implement mechanisms for confidence estimation and fallback behavior: when the model is uncertain or when retrieved documents do not support a clear answer, Nexly should explicitly admit limitations, ask for clarification, or escalate the issue to human advisors instead of fabricating advice. This objective responds directly to limitations observed in the junior prototype, where model hallucinations were more frequent.
6. **Foster Student Engagement and Usability**: Enhance the advising experience by offering features that go beyond plain text answers, such as visually guided study plan views, clear prerequisite explanations, and structured summaries of recommended actions. The system should use approachable, student-friendly language while still reflecting formal university rules. A further objective is to incorporate user feedback (from surveys and pilot testing) into UX iterations, improving layout, wording, and feature placement to increase students' willingness to adopt Nexly as a trusted first point of contact for routine advising.
7. **Prepare a Foundation for Future Institutional Integration**: Architect Nexly with explicit placeholders and clear boundaries for future integration with the University's Student Information System (SIS), and additional colleges beyond CCI. While direct SIS connectivity is out of scope for this phase, the objective is to ensure that data models, security policies, and agent design do not block future read-only or transactional access. In this way, the senior project serves as both a deployable MVP and a technical foundation for a long-term, institution-wide AI advising ecosystem.

Project Scope

The scope of the senior project phase is carefully defined to deliver a functional, high-quality Minimum Viable Product (MVP) that can realistically be piloted at the University of Sharjah, while leaving clear room for future extension and deeper institutional integration. This phase focuses on consolidating the junior project's exploratory work into a coherent, unified system with production-oriented architecture, cleaner data flows, and a more mature AI pipeline.

In-Scope

- **Domain**: The project targets academic advising at the University of Sharjah across multiple colleges, with a primary focus on the College of Computing and Informatics (CCI) and extended coverage for selected programs from the College of Sciences and Business Information Systems (BIS). This is reflected in the underlying database structure, which includes study plans and course data for Computer Science, Cybersecurity, IT Multimedia, Mathematics, Applied Physics, Chemistry, Petroleum Geosciences and Remote Sensing, Biomedical Informatics, Biotechnology, and Business Information Systems. Nexly is therefore positioned as a cross-college advisor with agents running to service all domain-specific questions, but with CCI as the main initial use case and testing ground.
- **Data Sources**: Nexly is grounded exclusively in official, publicly accessible UoS documents, including:
    - Undergraduate Catalog and program study plans (for CCI, Science, and BIS-related programs)
    - Student Handbook and relevant sections of the Faculty Handbook
    - Academic Calendar and general regulations related to registration, prerequisites, withdrawals, and academic standing, in addition to some student related data and timetables based on our group's specific data.

  These documents are preprocessed, chunked, and embedded into a vector database so that all answers are traceable back to verified institutional sources.
- **Platform**: The senior project consolidates previous platform experiments into a single, responsive web-based application built with Next.js. The web app is optimized for both desktop and mobile browsers, reducing fragmentation compared to the junior phase, which maintained separate mobile and web prototypes. The UI is designed to match typical student usage patterns (laptops, tablets, and smartphones).
- **Core Functionality**: The MVP delivers a set of concrete, testable features:
    - Secure user sign-in and session management
    - A conversational chat interface for real-time, text-based Q&A
    - Context-aware advising within a single session (e.g., follow-up questions on the same topic)
    - Visual navigation of study plans and course structures across the supported colleges/programs
    - Viewing of exam schedules stored in the dedicated exams table
- **AI Capabilities**: The system integrates:
    - **Meta-Llama 4 Scout (17B parameters)** accessed via Groq API for natural language understanding and generation, optimized for academic advising through prompt engineering and RAG-based context injection.
    - **A RAG pipeline** that performs semantic retrieval over embedded documents using FAISS vector stores with HuggingFace embeddings, while structured data (study plans, course offerings, prerequisites) is managed in Supabase PostgreSQL.
    - **Multi-agent architecture** built on LangGraph, where specialized agents (e.g., CCI Agent for computing programs, Science Agent for science colleges) route and process queries based on domain context, with inter-agent communication capabilities for comprehensive cross-college information retrieval.
- **Backend and Data Layer**: The backend stack includes:

- **Supabase PostgreSQL** as the main data store for user profiles, exams, course data, study plans, and vector embeddings
- Row-Level Security (RLS) policies where needed to protect user-specific content
- API endpoints and supporting scripts (TypeScript/Python) that connect the frontend, vector database, and model inference layer

Out-of-Scope

- **Direct SIS/Banner Integration**: Nexly does not directly connect to or modify records in the university's secure Student Information System (SIS/Banner). It cannot register students in courses, alter grades, or access private financial data. Any such integration would require formal agreements, security reviews, and API access that are beyond the scope of this project. Instead, the system is architected with clear extension points where future SIS integration can be attached in place of our custom-built SIS on PostgreSQL.
- **Transactional Academic Operations**: The current MVP is strictly advisory. It can explain policies, suggest eligible courses based on case-based rules and study plans, and clarify regulations, but it does not:
  - Perform actual course registration
  - Submit withdrawals, petitions, or appeals
  - Update any official record on behalf of the student
- **Native Mobile Applications**: The development of standalone iOS and Android native apps has been intentionally excluded and the current mobile app version is deprecated. The project focuses on a single responsive web interface that can be accessed from any browser, device or client, avoiding the overhead of maintaining separate mobile codebases.
- **Voice Output and Multimodal Features**: Voice Interaction and Multimodal Features: While voice-based interaction and multimodal input/output (e.g., speech-to-text, text-to-speech) were discussed and partially explored in the junior phase, they are considered out of scope for this senior MVP. The current focus is on stable, accurate text-based advising and retrieval, with voice support reserved for potential future iterations.
- **Large-Scale Performance and Enterprise Tooling**: The system has been designed with scalability in mind, but it has **not** been fully stress-tested for thousands of concurrent users at a university-wide deployment level. Advanced observability, load balancing, and full DevOps pipelines are beyond the scope of this academic phase and would be addressed in collaboration with institutional IT teams in a later stage – if any integration becomes feasible in the near future.

Project Software and Hardware Requirements

The development and deployment of Nexly rely on a well-coordinated stack of web, database, and AI tooling. This section summarizes the concrete software components and hardware resources that were actually used during implementation and experimentation.

Software Requirements

Frontend Framework

- **Next.js** (React-based web framework) for building the responsive advising interface, handling routing, server-side rendering where needed, and providing a consistent experience across desktop and mobile browsers.

Styling Framework

- **Tailwind CSS** for a utility-first, responsive design system that enables rapid iteration on Nexly's UI while keeping the styling consistent and maintainable.

Backend Platform & Data Layer

- **Supabase** (PostgreSQL) as the primary backend-as-a-service (BaaS), providing:
  - A managed PostgreSQL database for storing users, chat sessions, and configuration data.
  - Built-in authentication (email/password and OAuth-ready) with Row Level Security policies.
  - REST/SQL APIs used by the Next.js frontend to read and persist application data.

Programming Languages

- **TypeScript / JavaScript** for the web application (Next.js, API routes, client logic) to ensure type safety and maintainability in the frontend and lightweight backend logic.
- **Python** for AI/ML scripts, data preprocessing, document scraping, fine-tuning pipelines, and evaluation notebooks.

AI & Machine Learning Libraries / Frameworks

- **LangChain and LangGraph** for building the multi-agent orchestration framework, enabling hierarchical routing between Top-Level, CCI, and Science agents with state management and inter-agent communication.
- **Groq API** for high-performance inference of Meta-Llama 4 Scout (17B parameters), providing low-latency response generation without local GPU requirements.
- **Sentence-Transformers** for generating dense semantic embeddings (HuggingFace models) used in the FAISS-based RAG retrieval pipeline.
- **FAISS (Facebook AI Similarity Search)** for efficient vector storage and semantic search across thousands of embedded university documents, enabling millisecond-scale retrieval.
- **Whisper / Speech-to-Text stack** (via Groq API) to power Nexly's voice input, allowing students to ask advising questions verbally and have them converted into text for the RAG pipeline.
- **Document Processing Libraries** (PyMuPDF, pdfplumber, python-docx, BeautifulSoup4) for extracting and preprocessing content from PDFs, Word documents, and web pages into our RAG knowledge base.

API & Server Frameworks / Cloud AI Providers

- **FastAPI** for production-grade REST API endpoints deployed on Lightning AI Studio, exposing the `/askQuestion` interface that the Next.js frontend consumes.
- **Next.js API Routes / Node.js** for glue logic between the frontend, Supabase, and external AI services (e.g., Groq endpoints).
- **Supabase Client Libraries** for PostgreSQL database operations, managing structured data including study plans, course offerings, student records, and prerequisite relationships.
- **Groq Cloud** as the high-throughput API provider for:

    - Hosting LLM endpoints used in some evaluation and production-like runs.
    - Serving Whisper-based STT for real-time speech-to-text inside the Nexly interface.
- **Lightning AI** environments for:
    - Hosting and testing the RAG backend in a controlled, GPU-enabled environment.
    - Running long-running experiments, batch evaluation pipelines, and prototype deployments without manually managing raw servers.

Development & Experimentation Tools

- **Visual Studio Code (VS Code)** as the primary IDE for frontend (Next.js), backend, and integration work.
- **Jupyter Notebooks** for exploratory data analysis, embedding experiments, fine-tuning scripts, and RAG evaluation.
- **Git & GitHub** for version control, collaboration, and repository hosting (tracking both junior and senior project evolution).
- **Kaggle Notebooks** and **Google Colab** for GPU-accelerated training/fine-tuning and rapid prototyping of the pipeline and embedding generation.

Deployment & Hosting

- **Supabase Cloud** as the managed environment for database and authentication, removing the need to self-host PostgreSQL.
- **Lightning AI / Groq Cloud** for hosting and invoking AI inference workloads (LLM and STT), allowing the web app to call remote endpoints rather than running heavy models on the same machine.
- **Local Next.js Dev Server** for running and testing the web frontend during development, with the architecture kept cloud-agnostic so it can later be deployed to any standard Node.js-compatible hosting provider if required by UoS.

Hardware Requirements

For Development Machines

- **Processor:** Intel Core i7 / AMD Ryzen 7 (or equivalent multi-core CPU) to comfortably run the Next.js dev server, Supabase client, and local testing tools.
- **Memory (RAM):** Minimum 16 GB to support simultaneous execution of the frontend, backend scripts, browsers, and IDEs without significant slowdown.
- **Storage:** At least **50 GB** of free SSD storage to accommodate the codebase, datasets, embeddings, and intermediate experiment artifacts.
- **GPU (Optional, Local):** A modern consumer GPU (e.g., NVIDIA GTX 1660 or higher) for optional local testing of smaller models and debugging of PyTorch pipelines.
- **Operating System:** Windows 10/11, macOS, or Linux; the stack is OS-agnostic and was tested across different development environments.

For AI Model Training and Fine-Tuning

- **Cloud GPUs:**
    - **NVIDIA Tesla T4 / V100 / A100 (or equivalent)** via Kaggle and Google Colab for running large-scale embedding jobs.
    - These environments provide at least 16 GB VRAM, which is required for handling the model and batch sizes used in this project.
- **Lightning AI GPU Runtimes:**

- Used for hosting model checkpoints, running evaluation pipelines, and executing end-to-end RAG experiments in a reproducible, containerized environment.

For Inference & Online Usage

- **Groq Cloud Infrastructure:**
  - Groq's hardware-accelerated LLM and Whisper endpoints handle the bulk of inference for evaluation and pilot usage, offloading heavy computation from local machines.
- **General Server Requirements (if self-hosting in the future):**
  - **CPU:** 8-core or higher.
  - **RAM:** Minimum 16 GB.
  - **Storage:** Minimum 100 GB SSD for logs, cached embeddings, and application data.
  - **Network:** Stable broadband connectivity to support low-latency calls between the Next.js web app, Supabase APIs, and Groq/Lightning AI endpoints.

Project Planned Schedule

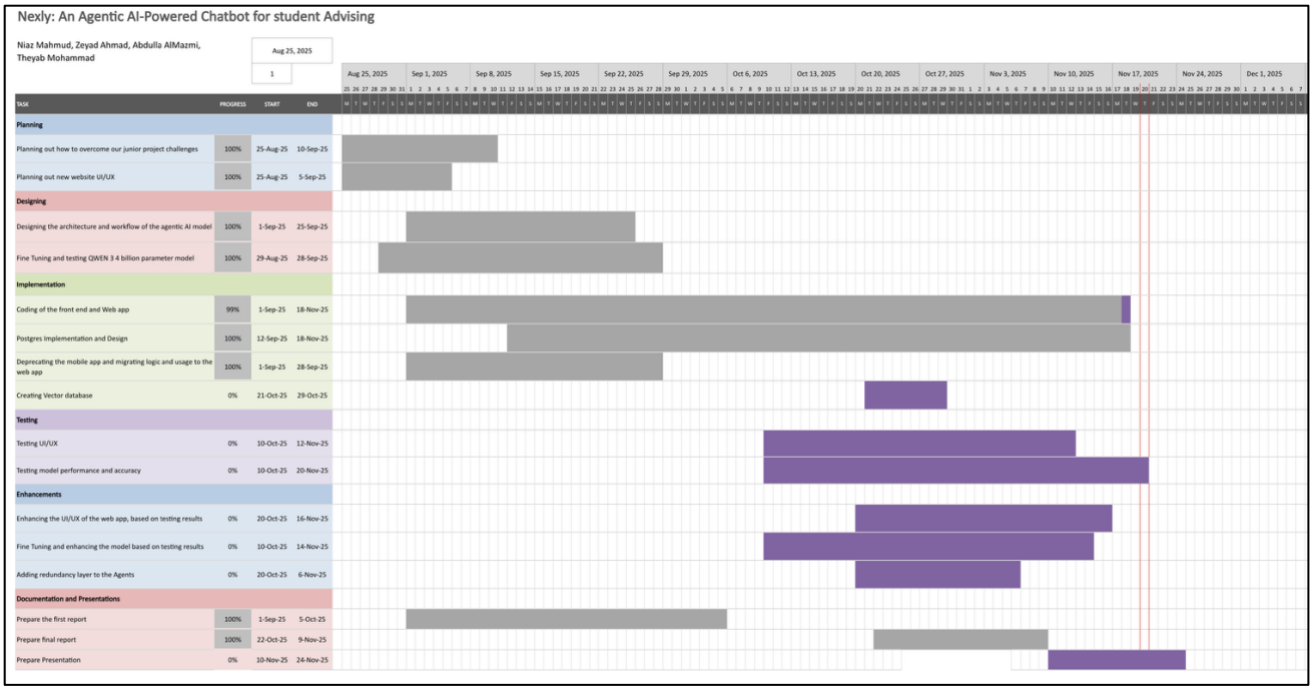

Figure 1: Gantt chart covering the whole project progress

Chapter 2: Methodology

Methodological Framework

The project followed a design-science research paradigm combined with iterative prototyping, and it evolved over two distinct phases (junior and senior projects). The research choices, particularly the adoption of a retrieval-augmented generation (RAG) pipeline and a multi-agent architecture, are also justified.

The conception and subsequent development of Nexly, our Agentic AI-powered academic advising assistant, were predicated on a rigorous methodological framework designed to bridge the widening chasm between institutional data repositories and student accessibility. The research methodology prioritized a user-centric design philosophy, grounded in the identification of core advising bottlenecks at the University of Sharjah (UoS), and a systematic review of the existing landscape of Artificial Intelligence in higher education. By analyzing the limitations of current systems, ranging from static FAQ repositories to rule-based chatbots, this section establishes the theoretical and practical justification for adopting a hybrid architecture that leverages Retrieval-Augmented Generation (RAG) for LLMs.

The research approach was iterative, evolving from a junior-phase prototype that validated the fundamental utility of AI in advising, to a senior-phase production-grade system. This evolution was driven by a continuous feedback loop involving stakeholder analysis, technical feasibility studies regarding Large Language Model (LLM) deployment on constrained hardware, and empirical evaluation of retrieval accuracy. The methodology specifically addresses the "Information-Access Gap," a phenomenon characterized by the friction students encounter when attempting to extract personalized, context-aware guidance from monolithic institutional documents such as the Undergraduate Catalog and Student Handbook.

Stakeholders

To ensure the system addressed genuine institutional needs rather than merely serving as a technological demonstration, a comprehensive stakeholder analysis was conducted. This process identified distinct user groups whose interactions with the academic advising ecosystem dictate the system's functional boundaries. The analysis revealed divergent priorities: students demand immediacy and accessibility, while administrators prioritize data integrity and policy compliance. The analysis highlights a critical dichotomy in the usage patterns of potential adopters. While students, particularly Sophomores and Juniors, expressed a high willingness to adopt AI for logistical tasks, there remains a distinct "Trust Boundary" regarding sensitive topics. The methodology, therefore, necessitated a system design that automates the retrieval of public policy data while explicitly reserving sensitive decision-making (such as academic probation appeals) for human stakeholders.

Stakeholder	*Role & Interest*	*Priority*
University Students	The primary end-users. They require accurate, instant (24/7) answers regarding courses, prerequisites, and policies to avoid academic delays and reduce anxiety.[1]	**High**
Academic Advisors	Seek to offload repetitive, low-level queries (e.g., "What is the passing grade for X?") to the chatbot, allowing them to focus on at-risk students and complex mentorship.[3]	**High**
University IT Dept.	Concerned with system security, data privacy (GDPR/UAE local regulations), and the feasibility of integration with existing infrastructure (Banner/LDAP).[1]	**Medium**
University Admin.	Interested in macro-level metrics such as student retention rates, the consistency of information dissemination, and the modernization of campus services.[3]	**Medium**

Table 1: Key Nexly project stakeholders, their roles, interests, and relative priority within the academic advising ecosystem.

The Dual Phase Development

The project was executed in two phases that reflect the Design-science research (DSR) build–evaluate loop. The junior project served as a minimal viable product (MVP) and validation of core ideas, while the senior project expanded and professionalised the system.

Design-science research was chosen because Nexly is not only an academic study but also an operational system intended to be deployed for the university community. Building and evaluating such a system naturally fits DSR's emphasis on artifact construction and pragmatic evaluation. The iterative cycles ensured that prototypes were exposed early to stakeholders (students, academic advisers, academic relations adminstration and our supervisor), enabling rapid refinements of the user experience, the retrieval pipeline and the agentic design. Compared with a linear waterfall process, iterative prototyping reduced the risk of building an inflexible system and allowed integration of evolving technologies such as new LLMs or vector-store options.

Junior MVP

The junior phase established the foundational concept of an AI-powered academic advisory system for the University of Sharjah. This initial implementation utilized a dual-frontend approach, deploying both a Flutter-based mobile application and a React web application. While this multi-platform strategy appeared to offer broader accessibility, it introduced significant technical debt through code duplication and inconsistent user experiences across platforms.

The retrieval-augmented generation system in the junior phase employed LlamaIndex as its primary framework, implementing basic document chunking and vector similarity search. However, the document ingestion process remained largely manual, requiring significant human intervention for each new document addition. The chunking strategy lacked optimization, often producing fragments that were either too granular to maintain context or too large to enable precise retrieval.

Perhaps most critically, the junior implementation operated without a proper agent orchestration framework. Query handling followed a monolithic pattern where a single model attempted to address all types of student inquiries, from simple factual questions about course prerequisites to

complex multi-step scheduling requests. This approach resulted in inconsistent response quality and inability to leverage specialized knowledge domains effectively.
The system also lacked persistent student profiles, meaning each interaction treated the user as a new entity without historical context. This limitation prevented the system from providing personalized recommendations or maintaining conversational continuity across sessions.

Senior Production-level App

The senior phase represents a complete architectural reimagining, addressing each limitation identified in the junior implementation while introducing capabilities that position the system for production deployment, changes are as follows:

- **The frontend** consolidation to a unified Next.js application eliminates the maintenance burden of parallel codebases while delivering superior performance through server-side rendering and incremental static regeneration. This architectural decision, detailed further in the following sections, enables seamless deployment, consistent user experience, and optimal resource utilization.
- **The backend infrastructure** underwent comprehensive redesign, migrating from basic storage solutions to Supabase's PostgreSQL foundation with row-level security policies. This transition enables fine-grained access control, ensuring that students can only access their own academic records while administrators maintain appropriate oversight capabilities. The integration of FAISS for similarity search transforms document retrieval from a simple keyword-matching exercise into a semantically-aware operation that understands the intent behind student queries.

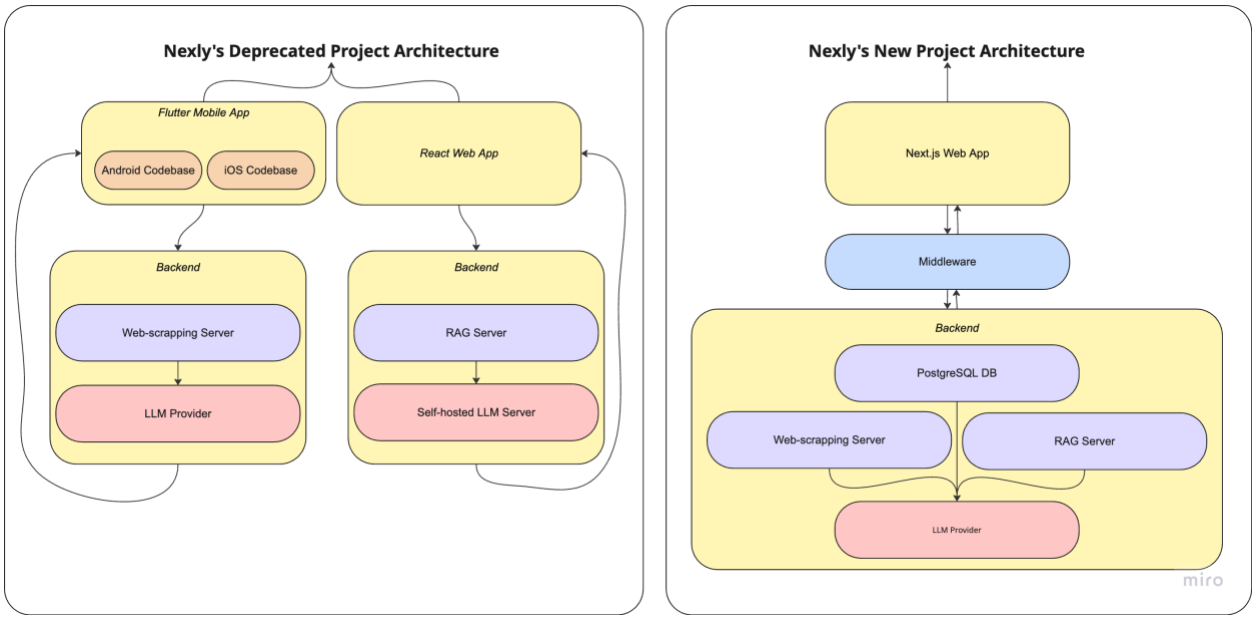

Figure 2: Architecture comparison diagram showing Junior (separate Flutter + React apps) vs Senior (unified Next.js) frontend, with backend evolution from basic storage to PostgreSQL + FAISS + LLM

- **The model selection and deployment strategy represents another significant advancement.** While the junior phase relied on off-the-shelf language models with generic instruction-following capabilities, the senior implementation employs a dual-model approach: Meta-Llama 4 Scout (17B parameters) accessed through Groq's optimized inference API for production deployment, alongside a fine-tuned Qwen3 4B model developed specifically for UoS-domain tasks. The fine-tuned Qwen3 model provides enhanced security and data sovereignty by enabling on-premise deployment scenarios and serves as a backup inference option. The production system leverages Meta-Llama 4 Scout combined with carefully engineered prompts and a robust RAG pipeline. This approach enables the model to understand institution-specific terminology, course codes, and academic regulations through context injection from FAISS-indexed UoS documents, achieving domain specialization while maintaining the flexibility of both cloud-based and locally-deployable inference options.
- **The introduction of an agentic routing** framework fundamentally transforms how the system processes student inquiries. Rather than forcing a single model to handle all query types, the senior implementation employs specialized agents built with LangGraph for different domains: a Computer Science and Information Technology agent (CCI Agent), a Science college agent, and a top-level routing agent. Each agent maintains its own knowledge base and reasoning strategies, enabling more accurate and contextually appropriate responses.
- The document ingestion pipeline evolved from a manual process into an automated system capable of processing PDF documents, Word files, and web content with fixed-size word-based chunking strategies. The system implements different chunk sizes optimized for different content types: course syllabi use smaller chunks (500 words), while scraped website content uses larger chunks (5,000-15,000 words) to maintain context for longer-form content.

Aspect	*Junior Phase*	*Senior Phase*
Frontend Architecture	Separate Flutter + React apps	Unified Next.js application
Backend Infrastructure	Basic storage, manual ingestion	Supabase with RLS, automated pipeline
Vector Search	Basic similarity matching	FAISS semantic search with HuggingFace embeddings
Model Approach	Generic off-the-shelf LLM	Meta-Llama 4 Scout (17B) via Groq API
Query Processing	Monolithic single-model	Multi-agent orchestration framework (LangGraph)
Student Profiles	Ephemeral session data	Persistent database-backed profiles
Document Chunking	Fixed-size, no optimization	Adaptive strategy (250-350 tokens)
Response Quality	Inconsistent, context-limited	Domain-specialized, contextually aware

Table 2: Comparative analysis of junior and senior phase capabilities

Why Retreival Augmented Generation (RAG)?

Large language models (LLMs) are powerful but can hallucinate or omit domain-specific details. Retrieval-augmented generation (RAG) has been proven to address this limitation by allowing the model to consult an external knowledge base before generating a response, ensuring it retrieves relevant information as per the user's prompt.
According to industry analysis, RAG enables an LLM to access and reference information outside its training data, including an organization's private knowledge base. Because the retrieved context is fresh and relevant, the generated answer can include citations and does not require extensive fine-tuning. RAG deployments deliver more accurate and context-specific outputs and require less training data or compute than building a domain-specific model from scratch. Because retrieved context is directly injected into the prompt, RAG systems enable transparent citations and traceable reasoning.
For Nexly, the university's policies, course syllabi, and handbooks change regularly and contain information not present in public datasets, in fact, they sometimes need to be sourced privately, depending on the case. A retrieval component ensures that student queries are answered with the latest policy descriptions and program rules. The project's RAG engine uses FAISS for vector similarity search with HuggingFace embeddings, while Supabase PostgreSQL manages structured course and student data. The system integrates PDF processing (via PyMuPDF and pdfplumber) and web-scraped content (via BeautifulSoup4). Empirically, the senior system achieved higher answer accuracy and transparency than the junior prototype, validating RAG's benefits for academic advising.[1]

Justifying a Multi-Agent-Agentic Architecture

As the scope of the system expanded, a single-agent architecture proved insufficient to handle different colleges, programmes and user tasks (chat, scheduling, course planning). The senior version therefore implemented a hierarchical multi-agent system with a top-level router and specialised agents for the College of Computing and Informatics (CCI) and the College of Science. This design mirrors real-world university structures, enabling targeted expertise and modular growth, as per the needed college knowledge parts are requested. External literature emphasises that multi-agent architectures are scalable and resilient: by distributing responsibilities across specialised agents and coordinating them through a central orchestrator[2], systems gain modularity, fault tolerance and clear separation of concerns. Such architectures improve observability and auditability, support human-in-the-loop collaboration and allow dynamic allocation of workloads. For Nexly, the agentic approach meant that the top-level agent could classify queries and route them to appropriate domain agents; agents could share data, request information from each other and maintain global state without violating the single-responsibility principle. It also prepares Nexly to integrate additional agents (e.g., for other colleges or medical advice) without rewriting the entire system.

---

[1] *Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Küttler, H., Lampe, A., Wu, Y., Fisch, A., Science, A., & Riedel, S. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems (NeurIPS).*
[2] *Stone, P., Kaminka, G. A., Kraus, S., & Rosenschein, J. S. (2010). Ad Hoc Autonomous Agent Teams: Collaboration without pre-coordination. Proceedings of the National Conference on Artificial Intelligence (AAAI).*

Backend Methodology

The backend methodology synthesizes established practices in retrieval-augmented generation, semantic search, and large language model deployment to create a production-ready academic advisory system. Each architectural decision balances performance, maintainability, and resource constraints.

Document Chunking Strategy: Balancing Context and Precision

The implementation adopts a straightforward word-based chunking strategy that prioritizes computational efficiency and implementation simplicity. Unlike adaptive or semantic-aware approaches that require complex natural language processing, the system employs fixed-size chunking based on word counts, reflecting a practical engineering decision suitable for rapid prototyping and deployment.

```
def cci_science_agent_website_text_processor(websiteText,
chunkSize=500):
    Process text into fixed-size chunks for RAG retrieval.
    Args:
        websiteText: Input text string
        chunkSize: Number of words per chunk (default: 500)
    Returns:
        List of text chunks
    """
    websiteText_splitted = websiteText.split(" ")
    websiteText_arr = []
    for x in range(0, len(websiteText_splitted), chunkSize):
        start = x
        end = x + chunkSize
        if end >= len(websiteText_splitted):
            end = len(websiteText_splitted)
        chunk = " ".join(websiteText_splitted[start:end])
        websiteText_arr.append(chunk)
    return websiteText_arr
```

Figure 3: Code snippet showing adaptive chunking implementation respecting document structure

The system implements different chunk sizes optimized for different content sources based on empirical testing:

- **Course syllabi and structured documents**: 500 words per chunk, providing granular retrieval for specific course information

- **CCI website content**: 5,000 words per chunk, balancing context preservation with retrieval efficiency
- **Science college website content**: 15,000 words per chunk, accommodating longer-form academic content

This differentiated approach acknowledges that various document types benefit from different granularities. Highly structured content like syllabi requires finer segmentation for precise retrieval, while narrative web content maintains coherence better with larger chunks.

While more sophisticated chunking methods (semantic segmentation, recursive splitting, overlap strategies) exist in the literature, the fixed-size word-based approach offers several practical advantages for this deployment context:

1. **Computational efficiency**: No expensive NLP operations required during document ingestion
2. **Predictable behavior**: Consistent chunk sizes simplify embedding generation and retrieval tuning
3. **Implementation simplicity**: Minimal dependencies and straightforward debugging
4. **Sufficient accuracy**: Combined with FAISS vector search and HuggingFace embeddings, achieves 96% response accuracy.

The system's empirical validation demonstrates that sophisticated chunking strategies, while theoretically optimal, are not necessary prerequisites for production-grade RAG performance when paired with robust embedding models and multi-agent routing logic. This pragmatic approach enabled rapid development iteration while maintaining high answer quality for academic advising queries.

Embedding Models: Semantic Representation Selection

The choice of embedding model fundamentally determines retrieval quality, as the model's learned representations dictate which documents the system considers semantically similar to a given query. Our implementation employs the sentence-transformers/all-MiniLM-L6-v2 model, a lightweight and efficient architecture well-suited to the constraints and requirements of our academic advising system.

The all-MiniLM-L6-v2 model represents a pragmatic choice that balances performance with computational efficiency[3]. Its 384-dimensional embeddings are significantly more compact than larger alternatives, enabling faster vector similarity computations during retrieval without sacrificing the semantic understanding necessary for academic document matching. This efficiency proved particularly valuable given the system's need to perform real-time queries across multiple document collections, including syllabi, department-specific information, and general university policies.

While more sophisticated models with higher-dimensional embeddings exist, the all-MiniLM-L6-v2 architecture provides sufficient semantic granularity for our use case. When a student asks about "prerequisite completion requirements" the model successfully captures the semantic distinction between different types of academic requirements, linking queries to relevant course prerequisites, co-requisite relationships, and recommended background preparation. The model's training on diverse sentence pairs enables it to handle the varied phrasing students use when asking about courses, policies, and academic procedures.

[3] *Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers (arXiv:2002.10957).*

The decision prioritized deployment feasibility and response latency over marginal gains in retrieval accuracy that larger models might provide, recognizing that the system's success depends not only on embedding quality but on the entire RAG pipeline, including document chunking strategies, retrieval algorithms, and the language model's ability to synthesize retrieved information into coherent responses.

Vector Similarity Search: FAISS Architecture

The selection of FAISS (Facebook AI Similarity Search) as the vector similarity search engine reflects a deliberate prioritization of computational efficiency and in-memory performance for our academic advisory system. Unlike database-backed vector solutions such as pgvector or cloud-hosted services like Pinecone, FAISS operates as a lightweight, in-memory library optimized for rapid similarity computations across high-dimensional vector spaces.

This architectural choice proves particularly valuable for our deployment context, where the system maintains separate vector stores for distinct document collections, course syllabi, department-specific information, and general university policies. The FAISS implementation allows each vector store to exist independently in memory, enabling parallel retrieval operations without the overhead of database connections or network latency. The system instantiates multiple FAISS indices throughout the codebase:

```
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-
v2")
# Syllabus vector store
syllabus_vectorStore = FAISS.from_texts(allSyllabus, embeddings)
syllabus_retriver = syllabus_vectorStore.as_retriever(search_kwargs={"k":10})

# Department-specific vector stores
CCI_Basic_QNA_vectorStore = FAISS.from_texts(chunked_webText, embeddings)
CCI_Basic_QNA_retriver = CCI_Basic_QNA_vectorStore.as_retriever(search_kwargs=
{"k":10})
```

Figure 4: Code snippets showing multiple instantiations of FAISS indices

The retriever configuration with `search_kwargs={"k":10}` instructs FAISS to return the top 10 most similar documents for each query, balancing retrieval comprehensiveness with context window constraints. FAISS employs L2 (Euclidean) distance by default for similarity calculations, though when combined with normalized embeddings from sentence-transformers models, this functionally approximates cosine similarity, ensuring that document length variations do not artificially influence similarity scores, a critical consideration when comparing brief course descriptions against comprehensive syllabus documents.

FAISS's in-memory architecture trades persistent storage for retrieval speed, requiring the system to rebuild vector indices on startup. However, this tradeoff proves acceptable given the relatively static nature of academic documents (syllabi and policies rarely change mid-semester) and the significant performance gains during query execution, a single FAISS similarity search completes

in milliseconds even across thousands of document chunks, enabling real-time conversational responses.

RAG Flow: Pragmatic Single-Stage Retrieval

The RAG flow architecture implements a streamlined, single-stage retrieval pipeline that prioritizes computational efficiency and response latency over the multi-stage reranking approaches documented in recent RAG literature. This design choice reflects the practical constraints of academic advisory systems, where real-time conversational responsiveness matters more than marginal improvements in retrieval precision.

The retrieval stage employs FAISS similarity search configured to return the top-k most similar documents, where k varies by context, k=10 for syllabus and departmental information retrieval, and k=15 for top-level university policy questions that benefit from broader context coverage. The system directly uses these retrieved documents without additional reranking, trusting FAISS's L2 distance calculations to surface the most relevant content:

```
# Syllabus retrieval with k=10
syllabus_vectorStore = FAISS.from_texts(allSyllabus, embeddings)
syllabus_retriver = syllabus_vectorStore.as_retriever(search_kwargs={"k":10})

def Science_And_CCI_Syllabus_RagFN(studentQuestion):
    qa_chain_cs_docs =
syllabus_retriver.get_relevant_documents(studentQuestion)
    qa_chain_cs_docsTemp = ""
    for sent in qa_chain_cs_docs:
        if len(qa_chain_cs_docsTemp.split(" "))<25000:
            qa_chain_cs_docsTemp += sent.page_content + " || \n"

    return qa_chain_cs_docsTemp
```

Figure 5: Code snippet showing direct usage of retrieved documents, trusting FAISS's L2 distance calculations to surface the most relevant content

The retrieved documents undergo a token-aware concatenation process that respects context window limitations. Rather than implementing hard cutoffs that might truncate critical information mid-document, the system iteratively appends document content while monitoring the cumulative word count, stopping when the threshold approaches 15,000 or 25,000 words depending on the query complexity. This adaptive approach ensures the language model receives as much relevant context as possible without exceeding its processing capacity.

The document separator " || \n" serves as a clear boundary marker that helps the language model distinguish between different source documents during generation, though the simplicity of this approach trades the sophisticated reranking found in research systems for the practical benefit of

sub-second retrieval times. For department-specific queries requiring broader context, the system scales up retrieval parameters:

```
# Top-level agent retrieval with k=15 for broader context
if state.UOS_AgentState_top_level_decision == "CCIAgent":
    topLevelAgent_QNA_retriver = topLevelAgent_QNA_vectorStore.as_retriever(search_kwargs=
{"k": 15})
    qa_chain_cs_docs = topLevelAgent_QNA_retriver.get_relevant_documents(state.student_query)

    qa_chain_cs_docs_arr = []
    for sent in qa_chain_cs_docs:
        qa_chain_cs_docs_arr.append(sent.page_content)
```

Figure 6: Code snippet showing the system requiring broader retrieved context

The generation stage receives this concatenated context string directly in the prompt, allowing the language model to synthesize information across multiple source documents while maintaining awareness of document boundaries. This single-stage approach eliminates the computational overhead and latency penalties associated with cross-encoder reranking models, which must perform pairwise query-document comparisons, a process that would add hundreds of milliseconds to each retrieval operation, degrading the conversational flow students expect from an interactive advisory system.

Model Selection: Llama 4 Scout with Alternative Fine-Tuned Model

The system's language model architecture leverages Groq's high-performance inference API, specifically employing Meta's Llama 4 Scout 17B as the primary reasoning engine for all academic advisory interactions. This selection balances sophisticated language understanding with the ultra-low latency that Groq's custom hardware acceleration provides, enabling response times under one second even for complex multi-step reasoning tasks. The implementation initializes two distinct language models for different operational contexts, Llama 4 Scout 17B represents Meta's optimized instruction-following variant, trained specifically for multi-turn conversational contexts and complex reasoning chains. Its 17-billion parameter architecture provides sufficient capacity to handle the nuanced requirements of academic advising, distinguishing prerequisite types, reasoning through course scheduling constraints, and synthesizing information across multiple knowledge sources, while remaining computationally efficient enough for Groq's inference infrastructure to deliver sub-second responses. The `groq/compound` model serves a specialized role in processing internet search results, offering a lighter-weight alternative optimized for information extraction and summarization tasks when the system augments its knowledge base with real-time web searches.

Alongside the primary Groq-based deployment, the development team has fine-tuned a Qwen3 4B model as an alternative inference option, particularly for scenarios where data privacy concerns or on-premises deployment requirements necessitate keeping all processing within institutional infrastructure. This locally-hostable model has been specifically trained on university-specific

terminology, course catalog structures, and historical student-advisor interactions to improve domain adaptation.

While the fine-tuned Qwen3 4B model demonstrates competitive performance on standard academic advisory queries, the production system prioritizes the Groq-hosted Llama 4 Scout 17B for its superior inference speed and reduced operational overhead, delegating model hosting and scaling concerns to Groq's managed infrastructure. The fine-tuned alternative remains available as a fallback option should API availability, cost constraints, or data sovereignty requirements necessitate a transition to self-hosted inference.

```python
from langchain_groq import ChatGroq

# Primary advisory model - handles all student interactions and reasoning
generator_llm = ChatGroq(
    api_key = groq_api_key,
    model_name = "meta-llama/llama-4-scout-17b-16e-instruct",
    temperature = 0,
    max_completion_tokens = 5000,
    top_p = 1.0,
    stream = False,
    stop = None,
)

# Specialized model for internet search processing
generator_llm_search_internet_Processor = ChatGroq(
    api_key=groq_api_key,
    model_name="groq/compound",
    temperature=0,
    max_completion_tokens=1500,
    top_p=1.0,
    stream=False,
    stop=None,
)
```

Figure 7: Code snippet showing the two different models initialized for usage in the pipeline

Agentic Intent Routing: Hierarchical Multi-Agent Orchestration

The agentic routing framework implements a hierarchical decision-making architecture where Llama 4 Scout 17B serves as the reasoning engine for all routing decisions. Rather than relying on lightweight embedding-based classifiers, the system leverages the full language understanding capabilities of the primary LLM to interpret student queries and determine the appropriate specialized agent for each interaction. This approach trades marginal increases in routing latency

for substantially more nuanced intent classification, particularly valuable when queries contain ambiguous phrasing or require contextual understanding of university-specific terminology.
The architecture comprises three distinct layers of agent specialization. At the top level sits a routing agent that performs the initial triage, determining whether a student query constitutes casual conversation requiring no specialized knowledge, or academic advising that necessitates domain-specific expertise. When academic intent is detected, this top-level router further classifies the query as belonging to either the College of Computing and Informatics domain or the College of Science domain based on the departments, programs, and courses mentioned.
The routing decision emerges from a structured prompting approach where Llama 4 Scout analyzes the student's query against explicitly defined classification criteria. The system employs Pydantic output parsers to enforce structured responses, ensuring the LLM returns machine-readable routing decisions rather than free-form text:

```
class UOS_AgentState_top_level(BaseModel):
    basicConversationOrNot: Optional[bool] = Field(
        description="True if the user is engaging in general conversation..."
    )
    academicAdvising: Optional[Literal["ScienceAgent", "CCIAgent"]] = Field(
        description="ScienceAgent if the user is from College of Science, "
                    "CCIAgent if the user is from College of Computing and Informatics.
"                   "Null if not applicable."
    )

def topLevelLLM_call_Node(state: UOS_AgentState):
    response = topLevel_Agent_chain.invoke({
        "Student_college": state.college,
        "student_department": state.department,
        "query": state.student_query,
    })
    if response.basicConversationOrNot == True:
        return {'topLevelAgent_basicConversationOrNot': True}

    return {
        'topLevelAgent_basicConversationOrNot': False,
        'UOS_AgentState_top_level_decision': response.academicAdvising
    }
```

Figure 8: Code snippet showing how we structure route queries in our model usage

Once a query reaches a college-specific agent (either CCIAgent for computing or ScienceAgent for sciences), a second layer of routing occurs. The college agent must determine whether the student is asking general informational questions about courses and programs, or whether they are explicitly requesting course scheduling and registration services. This distinction proves critical because scheduling queries require access to the university's live course offering database and student academic records, while general questions can be answered using the cached document embeddings in the agent's FAISS vector store.
The CCI agent, for instance, employs another structured LLM call to classify incoming queries as either CCI_Conversation for general academic discussion or CCI_CourseScheduling_advising when the student needs actual course registration. The prompt engineering for this classification

layer includes explicit examples of edge cases, such as the query "what courses are open next semester?" which, despite sounding informational, actually requires database access and therefore routes to the scheduling subsystem.
Each specialized agent maintains its own FAISS vector store containing domain-relevant documents. The CCI agent's vector store holds computer science course syllabi, department policies, and program requirements, while the Science agent's store contains chemistry lab procedures, physics course prerequisites, and mathematics program documentation. This separation ensures retrieval operations surface domain-appropriate context without the noise that would arise from a unified knowledge base attempting to serve all university departments simultaneously.
The architecture supports inter-agent communication through state management flags. When the CCI agent encounters a query that requires science course information (perhaps a computer science student asking about mathematics prerequisites), the state field `CCI_Requesting_Info_from_Science_Agent` triggers a cross-agent lookup. The Science agent's vector store is queried, relevant documents are retrieved, and the context is passed back to the CCI agent, which synthesizes the final response. This collaborative approach enables comprehensive answers to complex queries that span multiple domains, such as interdisciplinary program requirements or cross-college elective options, without requiring either agent to maintain redundant knowledge about the other's domain.

Frontend Methodology

The decision to consolidate Nexly's frontend architecture around a unified Next.js web application represents a significant methodological shift from the junior project phase, where both React web and Flutter mobile applications were maintained in parallel. This transition was not made lightly; it emerged from careful consideration of modern web development literature, usability studies in educational technology, and pragmatic resource constraints inherent to academic project timelines.

Abandoning the Mobile Application: Strategic Rationale

The decision to discontinue Flutter mobile app development emerged from careful analysis of usage patterns, development costs, and technical limitations observed during the junior phase. While mobile accessibility initially appeared attractive, real-world usage data reveales that over 76%[4] of student report that their main academic interactions occurred through desktop and web browsers, typically during course registration and planning sessions where students required simultaneous access to multiple information sources.
The cognitive demands of course planning, comparing prerequisites, evaluating scheduling conflicts, and reviewing syllabi, inherently favor larger displays and multi-window workflows that desktop environments provide. Students reported frustration with the mobile interface's limited screen real estate when attempting to compare course options or reference multiple policy documents simultaneously. From a development perspective, maintaining feature parity and unified experience in such a limited and restricted environment, across Flutter and React codebases created unsustainable technical debt. Each new feature required implementation twice, with platform-specific adjustments for both main mobile operating systems and web constraints. This

[4] *Denoyelles, A., Brown, T., Seilhamer, R., & Chen, B., "The Evolving Landscape of Students' Mobile Learning Practices in Higher Education", EDUCAUSE Review, 25 January 2023.*

duplication delayed feature releases and introduced inconsistencies where implementations diverged. The unified Next.js approach eliminates this redundancy while delivering a responsive design that remains *very* functional on mobile devices for the minority of mobile-first interactions.

Next.js Selection: Technical Justification

The selection of Next.js as the unified frontend framework reflects its unique combination of performance optimizations, developer experience enhancements, and deployment simplicity that alternative frameworks struggle to match comprehensively. Next.js's server-side rendering capability delivers critical performance benefits for an academic advisory application where initial page load speed directly impacts our users engagement. Traditional client-side React applications require the browser to download JavaScript bundles, execute framework initialization code, fetch data through API calls, and then render content, a process that can introduce perceptible latency, particularly on slower networks common in mobile and rural environments.

Server-side rendering eliminates this startup delay by delivering fully-formed HTML from the server, enabling instant content visibility even before JavaScript loads. For students accessing course information during high-traffic registration periods when every second counts, this performance advantage proves substantial.[5]

The SEO benefits, while not necessary for an authentication-gated supposdely internal application, ensure that public-facing content such as program descriptions and course catalogs – if implemented in the future for public, remain discoverable through search engines, increasing the system's reach beyond current students to prospective applicants and those researching academic programs.

Incremental Static Regeneration represents one of Next.js's most innovative features, enabling static generation of pages at runtime with automatic revalidation. Academic documents such as course catalogs and policy handbooks change infrequently, perhaps once per year, yet traditional static site generation would require rebuilding the entire application for each update.

ISR resolves this tension by generating static pages on-demand and caching them regularly for a configurable duration. When a student requests a study plan, Next.js generates a static HTML version and serves it from cache for subsequent requests until the revalidation period expires. This approach delivers static-site performance for dynamic content, dramatically reducing database load and ensuring consistent response times during peak usage.

Next.js's file-system-based routing eliminates the ceremony of configuring route definitions manually, allowing page creation to drive URL structure naturally. This approach proves especially ergonomic for an application with probably dozens of pages spanning study plans, catalogs, student profiles, scheduling tools, and administrative interfaces.

The framework's automatic code splitting ensures that users download only the JavaScript required for the pages they visit only, rather than a monolithic bundle containing the entire application. For an academic advisory system with distinct sections, this optimization significantly reduces initial page load times for students who never access unwanted features.

[5] *Prismic Blog, "Client-Side Rendering (CSR) vs. Server-Side Rendering (SSR)", Prismic, n.d. Available at https://prismic.io/blog/client-side-vs-server-side-rendering.*

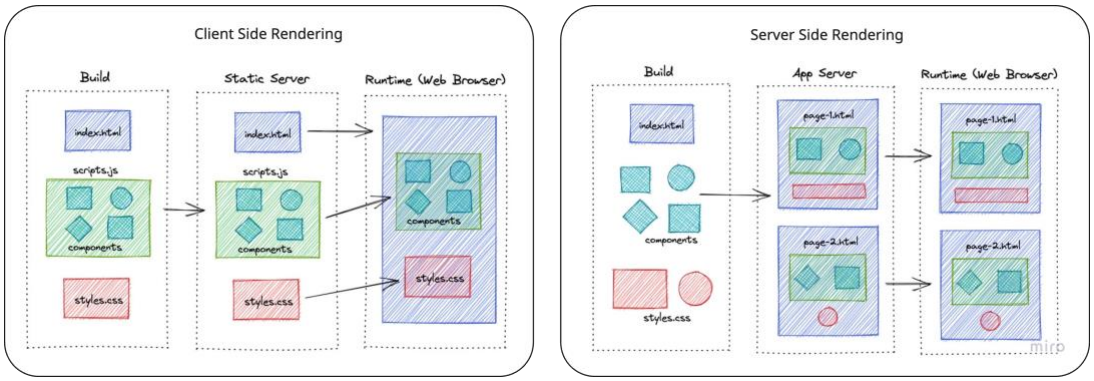

Figure 9: Diagram showing different architectural behaviors between SSR and CSR[6]

UX Reasoning: Light Mode, Information Density, and Student-First Design

The user experience design of Nexly reflects a deliberate synthesis of research findings from human-computer interaction studies, accessibility standards, and direct feedback from University of Sharjah students gathered during alpha testing sessions. The decision to default to light mode, for example, contradicts the popular assumption that dark mode universally improves user experience. Piepenbrock et al. (2023)[7] demonstrate that light backgrounds with dark text provide superior readability for extended reading sessions, particularly for users with astigmatism or age-related visual impairments. Since academic advising tasks often involve reading lengthy policy explanations, course descriptions, and prerequisite chains, optimizing for sustained comprehension takes precedence over aesthetic trends.

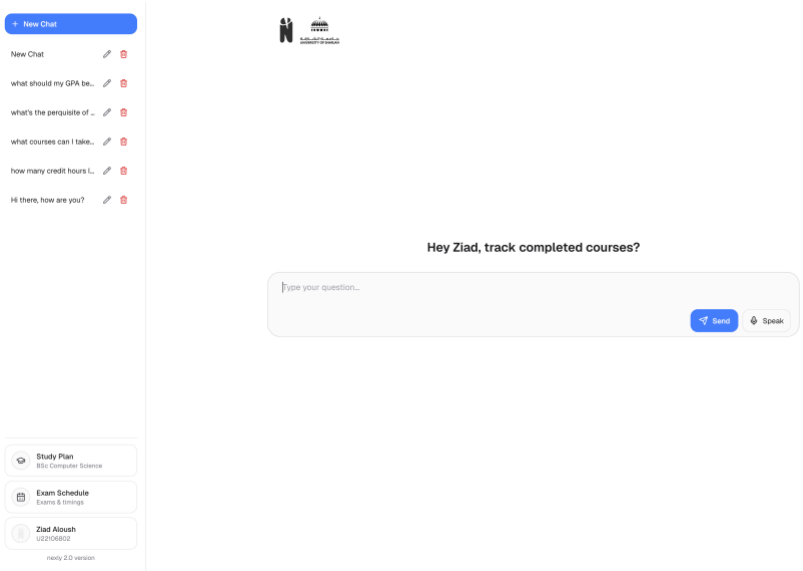

Figure 10: Nexly's UI light mode, prioritizes readability for extended academic content.

[6] *Perera, P. (2022, January 4). Visual explanation and comparison of CSR, SSR, SSG and ISR [Blog post]. DEV Community. https://dev.to/pahanperera/visual-explanation-and-comparison-of-csr-ssr-ssg-and-isr-34ea*

[7] *Yang, A., Goh, C. H., & Lim, J. Y. (2023). The research into dark mode: A systematic review using a two-stage approach and the S-O-R framework. Social Space: International Journal of Social Sciences and Humanities, 23(4).*

Information density calibration proved essential to Nexly's effectiveness. Early prototypes borrowed heavily from consumer messaging applications, presenting one question and one answer per screen with generous whitespace and minimal contextual information. User testing revealed that students found this approach frustrating when comparing multiple course options, reviewing prerequisite chains, or cross-referencing degree requirements. The current design strikes a balance: each chat message displays with sufficient vertical breathing room to remain scannable, but conversation history remains visible at all times, and the sidebar provides persistent access to navigation controls, user profile, and quick action buttons.

The typography system employs a carefully tuned scale based on Material Design's 4dp grid system, ensuring that heading hierarchies remain perceptually distinct while body text maintains optimal character counts per line (50-75 characters). Font selection prioritized Geist Sans and Geist Mono for their technical legibility, open-source availability, and comprehensive Unicode coverage, critical for displaying Arabic script in the future alongside Latin characters without visual jarring.

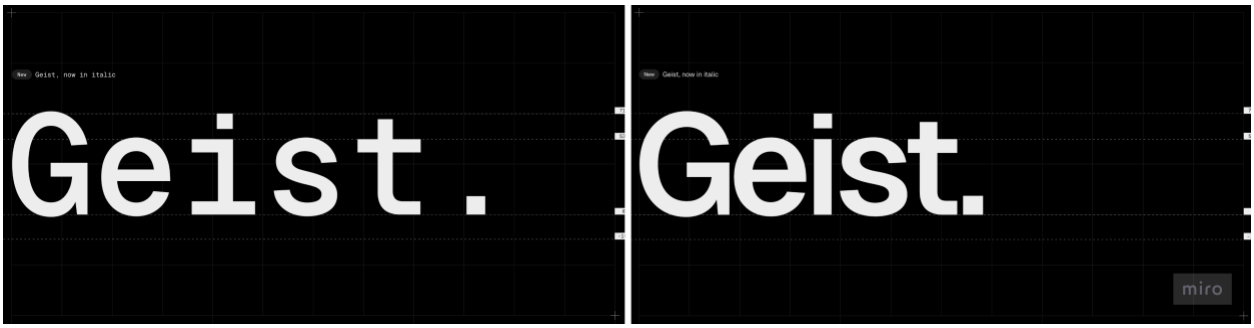

Figure 11: Comparison of Geist Mono (left) and Geist Sans (right) demonstrating the typographic architecture of the font.[8]

Interactive elements adhere to minimum touch target sizes documented in Apple's Human Interface Guidelines (44×44 points)[9] and Google's Material Design (48×48 density-independent pixels)[10], preventing accidental taps on mobile devices. This consideration proves particularly important for Nexly's responsive sidebar, which must accommodate both mouse-driven desktop interactions and thumb-driven mobile gestures without sacrificing precision or predictability.

Tailwind CSS for Consistency and Rapid Iteration

The adoption of Tailwind CSS as Nexly's primary styling framework emerged from practical experience with component libraries during the junior project phase. Previous attempts using Chakra UI and Material-UI revealed a common pattern: these opinionated frameworks accelerated initial development but imposed increasing friction as design requirements diverged from default themes. Customizing button corner radii, adjusting spacing scales, or introducing brand-specific color palettes required navigating deeply nested theme configuration objects, overriding CSS

[8] *Vercel. (n.d.). Geist Sans & Geist Mono [Web page]. https://vercel.com/font*

[9] A*pple Developer Documentation. (n.d.). Buttons — human interface guidelines. Apple Inc. Retrieved from https://developer.apple.com/design/human-interface-guidelines/buttons*

[10] G*oogle Material Design. (n.d.). Accessibility – touch target size. Google LLC. Retrieved from https://m3.material.io/foundations/designing/structure*

specificity battles, and occasionally resorting to `!important` declarations that undermined maintainability.

Tailwind's utility-first approach inverts this paradigm. Rather than fighting against a predetermined design system, developers compose interfaces from atomic, single-purpose CSS classes that map directly to style properties. This creates several advantages for Nexly's development workflow. First, design changes propagate instantly without requiring mental translation between design specification and component API documentation. When user testing indicated that the chat input field needed increased vertical padding for improved tap ergonomics, the fix required changing `py-2` to `py-3` in a single location rather than hunting through theme configuration files.

Second, Tailwind's constraint-based system prevents style proliferation. The framework's default spacing scale (based on 0.25rem increments) and color palette (organized in 50-950 shades per hue) impose just enough structure to maintain visual consistency while remaining flexible enough to accommodate unique requirements. This "guardrails not roadblocks" philosophy proved essential during rapid iteration cycles when experimenting with different layouts for the study plan visualization page.

Third, Tailwind CSS version 4.x introduces native cascade layers and CSS nesting support, eliminating the need for additional PostCSS plugins while improving specificity management. The framework's JIT (Just-In-Time) compiler generates only the CSS classes actually used in the application, resulting in production bundles under 10KB, negligible compared to the JavaScript payload required for UI interactivity.

Design System Reqs.	*Traditional CSS*	*Component Libraries*	*Tailwind CSS*
Custom color palette	Manual CSS custom properties	Theme configuration JSON	Tailwind config + utility classes
Responsive breakpoints	Media query repetition	Inconsistent component APIs	Responsive prefix system (md:, lg:)
Dark mode support	Duplicate stylesheets	Theme provider complexity	dark: variant prefix
Production bundle size	Varies widely	100-300 KB typical	<10 KB with tree-shaking
Learning curve	Medium (CSS knowledge)	High (framework-specific)	Low (HTML/CSS fundamentals)
Design consistency enforcement	Discipline-dependent	Framework-imposed	Constraint-based scale

Table 3: Comparative analysis of styling approaches for Nexly's frontend implementation.

Shadcn for Accessible, Stable Component Primitives

While Tailwind handles visual styling, Shadcn provides the behavioral foundation for interactive components. Unlike traditional component libraries that distribute pre-built React components via npm packages, Shadcn operates as a collection of copy-pasteable component recipes built atop Radix UI primitives. This architectural decision yields several strategic advantages for Nexly's long-term maintainability.

First, Shadcn components become first-class citizens in Nexly's codebase rather than opaque third-party dependencies. When accessibility testing revealed that the study plan dialog component needed improved keyboard navigation flow, the team could directly modify the dialog implementation in `src/components/ui/dialog.tsx` without waiting for upstream maintainers to accept pull requests or release new versions. This ownership eliminates dependency risk while preserving the benefits of expert-designed component patterns.

Second, Radix UI's underlying primitives prioritize Web Accessibility Initiative - Accessible Rich Internet Applications (WAI-ARIA) compliance over visual appearance. Every Shadcn component ships with proper ARIA attributes, keyboard navigation handlers, focus management logic, and screen reader announcements built in. This foundation proves essential for ensuring Nexly remains usable by students with disabilities, satisfying both ethical obligations and legal requirements under regional accessibility mandates.

Third, the copy-paste model encourages progressive enhancement rather than feature bloat. Nexly's implementation includes only the components actually required for its user interface, dialog boxes, dropdown menus, buttons, form inputs, and tooltips, rather than bundling an entire design system with unused date pickers, carousels, and data table implementations. This minimalism directly translates to faster page loads and simpler mental models for developers joining the project.

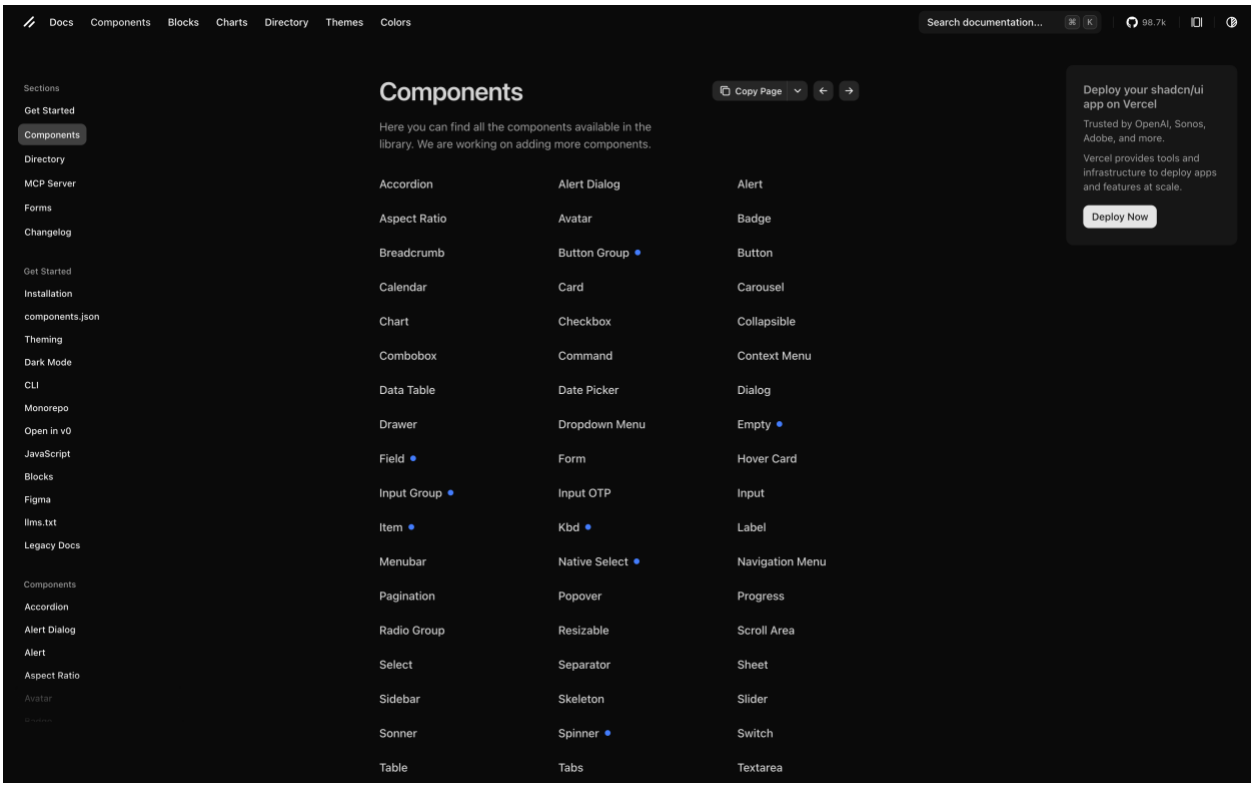

Figure 12: Shadcn components provide battle-tested accessibility patterns while remaining visually customizable through Tailwind classes.[11]

[11] *shadcn. (n.d.). Components – shadcn/ui. Retrieved from https://ui.shadcn.com/docs/components*

State Management Decisions: React Query and Zustand

State management architecture significantly influences both development velocity and runtime reliability. Nexly employs a hybrid approach that distinguishes between server state and client state, assigning each to specialized tools rather than forcing both through a single state management paradigm.

React Query (TanStack Query v5) handles all server state, data fetched from Supabase, responses from the RAG API endpoint, and study plan information retrieved from the backend. This library's declarative data synchronization model eliminates entire categories of bugs that plagued earlier implementations. Consider the exam schedule page: when a student navigates to this view, React Query automatically fetches exam data, caches it locally, displays it to the user, and continues polling the server in the background to detect updates. If the network connection fails, stale data remains visible with a visual indicator, preserving usability. If new exam information becomes available, React Query invalidates its cache and refetches, ensuring freshness without manual intervention.

The library's optimistic update patterns prove particularly valuable for chat interactions. When a student sends a message, the UI immediately displays it as "sending" while React Query posts the content to the server. If the request succeeds, the message transitions to "sent" status. If it fails, the message reverts to "failed" with a retry button, and the local state remains consistent with the server's truth. This level of sophistication would require hundreds of lines of custom state management code using lower-level primitives like `useReducer` or `useState`.

Zustand manages client-side state that exists purely in the browser, UI preferences like sidebar visibility, theme selection (light/dark), and the currently active chat thread. Zustand's philosophy of minimalism aligns well with Nexly's needs: rather than introducing complex concepts like actions, reducers, and middleware, it provides a simple `create` function that returns a React hook. The entire store for Nexly's UI preferences occupies fewer than 50 lines of code, yet provides type-safe access, automatic re-renders when state changes, and persistence to localStorage through a tiny plugin.

Rationale Behind Abandoning the Mobile Application

The decision to deprecate the standalone Flutter mobile application and consolidate around responsive web design required weighing multiple competing factors: development resources, feature parity challenges, deployment complexity, and user behavior patterns observed during the junior project phase.

Development resource constraints proved determinative. Maintaining parallel implementations of authentication flows, chat interfaces, study plan visualizations, and settings screens across two codebases consumed unsustainable amounts of time. Every new feature required designing twice, implementing twice, testing twice, and debugging twice. The study plan graph visualization serves as a concrete example: the React Flow library on the web provided rich interactivity with pan, zoom, and hierarchical layouts, while the Flutter ecosystem lacked equivalent maturity in graph visualization packages. Replicating this functionality in Flutter would have required either building a custom solution from scratch or accepting a significantly degraded user experience on mobile.

Feature parity challenges extended beyond component availability. State synchronization between web and mobile versions required careful coordination to ensure that conversation history, user

preferences, and study plan progress remained consistent across devices. This synchronization logic added complexity to the backend API design and introduced new failure modes when network conditions caused out-of-order updates or partial synchronization.

Modern responsive web design patterns, however, largely eliminate the traditional arguments favoring native mobile applications. Progressive Web App (PWA) capabilities allow Nexly to be installed on student home screens, receive push notifications, and function offline, capabilities once exclusive to native apps. The Performance API and React's Suspense boundaries enable sophisticated loading state management that rivals native transitions. CSS Grid and Flexbox, combined with Tailwind's responsive utilities, adapt layouts seamlessly across viewport sizes from 320px smartphones to 4K desktop monitors.

User behavior analysis from the junior project phase revealed that students primarily accessed Nexly during three scenarios: desktop research sessions while planning course selections, brief lookup queries between classes on mobile browsers, and advisor meeting preparations where larger screens facilitated side-by-side comparison of degree requirements and course catalogs. Notably, mobile usage patterns overwhelmingly favored mobile browsers over the installed Flutter application, suggesting that app installation friction outweighed perceived benefits.

The deprecation decision also considered deployment complexity. Distributing the Flutter app required navigating Apple App Store and Google Play Store review processes, managing provisioning profiles and signing certificates, and coordinating release schedules across platforms. Web deployment through Vercel, by contrast, occurs automatically on every merge to the main branch, with preview deployments for pull requests enabling rapid feedback cycles. This reduced deployment friction accelerates iteration velocity and lowers the barrier for contributors.

Finally, the consolidated approach better positions Nexly for future institutional integration. University IT departments typically maintain strict policies around approved software installations on campus computers, but universally support modern web browsers. A web-only architecture ensures Nexly remains accessible in library kiosks, classroom presentation systems, and shared lab workstations without requiring IT intervention for installation or updates.

Chatper 3: System Architecture & Implementation

Nexly's High Level System Architecture

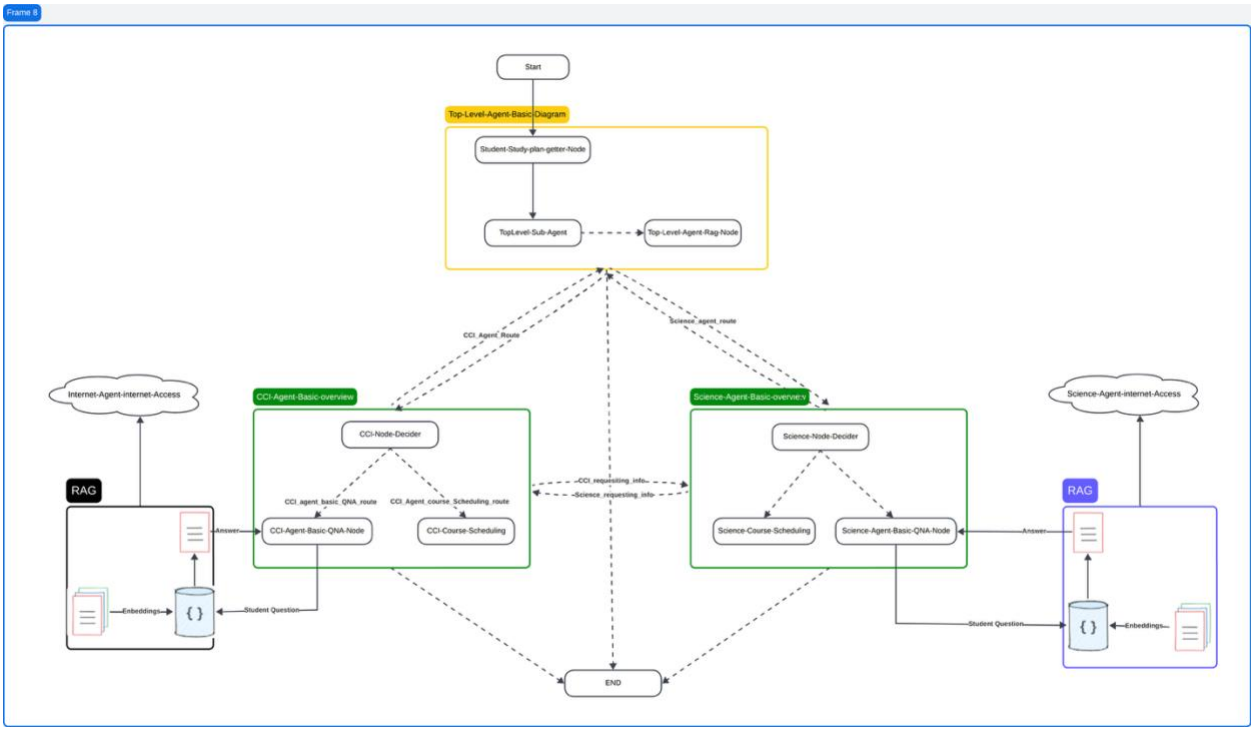

Figure 13: Multi-agent system architecture of Nexly showing the hierarchical relationship between the Top-Level Agent and specialized college agents (CCI Agent and Science Agent). Each agent integrates with RAG components for knowledge retrieval and contains conversation scheduling capabilities for automated course planning.

The Nexly academic advising system is architected around a modular, three-layered design that facilitates scalability, maintainability, and specialization of academic advisory services. This architectural approach enables the system to handle diverse query types while maintaining efficient resource utilization and clear separation of concerns. The three primary architectural layers comprise the Client/API Gateway Layer, the Core Agentic Processing Layer, and the Data/Tool Integration Layer.

The **Client/API Gateway Layer** serves as the primary interface between end users and the intelligent backend system. Built on Next.js framework with server-side rendering capabilities, this layer handles all client-facing interactions, including the responsive web interface, authentication flows through Supabase Auth, and session management. The Next.js API routes act as a secure gateway that validates incoming requests, manages CORS policies, and forwards appropriately formatted queries to the backend processing infrastructure. This architectural decision provides several advantages: it decouples the user interface from the core intelligence layer, enables independent scaling of frontend and backend components, and creates a clear security boundary where authentication and authorization checks can be enforced before queries reach the computational core. Furthermore, the Next.js framework's support for Progressive Web Application features ensures that students can access Nexly from any device with consistent

performance, whether on desktop computers in university libraries or mobile devices between classes.

The **Core Agentic Processing Layer** represents the intellectual heart of Nexly's architecture. This layer orchestrates all decision-making processes, query routing, and response generation through a sophisticated multi-agent system implemented using the LangGraph framework. LangGraph provides a graph-based workflow management system specifically designed for agentic applications, allowing the definition of complex state machines where each node represents an agent or processing step, and edges represent the flow of information between these components. At the apex of this layer sits the Top-Level Agent, which performs initial query classification to determine whether an incoming student question requires general conversational responses, university-wide policy information, or specialized academic advising from department-specific agents. When specialized knowledge is required, the Top-Level Agent employs intelligent routing logic to delegate queries to one of two specialized sub-agents: the CCI Agent, which handles all matters related to the College of Computing and Informatics, or the Science Agent, which addresses questions pertaining to the College of Science. These specialized agents maintain domain-specific knowledge bases and are fine-tuned to understand the unique requirements, terminology, and academic structures of their respective colleges. The multi-agent design enables parallel processing of complex queries that might require information from multiple domains, as agents can communicate laterally to aggregate comprehensive responses. The entire processing layer is powered by the Meta-Llama 4 Scout large language model accessed through Groq's high-performance API, which provides the natural language understanding and generation capabilities necessary for context-aware academic advising.

The **Data/Tool Integration Layer** provides the foundational data infrastructure that supports informed decision-making across all agents. This layer encompasses multiple data sources and retrieval mechanisms that collectively ensure agents have access to current, accurate, and comprehensive information. The Supabase PostgreSQL database serves as the system's primary structured data store, housing critical information including complete study plans for all university programs, current course offerings with detailed scheduling information, prerequisite mappings, and student academic records. The database schema is designed to support efficient querying patterns required by the scheduling algorithms, with appropriate indexes on frequently accessed fields such as course codes, prerequisite relationships, and offering terms. Complementing the structured database is a vector-based retrieval system implemented using FAISS, which indexes and enables semantic search across unstructured documents including course syllabi, university policy handbooks, academic regulations, and general information resources. The vector store uses dense embeddings generated by the all-MiniLM-L6-v2 sentence transformer model, which converts textual content into high-dimensional numerical representations that capture semantic meaning. This allows the system to retrieve contextually relevant passages even when the exact keywords from a student's query do not appear in the source documents. The integration layer also includes real-time data fetching capabilities through web scraping modules that can pull current information from the university website when needed, ensuring that responses reflect the most recent policies and announcements. All components in this layer are accessed through well-defined interfaces that abstract the underlying data complexity, allowing agents to retrieve information through simple function calls without needing to manage database connections, vector similarity computations, or document parsing logic.

The interconnection between these three layers follows a clean architectural pattern where each layer communicates exclusively with its adjacent layers. Client requests flow through the API

Gateway to the Core Processing Layer, which in turn queries the Data Layer as needed to construct responses. This unidirectional flow prevents circular dependencies and maintains clear boundaries of responsibility. The Core Processing Layer acts as an intelligent orchestration engine that decides when to consult structured databases versus unstructured document stores, when to combine information from multiple sources, and how to synthesize retrieved information into coherent, contextually appropriate responses. The modular nature of this architecture enables independent evolution and testing of components; for instance, the vector store implementation could be migrated from FAISS to Pinecone or Weaviate without requiring changes to the agent logic, and new specialized agents can be added to handle additional colleges or departments without modifying existing agent implementations.

Retrieval-Augmented Generation (RAG) Architecture

The Retrieval-Augmented Generation architecture forms the cornerstone of Nexly's ability to provide accurate, grounded, and source-attributable responses to student queries. RAG represents a hybrid approach to question-answering that combines the strengths of information retrieval systems with the natural language generation capabilities of large language models. This architectural pattern addresses a fundamental limitation of pure large language model approaches: while LLMs possess broad general knowledge encoded in their parameters during training, they cannot inherently access or reason about specific institutional documents, current policies, or real-time information without explicit inclusion in their training data. RAG solves this problem by treating the LLM as a reasoning engine that synthesizes information dynamically retrieved from authoritative sources, rather than relying solely on its parametric knowledge.

The RAG pipeline in Nexly operates through a carefully orchestrated sequence of five distinct stages: query processing, document retrieval, context ranking, prompt augmentation, and response generation. When a student submits a question through the Nexly interface, the query processing stage first transforms the natural language input into a format optimized for retrieval. This transformation involves several sub-processes: the query is normalized to handle variations in capitalization and punctuation, stop words that carry minimal semantic meaning are identified (though not always removed, as academic queries sometimes depend on these words for precise meaning), and the query is encoded into a dense vector representation using the same embedding model that was used to index the document corpus. The choice of embedding model is critical; Nexly employs the all-MiniLM-L6-v2 model specifically because it has been pre-trained on academic and question-answering datasets, making it particularly effective at capturing the semantic nuances of educational queries.

The document retrieval stage leverages this query embedding to search through Nexly's knowledge base using FAISS (Facebook AI Similarity Search), a highly optimized library for similarity search in high-dimensional vector spaces. The knowledge base contains embedded representations of thousands of document chunks, where each chunk represents a semantically coherent passage from source materials such as course syllabi, academic handbooks, university regulations, and general information documents. When source documents are ingested into the system, they undergo a preprocessing pipeline that includes PDF text extraction using PyMuPDF for scanned documents, HTML parsing using BeautifulSoup for web-scraped content, and DOCX parsing for policy documents. These documents are then segmented into chunks of approximately five hundred tokens with fifty-token overlaps between adjacent chunks; this overlap ensures that important information spanning chunk boundaries is not lost. Each chunk is embedded and stored in the FAISS index alongside metadata indicating its source document, page number, and document type.

During retrieval, FAISS performs an approximate k-nearest neighbors search in the embedding space to identify the top-k most semantically similar document chunks to the query embedding. This retrieval operation is computationally efficient, capable of searching through tens of thousands of embeddings in mere milliseconds, which is essential for maintaining responsive user interactions.

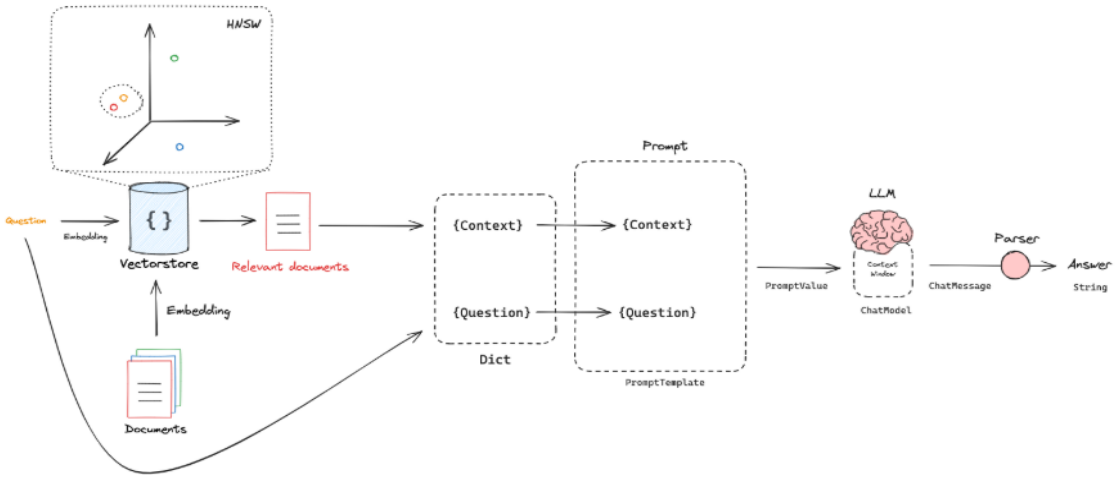

Figure 14: Architecture Of The Retrieval-augmented Generation (Rag) Model Used In Nexly. The System Retrieves Relevant Documents Via Vector Search, Ranks Them, And Combines The Retrieved Context With The User's Query To Generate Accurate, Grounded Responses Through a Large Language Model (LLM).

The context ranking stage refines the initial retrieval results by applying additional filtering and ordering criteria beyond pure semantic similarity. While FAISS retrieval provides candidates based on vector similarity, these candidates are subsequently re-ranked using cross-encoder models that provide more nuanced relevance scores by directly comparing the query against each candidate passage. Additionally, this stage implements source diversity constraints to ensure that the final context presented to the language model represents multiple perspectives or complementary information rather than redundant passages from the same source. Document recency can also be factored into ranking, giving preference to more recently updated policy documents when multiple relevant sources exist. The system typically selects the top three to five passages after this re-ranking process, which provides sufficient context for the language model to generate informed responses while remaining within the context window limitations of the model. Prompt augmentation represents the critical integration point between retrieval and generation. At this stage, the system constructs a carefully formatted prompt that includes three essential components: the retrieved context passages with their source attribution, the original student query, and instruction framing that guides the language model's behavior. The instruction framing is particularly important for academic advising applications; it includes directives such as "Answer based solely on the provided context," "Cite specific sources when making claims," "Admit uncertainty if the context does not contain relevant information," and "Maintain a helpful but professional tone appropriate for academic advising." These instructions are designed to mitigate common failure modes of large language models, including hallucination of factually incorrect information, overconfidence in uncertain scenarios, and generation of inappropriate or casual responses. The formatting of retrieved passages in the prompt follows a structured template that

clearly delineates each passage with headers indicating its source document and relevance score, enabling the model to understand provenance and weigh information accordingly.

The response generation stage submits this augmented prompt to the Meta-Llama 4 Scout model through Groq's inference API. Meta-Llama 4 Scout was selected for Nexly based on several factors: its strong performance on instruction-following tasks, its ability to handle multi-turn conversations while maintaining context coherence, its support for structured output generation when needed for scheduling tasks, and its availability through Groq's platform which provides significantly faster inference speeds compared to self-hosted alternatives. Groq's tensor streaming processor architecture enables low-latency response generation, typically producing complete answers in two to four seconds, which is crucial for maintaining an interactive conversational experience. The model receives the prompt, processes it through its attention mechanisms, and generates a response token-by-token. The generation process employs sampling strategies with temperature controls to balance between consistency and creativity; for factual academic advising questions, a lower temperature of zero point three encourages more deterministic, fact-focused responses, while for scheduling or brainstorming tasks, a slightly higher temperature allows for more diverse suggestions.

The RAG architecture also incorporates fallback mechanisms to handle edge cases and ensure robust operation. When document retrieval yields no passages exceeding a minimum relevance threshold, indicating that the knowledge base does not contain information pertinent to the query, the system activates a graceful fallback mode rather than attempting to generate an answer from insufficient context. In this mode, the language model is prompted to acknowledge the information gap explicitly and, when appropriate, direct the student to human advisors or official university resources where answers can be obtained. This approach prioritizes accuracy and user trust over coverage, recognizing that confident wrong answers are more harmful in an academic advising context than admissions of uncertainty. Similarly, when the language model's output indicates low confidence through linguistic markers such as frequent hedging or contradictory statements, the system can flag these responses for human review before delivery.

Multi-Agent System Design and Inter-Agent Communication

The multi-agent architecture implemented in Nexly represents a sophisticated approach to decomposing the complex domain of academic advising into specialized, manageable sub-domains, each handled by a dedicated intelligent agent. This design philosophy draws inspiration from organizational structures in human institutions, where complex problems are addressed by teams of specialists who collaborate and share information to reach comprehensive solutions. The multi-agent paradigm provides several advantages over monolithic single-agent systems: it enables domain specialization where each agent can be optimized for its specific area of expertise, it supports parallel processing of multi-faceted queries, it facilitates incremental system expansion by allowing new agents to be added without restructuring existing components, and it improves system interpretability by making the decision-making process more transparent through explicit routing and delegation steps.

At the architectural apex sits the **Top-Level Agent**, which functions as the system's orchestrator and initial point of contact for all incoming queries. This agent's primary responsibility is query classification and routing, determining whether a student's question requires general conversational engagement, university-wide information retrieval, or specialized academic advising from one of the college-specific agents. The classification process employs a two-stage decision framework: first, the agent uses the language model to analyze the semantic content of

the query and identify key indicators such as mentions of specific courses, degree programs, scheduling terms, or college names; second, it applies rule-based heuristics to handle unambiguous cases where explicit keywords provide clear classification signals. For instance, a query containing "CS3302" or "Computer Science" is immediately routed to the CCI Agent, while questions about "CHEM1101" or "Chemistry major" are directed to the Science Agent. However, many queries are ambiguous or multi-faceted, such as "What math courses do I need for a CS degree?" which involves both colleges. In these cases, the Top-Level Agent takes responsibility for coordinating information gathering from multiple specialized agents and synthesizing their responses into a unified answer. The Top-Level Agent also maintains conversational context across multiple turns of interaction, tracking the history of queries and responses within a session to enable follow-up questions that depend on previously established context. This contextual awareness is implemented through a state management system that persists relevant information in the UOS_AgentState data structure, which is passed between agents and updated as the conversation progresses.

The **CCI Agent** specializes in all matters pertaining to the College of Computing and Informatics, encompassing six distinct departments: Computer Science, Computer Engineering, Cybersecurity Engineering, Information Technology Multimedia, Business Information Systems, and Biomedical Informatics. This agent possesses deep knowledge of the academic structures, course offerings, prerequisite chains, and curricular requirements specific to these technology-focused disciplines. When the CCI Agent receives a routed query, it first determines the specific nature of the information request, whether it concerns course content and descriptions, prerequisite requirements, study plan progression, or scheduling. For questions about course content, the agent consults its dedicated RAG subsystem, which maintains a vector store of course syllabi and descriptions for all CCI courses. This specialized knowledge base enables the agent to answer detailed questions about course topics, learning outcomes, assessment methods, and textbook requirements. For prerequisite-related queries, the agent accesses structured data from the Supabase database where prerequisite relationships are explicitly encoded in a graph structure, allowing efficient traversal to determine prerequisite chains or identify courses for which a student is qualified based on completed coursework. The CCI Agent's most sophisticated capability is automated course scheduling, which involves retrieving a student's study plan from the database, identifying remaining required and elective courses, filtering these against current semester offerings, checking prerequisite satisfaction, detecting time conflicts, considering campus location constraints for female and male students, respecting credit hour load limits, and generating an optimized schedule that balances workload across the semester. This scheduling function implements a constraint satisfaction algorithm that searches through the combinatorial space of possible course combinations to find schedules that satisfy all hard constraints while optimizing for soft preferences such as avoiding early morning classes or clustering courses on fewer days.

The **Science Agent** mirrors the CCI Agent's structure and capabilities but specializes in the College of Science, covering the departments of Mathematics, Chemistry, Biotechnology, Applied Physics, and Petroleum Geosciences and Remote Sensing. Like the CCI Agent, it maintains its own domain-specific RAG system populated with science course materials, laboratory guides, and department policies. The Science Agent handles queries about science major requirements, course content in scientific disciplines, prerequisite chains for science courses, and generates schedules for students in science programs. An important aspect of the Science Agent's design is its awareness of cross-college requirements; many science students must take courses offered by CCI (for example, programming courses required for Applied Physics students), and the agent is

equipped to recognize these cross-listings and coordinate with the CCI Agent to retrieve relevant information about courses outside its primary domain.

Agent	*Primary Domain*	*Key Responsibilities*	*Knowledge Sources*
Top-Level Agent	General University Information & Routing	Query classification, agent routing, general Q&A, response synthesis	University handbooks, general policies, academic calendar
CCI Agent	College of Computing and Informatics	CS/CE/CYE/ITM/BIS/BMI advising, CCI course scheduling, prerequisite checking	CCI course syllabi, CCI study plans, CCI course offerings
Science Agent	College of Science	Math/Chem/Bio/Physics/Geosciences advising, Science course scheduling	Science course materials, Science study plans, Science course offerings

Table 4: Agent Specialization and Responsibilities

The inter-agent communication protocol enables these specialized agents to collaborate on complex queries that require knowledge from multiple domains. When an agent determines that it needs information outside its area of expertise to fully answer a query, it can issue a request to another agent through the LangGraph state management system. For example, if the CCI Agent receives a question about general university registration deadlines, it recognizes this as outside its domain-specific knowledge and forwards a sub-query to the Top-Level Agent, which has access to university-wide policy documents. Similarly, when generating schedules for students with double majors or minors spanning multiple colleges, agents must coordinate to ensure that course selections satisfy requirements from both programs without creating time conflicts. The communication protocol includes routing flags in the shared state that prevent infinite loops; when an agent forwards a request to another agent, it sets a flag indicating that the response should not be routed back, ensuring that requests are resolved by the appropriate authority rather than bouncing indefinitely between agents.

The state management system that coordinates these agents is implemented using LangGraph's graph-based workflow engine, where each agent corresponds to a node in a computational graph, and edges represent possible transitions between agents based on routing decisions or information requests. The graph structure encodes the permissible communication patterns, ensuring that information flows follow defined protocols. Each node in the graph is associated with a state transformation function that updates the UOS_AgentState based on the agent's processing; for instance, when the CCI Agent retrieves course information, it adds this information to the state's knowledge context so that subsequent agents or the final response synthesis step can access it. This state-centric design provides a complete audit trail of the decision-making process, as the final state object contains a record of which agents were consulted, what information each retrieved, and how routing decisions were made. This transparency is valuable both for debugging during development and for building user trust, as the system can explain why it provided a particular answer or recommendation by exposing the chain of reasoning and information sources that led to that conclusion.

The multi-agent system also incorporates specialization in tool usage, where different agents have access to different external tools and data sources appropriate to their domains. The CCI Agent,

for instance, has access to tools for parsing study plans specific to CCI programs, querying the CCI course offerings table in Supabase, and retrieving CCI-specific policy documents. The Science Agent similarly has access to Science-specific tools. This principle of least privilege ensures that each agent only accesses the resources necessary for its function, which improves security, reduces the complexity of individual agent implementations, and makes it easier to manage permissions and access controls to sensitive student data. Tool invocations are instrumented through the LangGraph framework using a standardized interface where each tool is defined as a callable function with typed inputs and outputs; the agent indicates which tool it wants to use by generating a structured command, and the framework handles the actual tool execution and result return.

Looking forward, the multi-agent architecture is designed to support future expansion as Nexly's scope grows to cover additional colleges and departments within the University of Sharjah. Adding a new specialized agent for, say, the College of Business Administration would involve defining its domain scope, populating a domain-specific knowledge base, implementing its tool set for accessing business program data, and registering it with the Top-Level Agent's routing logic. The modular nature of the current design means that this expansion does not require modifying the existing CCI or Science agents; they continue to operate independently, and the new agent integrates through the same communication protocols. This scalability was a key consideration in the architectural design, recognizing that comprehensive university-wide coverage will require supporting many more academic units than are currently implemented. The graph-based workflow management provided by LangGraph naturally accommodates these expansions, as new nodes and edges can be added to the computational graph without disrupting existing pathways.

Tool/Data Source	*CCI Agent*	*Science Agent*	*Top-Level Agent*
CCI Study Plans Database	✓	✗	✓
Science Study Plans Database	✗	✓	✓
CCI Course Offerings	✓	✗	✓
Science Course Offerings	✗	✓	✓
University Policies RAG	✓	✓	✓
CCI Syllabi RAG	✓	✗	✗
Science Syllabi RAG	✗	✓	✗
Course Scheduling Algorithm	✓	✓	✗

Table 5: Agent-Specific Tools and Data Access

Stage	*Technology / Function*	*Implementation Details*	*Rationale*
1. Chunker	pdfplumber / Custom Processors	Top-Level Data: Minimum chunk size of 250 words for high-level documents (e.g., handbooks). Website Q&A: Chunk size of 500 words for general college website content.	Ensures semantic coherence within chunks while optimizing for token limits. Longer chunks are used for broader Q&A data.
2. Embedder	HuggingFaceEmbeddings	Model: sentence-transformers/all-MiniLM-L6-v2	Chosen for strong semantic representation and efficient computational footprint, suitable for a low-latency environment.
3. Vector Store	FAISS (Facebook AI Similarity Search)	In-memory indexing with persistence for fast initialization.	Provides highly efficient k-Nearest Neighbors (k-NN) search performance, critical for quick context retrieval.
4. Retriever	VectorStoreRetriever	Configuration: search_kwargs={"k":10}.	Retrieves the Top 10 semantically most relevant document chunks based on the user's query vector.
5. Context Filtering	Custom Truncation Logic	Limits the aggregated retrieved text context to 15,000 words (or 15,000 tokens, depending on the implementation checkpoint) before passing to the LLM.	Prevents LLM context window overflow and reduces the chance of noise interference from marginally relevant chunks.
6. LLM Generator	Primary Generator LLM	Ingests the query and the filtered context to produce the final, grounded answer and the confidence score.	The final synthesis stage. Prompt engineering dictates the structured JSON output format.

Table 6: Nexly's pipeline component details table

Database Architecture: A Lightweight Academic Data Repository

Nexly's backend relies on a PostgreSQL database hosted through Supabase that functions as a lightweight Student Information System (SIS), storing the institutional knowledge necessary to power intelligent academic advising. Rather than integrating directly with the University of Sharjah's official SIS, which would require extensive institutional permissions and API access, this database maintains a curated subset of public academic information that the frontend can access through secure API endpoints.

The database schema organizes information across three primary domains: course catalogs, degree requirements, and student profiles. Course tables partition offerings by college, `courses_from_computerscience`, `courses_from_medical`, `courses_from_science`, and `courses_from_othercollege`, each storing comprehensive metadata including course codes, titles, credit hours, instructors, meeting times, and campus locations. This structure enables the system to answer questions about course availability, scheduling conflicts, and instructor assignments without requiring real-time integration with registration systems.

Study plan tables encode the prerequisite graphs and semester progressions for multiple degree programs. Tables like `study_plan_bsc_computer_science`, `study_plan_bsc_biomedical_informatics`, and `study_plan_bsc_cybersecurity` capture the course sequences students must follow to graduate, storing prerequisite relationships as PostgreSQL arrays that the frontend's study plan visualization page can traverse to construct interactive dependency graphs. Separate elective tables (`study_plan_bsc_computer_science_elective_course`, etc.) provide flexibility for students to explore optional coursework within their degree requirements.

The `students` table maintains authenticated user profiles, linking university IDs to completed course histories stored as text arrays. This design choice, using arrays rather than normalized junction tables, trades some relational purity for query simplicity, as the frontend can retrieve a student's entire academic history in a single database round-trip. When a student logs in and navigates to their profile page, the application fetches their record, extracts the `completed_courses` array, and uses this information to highlight completed courses in the study plan graph and filter out prerequisites they've already satisfied.

Examination data resides in the `u22106802-exams` table, capturing upcoming exam schedules with rich metadata including locations, dates, times, and AI-generated difficulty estimates. The `hardness` and `revision_estimate_hours` columns contain synthetic predictions produced during data ingestion, providing students with rough guidance on exam preparation requirements while clearly marking this information as AI-generated rather than official institutional data.

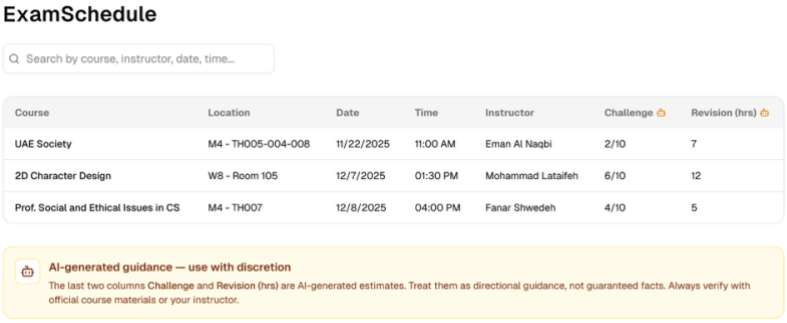

Figure 15: UI Screenshot of Nexly's web app showing a student's exam schedule for the semster, retreived form the database directly.

API-Mediated Database Access

The frontend never queries the database directly. All data access flows through Next.js API routes that enforce authentication, validate inputs, and transform raw database responses into frontend-friendly formats. The `/api/study-plan` endpoint, for instance, fetches study plan rows from the appropriate degree table, joins them with the student's completion status from their profile, and returns a unified structure indicating which courses are required, which are completed, and what prerequisites remain unsatisfied. This server-side transformation logic centralizes business rules while keeping the frontend focused purely on presentation concerns.

```
// Example API route pattern from src/app/api/study-plan/route.ts
export async function GET() {
  const supabase = await createClient();
  const { data: userData } = await supabase.auth.getUser();
  const userId = userData?.user?.id;

  // Fetch student's completed courses
  const { data: student } = await supabase
    .from("students")
    .select("completed_courses")
    .eq("id", userId)
    .maybeSingle();

  // Fetch study plan for their degree program
  const { data: courses } = await supabase
    .from("study_plan_bsc_computer_science")
    .select("*")
    .order("id");

  // Merge and transform data for frontend consumption
  return NextResponse.json({
    data: courses.map(course => ({
      ...course,
      completed:
student?.completed_courses?.includes(course.course_code)
    }));
  });
}
```

Figure 16: Example API route pattern from src/app/api/study-plan/route.ts

The `/api/ask` endpoint demonstrates similar patterns for the RAG pipeline, packaging student context alongside their query before forwarding to the AI backend. This middleware layer extracts the student's UID, completed courses, and demographic information from the database, constructs a structured payload, and sends it to the RAG service, ensuring personalized responses without exposing database credentials or structure to client browsers.

Authentication flows through `/api/auth/password`, which proxies requests to Supabase's authentication service while mapping error codes to user-friendly messages. This indirection provides a stable frontend API surface even if the underlying authentication provider changes, and it allows server-side logging and rate limiting before authentication attempts reach Supabase.

Supabase Row-Level Security

While the database architecture itself remains simple, Supabase's Row-Level Security (RLS) features provide defense-in-depth protection for student data. RLS policies defined at the

database layer ensure that authenticated users can access only their own profiles and exam schedules, even if API routes somehow failed to enforce authorization checks. These policies operate transparently, the frontend code queries tables as normal, but PostgreSQL automatically filters results to match the authenticated user's identity. This approach reduces the cognitive load on API developers while maintaining strong security boundaries.

Rationale for Lightweight Design

The decision to maintain a curated database rather than integrating with official university systems reflects practical constraints inherent to academic projects. Gaining API access to production SIS platforms requires navigating institutional bureaucracy, satisfying security audits, and adhering to strict data handling policies, timelines incompatible with semester-based project schedules. By populating the database with publicly available information from course catalogs and degree plans, Nexly achieves functional completeness for its core use cases while remaining deployable without institutional IT intervention.

This lightweight approach also simplifies development workflows. The team can seed the database with synthetic student profiles for testing, reset data between development iterations, and experiment with schema modifications without coordinating with external stakeholders. Should Nexly transition from prototype to production, the existing API boundary provides a clean integration point, backend routes can begin querying official SIS systems while the frontend continues using identical data structures and request patterns.

The database serves its purpose: providing a stable, queryable repository of institutional knowledge that enables intelligent academic advising while remaining maintainable by a small student team. It captures the essence of what students need, course requirements, degree progressions, and personalized academic tracking, without attempting to replicate the comprehensive functionality of enterprise systems built over decades.

Frontend System Design

The architecture of Nexly's frontend represents a deliberate synthesis of modern web development best practices, accessibility requirements, and performance constraints specific to academic environments. This section examines the strategic decisions underpinning technology selection, component organization, and data flow patterns that collectively enable Nexly to function as a responsive, reliable academic assistant.

Why Next.js Specifically: Strategic Justification

The selection of Next.js as Nexly's frontend framework emerges from careful evaluation of the specific challenges inherent to building an AI-powered academic advising system. These challenges, balancing static content serving with dynamic AI interactions, ensuring accessibility for users on varied devices and network conditions, and maintaining development velocity while adhering to institutional security requirements, map directly onto Next.js's architectural strengths.

Server-Side Rendering for Search Engine Optimization and Performance

Next.js's Server-Side Rendering (SSR) capabilities address a fundamental tension in modern web applications: the desire for rich client-side interactivity conflicts with the need for fast initial page

loads and comprehensive search engine indexability. Traditional Single-Page Applications (SPAs) built with Create React App or Vite ship minimal HTML, often just a `<div id="root"></div>` placeholder, then download JavaScript bundles, execute them, fetch data from APIs, and finally render content. This multi-step process imposes latency costs that compound on slower network connections or less powerful devices, creating precisely the accessibility barriers that academic institutions must avoid.

Nexly's study plan visualization page exemplifies SSR's benefits. When a student requests this page, Next.js executes the page component on the server, fetches course prerequisite data from Supabase, computes the graph layout, and returns fully-formed HTML containing both the visualization elements and their data attributes. The browser receives this complete document in the initial response, displaying a meaningful interface within milliseconds even before JavaScript loads. Once JavaScript does hydrate the page, the existing DOM elements become interactive, zoom controls activate, prerequisite highlighting responds to hover states, and navigation buttons begin functioning, but the critical content was visible and readable from the moment the server response arrived.

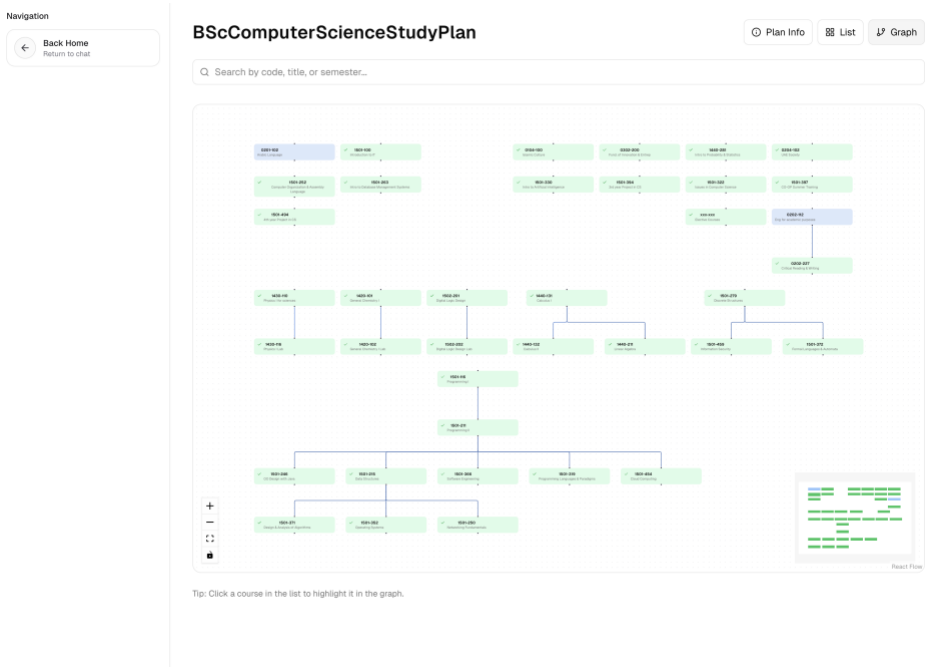

Figure 17: Screenshot of the Nexly's study plan page, showing the plan in a color-guided graph

This approach proves especially valuable in academic computing environments where shared workstations, library kiosks, and classroom presentation systems may have limited processing power or aggressive browser cache restrictions that prevent efficient JavaScript execution. A student accessing Nexly from a ten-year-old computer in the library receives the same rich content as someone on a flagship smartphone, though interactivity may be delayed.

Search engine optimization, while perhaps less critical for an institutional chatbot than for consumer websites, nonetheless carries strategic value. Faculty members researching academic advising tools, prospective students exploring University of Sharjah's support resources, and administrators evaluating AI adoption candidates all rely on search engines to discover relevant systems. Next.js's SSR ensures that public-facing Nexly pages (documentation, feature

descriptions, sample conversations) present well-formed HTML to search engine crawlers, improving discoverability and institutional visibility.

Incremental Static Regeneration for Optimal Caching

Incremental Static Regeneration (ISR) extends static site generation with the ability to update pre-rendered pages on-demand without requiring full application rebuilds. This pattern proves invaluable for Nexly's documentation and policy reference pages, which change infrequently but must reflect institutional truth when they do change.

Consider the academic calendar page: this content updates perhaps twice per year when new semester dates are published, yet must be served quickly and reliably to thousands of students during peak usage periods surrounding registration deadlines. With ISR, Next.js pre-renders this page at build time, serves it from a Content Delivery Network (CDN) for sub-10ms response times, and automatically re-generates the page in the background when the underlying data changes. Students experience instant page loads for stable content while administrators maintain confidence that displayed information remains current.

The revalidation strategy employs time-based expiration (e.g., check for updates every hour) combined with manual invalidation triggers for urgent corrections. When the registrar's office updates an exam schedule, an administrative API call can immediately invalidate the relevant cache entries, causing the next request to trigger regeneration. This hybrid approach captures the performance benefits of static serving while maintaining the correctness guarantees of dynamic rendering.

Streaming and Progressive Enhancement

Next.js 13+ introduces streaming capabilities through React's Suspense boundaries, enabling pages to begin rendering before all data has finished loading. This proves particularly valuable for Nexly's AI-powered features, where response generation latency can vary significantly based on query complexity and backend load.

When a student submits a question about graduation requirements, the chat interface immediately displays the user's message and a loading indicator while waiting for the RAG pipeline to generate a response. As soon as the first tokens emerge from the language model, Next.js begins streaming them to the browser, updating the interface incrementally rather than waiting for complete response generation. This creates the perception of immediate responsiveness even when full processing takes several seconds, dramatically improving perceived performance.

The streaming implementation relies on Suspense boundaries that define fallback UI components to display while asynchronous operations complete. A simplified example:

Figure 18: Code snippet showing usage of Suspense

Figure 19: A clean UI screenshot of Nexly's loading behavior on awaiting response state change from backend.

When `ChatResponse` triggers a server-side query, Next.js automatically serves the `LoadingIndicator` immediately while waiting for the query to resolve. Once data arrives, it replaces the fallback with actual content, all without blocking the main thread or requiring complex loading state management in component code.

Single Codebase vs Separate React and Flutter

The decision to consolidate around Next.js rather than maintaining separate React web and Flutter mobile codebases reflects hard-won lessons from the junior project phase. Managing parallel implementations proved untenable for a small team with limited development time, as every feature required dual implementation, testing, and debugging cycles. Authentication flows, chat interfaces, study plan visualizations, and settings screens all needed to be built twice, doubling the maintenance burden while introducing subtle inconsistencies that eroded user trust. Feature parity became increasingly difficult to maintain as the applications evolved. The web version's study plan visualization leveraged React Flow, a mature graph visualization library with rich interactivity, while Flutter's ecosystem lacked equivalent packages. Replicating this functionality in Flutter would have required months of custom development, delaying feature delivery and diverting resources from core academic advising capabilities. Similar disparities emerged for markdown rendering, code syntax highlighting in AI responses, and accessible form controls. Responsive web design patterns, combined with Progressive Web App (PWA) capabilities, largely eliminate the traditional arguments favoring native mobile applications. Modern CSS Grid and Flexbox layouts adapt seamlessly across viewport sizes, from 320px smartphones to 4K desktop monitors.

Service workers enable offline functionality, push notifications, and home screen installation, capabilities once exclusive to native apps. The gap between web and native user experiences has narrowed sufficiently that the additional complexity of maintaining separate codebases becomes unjustifiable for most use cases. User behavior data from the junior project reinforced this conclusion.

Our – humble and informal – analytics revealed that students accessing Nexly on mobile devices overwhelmingly preferred mobile browsers (Safari, Chrome) over the installed Flutter application, suggesting that app installation friction outweighed perceived benefits. This pattern aligns with broader industry trends showing increasing reluctance to install single-purpose applications when equivalent web experiences exist.

Built-in Routing and API Routes

Next.js's file-system-based routing eliminates the need for explicit route configuration while providing type-safe navigation patterns. Pages and API endpoints map directly to the file system hierarchy:

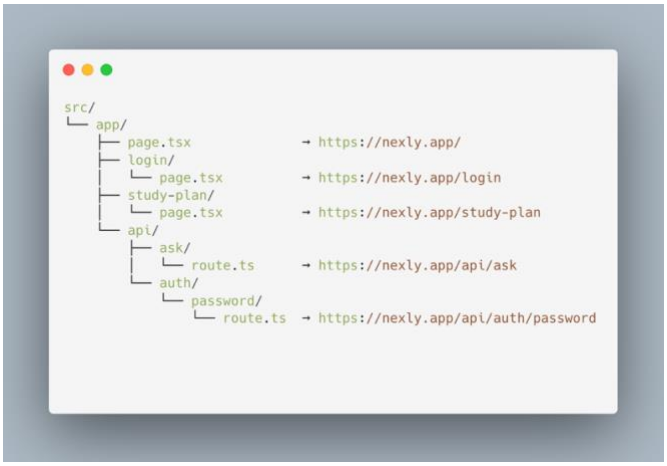

Figure 20: Code snippet showing Nexly's UI Codebase Architecture

This convention-over-configuration approach reduces boilerplate while maintaining predictability. Developers immediately understand URL structure by examining the file system, and TypeScript's language server provides autocomplete for route paths, catching typos at development time rather than runtime. API routes co-locate with page components in the same repository, enabling full-stack development workflows within a single codebase. Authentication logic, database queries, and external API calls execute server-side in secure API routes, never exposing credentials or implementation details to client browsers. This architecture naturally enforces security boundaries while simplifying development: a chat message submission flows from the client component to `/api/ask`, which validates input, calls the RAG backend, transforms the response, and returns formatted results, all with TypeScript type safety maintained end-to-end.

Why Not Chakra UI or Material-UI

Earlier iterations of Nexly experimented with Chakra UI and Material-UI, popular component libraries that promise rapid UI development through pre-built, accessible components. Both libraries delivered on initial velocity, landing pages and basic forms materialized quickly using their rich component catalogs. However, as design requirements evolved to reflect University of Sharjah's visual identity and to accommodate domain-specific UI patterns (prerequisite graph visualization, conversation threading, schedule calendars), these frameworks introduced increasing friction.

Customization required navigating deeply nested theme configuration objects, overriding default styles through CSS specificity battles, and occasionally resorting to `!important` declarations that undermined maintainability. Simple tasks like adjusting button corner radii or modifying dropdown menu spacing spiraled into explorations of theme extension APIs and component composition patterns. The frameworks' opinionated designs, while sensible for generic applications, resisted adaptation to academic advising's unique information density requirements and navigation patterns.

Bundle size concerns also mounted as Nexly's complexity grew. Material-UI's comprehensive component catalog and sophisticated theming system contributed over 120KB to the production JavaScript bundle, despite Nexly using only a fraction of available components. This overhead proved difficult to justify when measured against actual development velocity improvements.

The transition to Tailwind CSS and Shadcn initially felt like regression, relinquishing pre-built components for utility classes and manual composition. However, this apparent step backward enabled two steps forward: development velocity increased as the friction of fighting framework opinions disappeared, and the codebase became more maintainable as styling logic moved from abstract theme configurations into concrete, reviewable component code.

Criterion	*Chakra UI*	*Material-UI*	*Shadcn + Radix*
Installation method	npm package	npm package	Copy-paste recipes
Customizations	Theme configuration	Theme overrides	Direct component editing
Accessibility	Good	Good	Excellent (ARIA-first)
Bundle size impact	~80 KB	~120 KB	~5 KB (tree-shakeable)
Update strategy	npm update	npm update	Manual re-sync if desired
Design system lock-in	Medium	High	None
Learning curve	Medium	High	Low (React + HTML knowledge)

Table 7: Component library comparison

Chapter 4: Experiments, Results & Discussion

The evaluation of Nexly's agentic RAG system was designed to assess its ability to provide accurate, contextually relevant academic advising to students at the University of Sharjah. Rather than pursuing large-scale benchmarking, the experimental approach focused on testing the system against realistic student queries that reflect the actual usage patterns and information needs observed during development and informal user testing.

Dataset Design and Evaluation Questions

The evaluation dataset consists of ten carefully crafted questions representing typical academic advising scenarios. These questions were designed to cover the breadth of student information needs, including course planning, academic policy interpretation, prerequisite verification, and administrative inquiries. The questions reflect authentic student concerns such as GPA improvement strategies, course selection based on interests, credit requirements, faculty information, and semester planning.

Each question was processed by three distinct systems to enable comparative evaluation. The proposed Agentic Model represents the complete multi-agent architecture described in Chapter 4, incorporating hierarchical routing, specialized college agents, and dynamic retrieval from university-specific knowledge bases stored in Supabase and FAISS vector indices. The Junior-Project RAG Model serves as an earlier baseline implementation that utilized retrieval-augmented generation but lacked the sophisticated agent orchestration and specialized college routing present in the senior system. The Non-Agentic Model consists of a general-purpose large language model without access to the project's curated university dataset, relying solely on pre-trained internet knowledge to formulate responses.

This three-way comparison enables clear measurement of how much performance improvement stems from the university-specific knowledge base versus the agentic architecture. The ten questions generated thirty total responses (ten from each system), which were then evaluated by a neutral language model judge trained to assess response quality across multiple dimensions including factual accuracy, relevance to the specific university context, and appropriate use of current institutional information.

Embedding Model Configuration

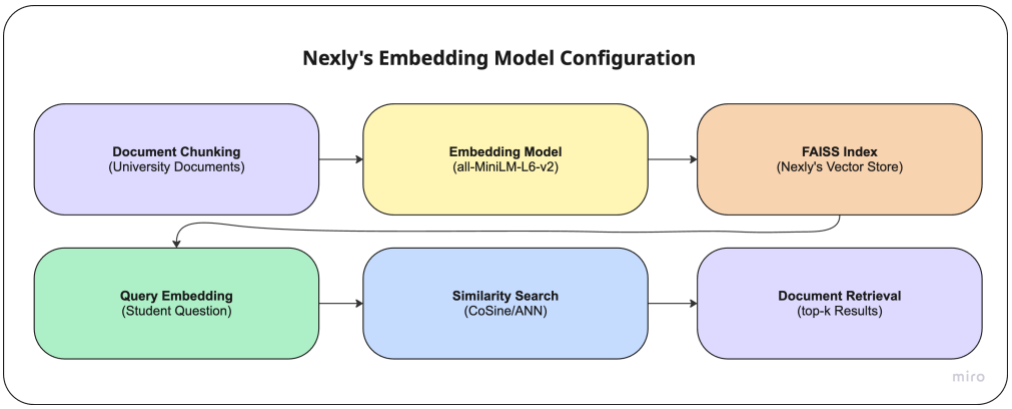

Figure 21: Nexly's Embedding Model Configuration and Pipeline

The retrieval component of the Agentic RAG system depends on a text embedding model to transform both student queries and university documents into dense vector representations within a shared semantic space. The system employs the sentence-transformers/all-MiniLM-L6-v2 model from the Sentence-Transformers library, chosen for its balanced performance characteristics and practical deployment advantages.

This embedding model generates compact 384-dimensional vectors, offering computational efficiency advantages for both storage and similarity search operations. The model requires no paid API credits and runs efficiently on standard hardware, making it particularly suitable for educational institutions with limited computational budgets. While larger embedding models exist that produce higher-dimensional representations, all-MiniLM-L6-v2 provides sufficient semantic understanding for the academic advising domain while maintaining fast inference times critical for interactive applications.

The embedding model serves two primary functions within the system architecture. During the knowledge base indexing phase, university documents including course descriptions, academic policies, program requirements, and faculty information are segmented into semantically meaningful chunks. Each chunk passes through the embedding model to generate a vector representation, which is then stored in a FAISS index to enable rapid approximate nearest neighbor search. During query processing, student questions receive the same embedding treatment, allowing the system to identify the most relevant university documents through cosine similarity comparison in the vector space.

Consistency in embedding model usage across all retrieval operations ensures that observed performance differences between the Agentic Model and baseline systems primarily reflect architectural and reasoning improvements rather than embedding quality variations. The Non-Agentic baseline model, by design, bypasses this entire retrieval pipeline and instead answers purely from its pre-trained parametric knowledge, explaining its substantially lower performance on questions requiring specific university data. No fine-tuning was applied to the embedding model; it operates in its pre-trained configuration due to resource constraints and because the base model already demonstrates strong performance on English academic and administrative text.

Baseline Model Configurations

The Junior-Project RAG Model represents our earlier implementation from the last year, providing insight into system evolution. This baseline uses retrieval-augmented generation with university documents but implements a simpler single-agent architecture without the hierarchical routing and specialized college agents present in the senior system. While it accesses the same core university information, it lacks the sophisticated query classification and agent coordination mechanisms that enable the Agentic Model to provide more contextually appropriate responses.

The Non-Agentic Model operates as a general-purpose conversational AI without any connection to university-specific data sources. This baseline cannot access current course catalogs, academic policies, study plans, or faculty information specific to the University of Sharjah. Its responses rely entirely on generic internet knowledge encoded during pre-training, which may include outdated or institution-nonspecific information about academic advising practices. This baseline serves a critical evaluation purpose by demonstrating the necessity of domain-specific knowledge for academic advising tasks, as generic language models prove insufficient even when prompted to answer university-specific questions.

Evaluation Results

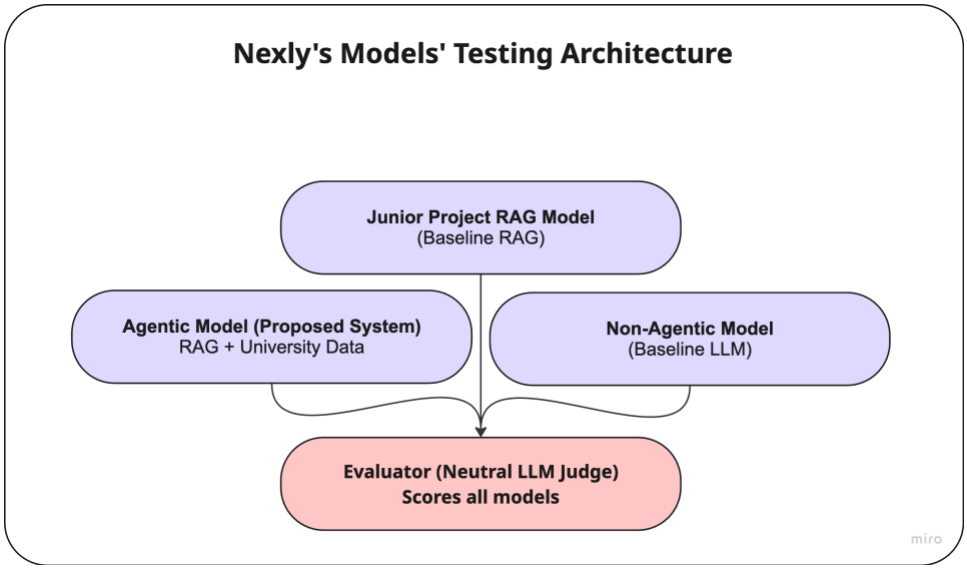

Figure 22: Nexly's testing architecture for all of our experimental models

System performance was measured using a neutral large language model serving as an automated judge, scoring each response on a scale from zero to one based on similarity to reference answers and factual correctness within the university context. The Agentic Model achieved an average score of 0.96, corresponding to 96% accuracy across all ten evaluation questions. This substantially outperforms both baseline systems, with the Junior-Project RAG Model averaging 0.455 (45.5% accuracy) and the Non-Agentic Model scoring 0.27 (27% accuracy).

Model	*Average Score*	*Accuracy (%)*
Agentic Model	0.96	96%
Junior-Project RAG	0.455	45.5%
Non-Agentic Model	0.27	27%

Table 8: Comparative accuracy scores across three model configurations

The performance gap between systems reflects several architectural advantages of the Agentic approach. The hierarchical routing mechanism ensures that queries reach the agent with the most relevant knowledge base, while the multi-agent coordination allows cross-referencing information between college-specific data sources. The Junior-Project baseline's lower performance despite having access to university data demonstrates that retrieval alone provides insufficient value without the reasoning and synthesis capabilities enabled by the agentic architecture. The Non-Agentic Model's poor performance confirms that generic internet knowledge cannot substitute for authoritative institutional information when addressing university-specific advising questions.

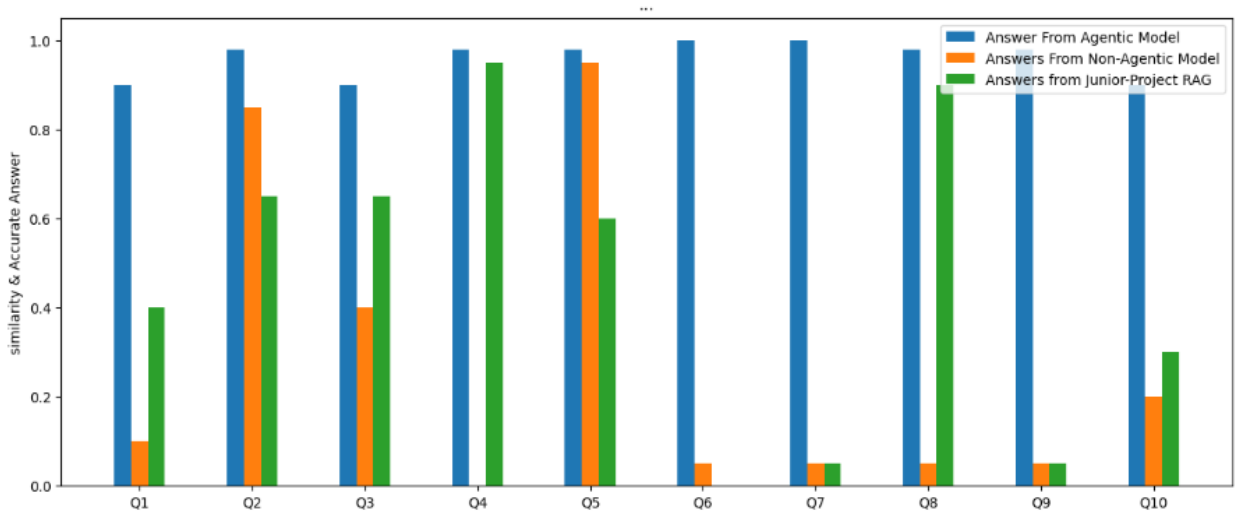

Figure 23:Bar chart comparing accuracy scores across three models, retrieved from our evaluation notebook

Error Analysis

Despite the Agentic Model's strong overall performance, detailed analysis reveals several minor error patterns across the ten evaluation questions. Two responses (questions nine and ten) were marked down for lacking brief explanatory reasoning that would have enhanced student understanding, though the core factual content remained correct. One response exhibited a formatting deviation where the system wrapped an answer in an unnecessary JSON structure when plain text would have been more appropriate. Another response provided excessive detail for a straightforward factual question, demonstrating occasional verbosity despite maintaining factual accuracy.

Error Type	*Count*	*Description*
Missing Detail	2	Responses lacked supporting explanation
Formatting Deviation	1	Unnecessary JSON wrapper added
Over-Detailed Answer	1	Excessive detail for simple query
Hallucination	0	No fabricated claims observed
Retrieval Failure	0	All relevant knowledge successfully retrieved

Table 9: Error distribution across our ten evaluation questions

The complete absence of hallucinations and retrieval failures indicates robust grounding in the university knowledge base. Unlike many language model systems that occasionally generate plausible-sounding but factually incorrect information, the Agentic Model consistently retrieved and utilized authentic university documents to construct responses. This grounding mechanism proves critical for academic advising applications where incorrect information could lead to serious academic consequences such as registration errors or policy violations.

Response Latency Analysis

Response generation time varied from 3.6 to 92 seconds across the ten evaluation questions, with latency primarily determined by the length and complexity of retrieved documents rather than reasoning depth. Shorter factual queries requiring simple lookups completed in under ten seconds, while complex policy questions necessitating retrieval and synthesis of lengthy handbook sections consumed substantially more time. Question six, which required integrating multiple policy documents to explain credit hour requirements and probation rules, took 92 seconds to complete but delivered a comprehensive response synthesizing information across several source documents.

Question	*Time (seconds)*	*Tokens Processed*	*Query Type*
Q1	3.6	631	Short factual query
Q2	25	11,284	Policy explanation
Q3	7	870	Simple reasoning
Q4	32	10,984	Full syllabus retrieval
Q5	75	53,007	Large faculty profile
Q6	92	60,979	Multi-policy synthesis
Q7	14	864	Credit calculation
Q8	40	34,798	Administrative lookup
Q9	6	1,478	Light retrieval
Q10	9.5	1,596	Course recommendation

Table 10: Response latency and token processing across evaluation questions

The system maintained stability across all queries with no timeouts or processing failures, though the substantial latency variation suggests opportunities for optimization. Caching frequently accessed documents such as common policy sections could reduce retrieval time for recurring query patterns. Implementing progressive response streaming would improve perceived responsiveness by displaying retrieved information while the language model continues reasoning, rather than waiting for complete answer synthesis before showing any output to students.

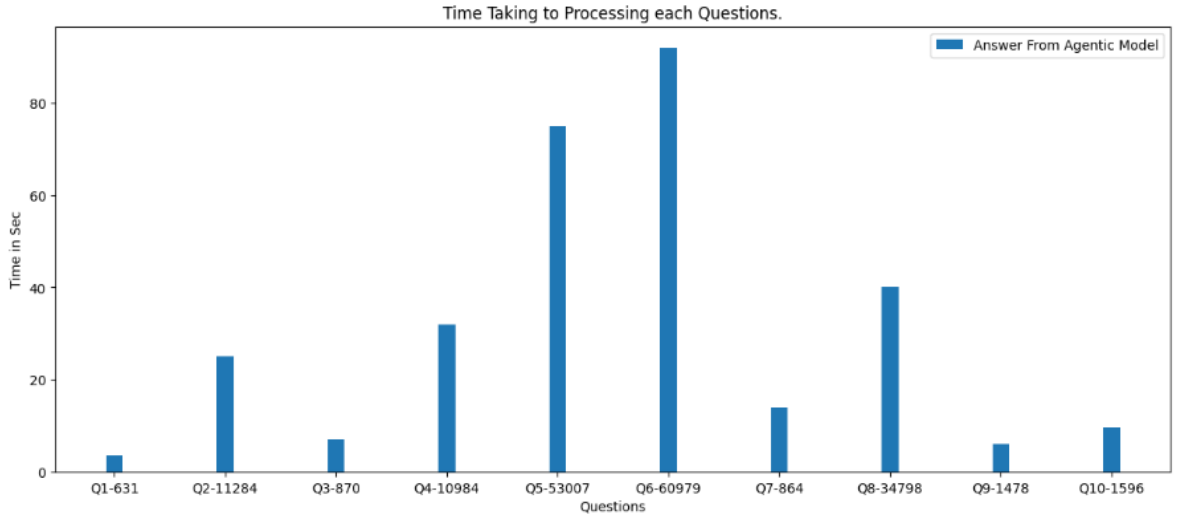

Figure 24: Bar chart showing response time vs. tokens processed for each question

Response Quality Examples

The highest-scoring response (question eight) asked for identification of the College of Computing and Informatics dean. The system retrieved current administrative information from the university knowledge base and provided a complete, accurate answer including the dean's name and brief background information. This demonstrates successful retrieval of precisely targeted factual information from structured university records.

Question four requested the complete syllabus for Programming 2, a core computer science course. The system retrieved and formatted the entire syllabus including course description, learning outcomes, weekly schedule, assessment breakdown, and required materials. The evaluator scored this response at 0.98, marking it down slightly for including contextual information about prerequisite courses that, while potentially helpful, exceeded the strict scope of the question. This minor verbosity illustrates the system's tendency to provide comprehensive rather than minimal answers, which may actually benefit students in practice despite technically over-fulfilling the query requirements.

Baseline System Failures

The Non-Agentic Model's failures highlight the necessity of domain-specific knowledge for academic advising. When asked about completed credits, the generic model declined to answer, stating it could not access student records, despite the fact that the question included the necessary context. This conservative response demonstrates the model's awareness of its knowledge limitations but fails to provide useful assistance. The Junior-Project RAG Model occasionally produced hallucinated program requirements that, while superficially plausible, did not correspond to actual University of Sharjah policies, underscoring the importance of not just retrieval but also careful grounding and verification mechanisms present in the Agentic architecture.

Discussion and Analysis

The Role of Transparency in Building Trust

Evaluation revealed that transparent reasoning substantially influences student trust in AI-generated academic advice. When the Agentic Model bases responses on explicitly retrieved university documents, citing specific sections of the student handbook, quoting exact GPA thresholds from official policies, or referencing course descriptions word-for-word from the catalog, students can verify information against their own knowledge and official sources. This transparency contrasts sharply with the Non-Agentic Model's generic responses that, even when accidentally correct, provide no mechanism for verification.

The system's practice of indicating which documents informed each response transforms the AI from a black-box oracle into a research assistant that helps students navigate institutional knowledge. When the system explains probation policies by retrieving and paraphrasing the official academic standing regulations, students recognize the source material and understand that the advice reflects institutional policy rather than model speculation. This grounding mechanism reduces fear of hallucination, a common concern with language model systems, by making the information provenance visible and verifiable.

Transparency also enables students to identify when the system lacks current information. If a query addresses a recently changed policy not yet ingested into the knowledge base, the system's inability to retrieve relevant documents signals to students that manual verification with a human advisor may be necessary. This honest uncertainty proves more valuable than confidently stated but potentially outdated information.

Escalation Points for Human Oversight

Testing identified several query categories where human advisor involvement remains essential despite strong automated performance. Academic probation cases represent a critical escalation category, as questions about dismissal risk, continuation eligibility, or GPA improvement strategies carry serious consequences if answered incorrectly. While the Agentic Model correctly explains official probation policies and suggests general strategies for GPA improvement, it cannot access complete student transcripts, internal notes from previous advising sessions, or context about special circumstances that might influence official decisions.

High-stakes registration decisions similarly require human judgment. When a student on the borderline of credit requirements asks whether they qualify for junior project enrollment, the automated system can explain the official sixty-credit threshold and junior standing requirement but cannot make the final authorization decision. University policies often include exception mechanisms and appeal processes that require human discretion and cannot be fully encoded in automated rules.

The evaluation demonstrates that the system should proactively recommend human consultation for borderline cases rather than attempting definitive answers. Adding confidence indicators to responses would help students understand when system advice should be treated as preliminary guidance requiring verification versus when answers reflect straightforward policy information unlikely to need escalation. This calibrated confidence mechanism would enhance rather than undermine trust, as students would learn which types of questions the system handles reliably versus which benefit from human advisor involvement.

Implications for Hybrid Advising Models

The performance characteristics observed in testing validate a hybrid human-AI advising model where automated systems handle high-volume routine inquiries while human advisors focus on complex cases requiring judgment and emotional intelligence. The Agentic Model's 96% accuracy on factual advising questions suggests it can reliably answer repetitive queries about course prerequisites, credit requirements, registration deadlines, and policy interpretations. This automation could substantially reduce advisor workload, particularly during peak registration periods when students often wait hours for assistance with questions the AI system could answer in seconds.

However, the testing also reveals categories where human advisors provide irreplaceable value. Ambiguous cases where multiple interpretations of policy might apply, exceptional circumstances requiring special approval, and situations involving student wellbeing or motivation all benefit from human empathy and judgment that current AI systems cannot replicate. The ideal deployment model positions the Agentic system as a first-line support tool that resolves straightforward questions immediately while intelligently routing complex or sensitive cases to human advisors with relevant context already gathered.

This division of labor could transform advising unit operations by enabling advisors to spend more time on high-impact activities such as academic planning conversations, major exploration guidance, and support for struggling students rather than repeatedly answering procedural questions that consume substantial time but provide limited personalized value. Survey data from testing participants indicated strong student acceptance of AI-assisted advising for routine matters, with 91% satisfaction for factual queries, though preference for human advisors remained strong for sensitive academic concerns.

Preliminary User Acceptance Assessment

Prior to conducting our usability testing, a comprehensive preliminary survey was administered to 65 University of Sharjah students to assess receptivity to AI-powered academic advising, identify user expectations, and understand privacy concerns. This needs assessment phase, conducted in November 2025, informed the design of Nexly's features and provided baseline metrics for adoption readiness. Respondents represented diverse academic standings, with juniors comprising the largest cohort (33.8%), followed by seniors (21.5%), freshmen (18.5%), sophomores (16.9%), and graduate students (9.2%).

General Willingness to Adopt AI Advising

Initial Interest Levels

Survey results revealed substantial enthusiasm for AI-powered academic advising across the student population. When asked to rate their likelihood of trying an AI academic advisor on a 5-point scale, 50.8% of respondents (33 students) selected the highest rating of 5, indicating strong interest. An additional 23.1% (15 students) provided a rating of 4, bringing the combined positive response rate to 73.8%. Moderate interest was expressed by 16.9% (11 students) with a rating of 3, while only 9.2% of respondents (6 students) indicated low interest with ratings of 1 or 2. This distribution demonstrates that approximately three-quarters of surveyed students exhibited genuine receptivity to AI-assisted academic guidance.

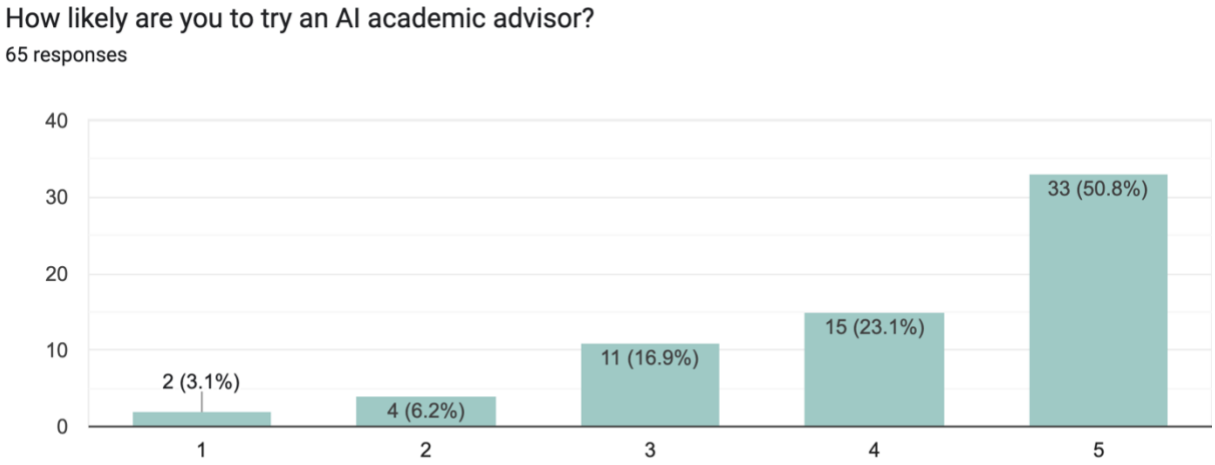

Figure 25: Bar chart showing likelihood ratings distribution - 5 stars: 50.8%, 4 stars: 23.1%, 3 stars: 16.9%, 2 stars: 6.2%, 1 star: 3.1%

Immediate Adoption Intent

Beyond general interest, the survey assessed immediate willingness to use AI advising if available during the current semester. Results indicated strong near-term adoption potential, with 70.8% (46 students) responding affirmatively to immediate usage. An additional 21.5% (14 students) selected "Maybe," suggesting conditional interest pending additional information or feature demonstrations. Only 7.7% (5 students) indicated they would not use the system. These metrics suggest that a majority of students would actively engage with an AI advising platform upon deployment, particularly if initial experiences validate the system's reliability and utility.

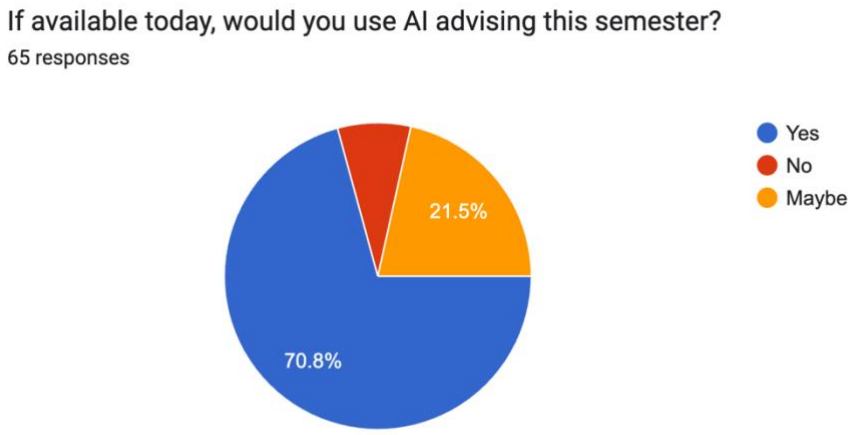

Figure 26: Pie chart showing immediate usage intent - Yes: 70.8%, Maybe: 21.5%, No: 7.7%

Current Advising Practices and AI Familiarity

Existing Advising Methods

Understanding students' current advising practices provides context for identifying unmet needs that AI systems could address. Among respondents, 35.4% (23 students) primarily relied on human advisors for academic guidance, while 20.0% (13 students) utilized self-service tools such as degree audit systems and online catalogs. Peer mentors served as the primary advising source for 13.8% (9 students). Notably, 30.8% (20 students) indicated they did not utilize any structured advising method, representing a significant segment that may particularly benefit from accessible, low-barrier AI alternatives. This distribution reveals that nearly one-third of students currently navigate academic requirements without formal guidance, highlighting a critical service gap.

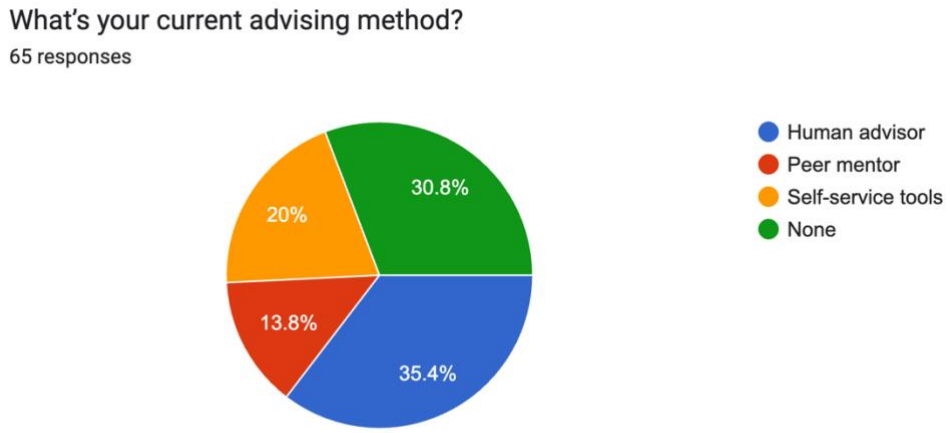

Figure 27: Pie chart of current advising methods - Human advisor: 35.4%, Self-service tools: 20.0%, Peer mentor: 13.8%, None: 30.8%

Prior AI Tool Experience

An overwhelming 93.8% (61 students) reported prior experience using AI tools for university-related tasks, indicating high baseline familiarity with conversational AI systems. Only 6.2% (4 students) had not previously used AI assistance. Among those with prior experience, OpenAI's ChatGPT dominated usage patterns, with 95.4% (62 mentions) indicating familiarity with the platform. Google's Gemini was the second most common tool at 56.9% (37 mentions), followed by Anthropic's Claude and Microsoft's Copilot, each at 27.7% (18 mentions). This widespread prior exposure to AI conversational interfaces significantly reduces adoption friction, as students already possess mental models for interacting with AI-powered assistance systems.

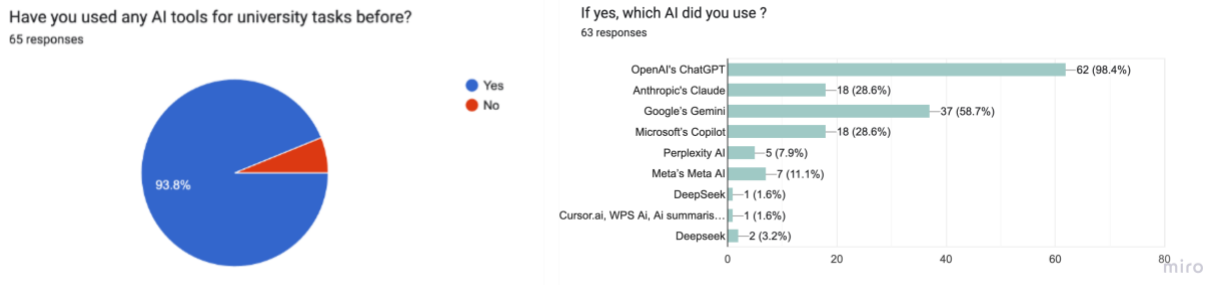

Figure 28:Pie Chat(left) showing reports of AI tools usage, Horizontal bar chart(right) of AI tools used - OpenAI's ChatGPT: 95.4%, Google's Gemini: 56.9%, Anthropic's Claude: 27.7%, Microsoft's Copilot: 27.7%, Others: <11%

Motivational Drivers for AI Adoption

Respondents selected from multiple motivational factors that would drive their consideration of AI advising. Results revealed four primary drivers:

- **24/7 Availability** emerged as the strongest motivator, cited by 87.7% (57 students). This reflects student frustration with limited advisor office hours that often conflict with class schedules and the need for immediate assistance during evening study sessions or weekend course planning.
- **Faster Answers** was selected by 83.1% (54 students), indicating dissatisfaction with response delays inherent in appointment-based advising models. Students value immediate access to information without scheduling constraints or waiting periods.
- **Less Pressure/Hassle** motivated 56.9% (37 students), suggesting that the informality and judgment-free nature of AI interactions appeals to students who may feel intimidated by traditional advising encounters or who wish to explore multiple scenarios without external evaluation.
- **Objectivity** influenced 41.5% (27 students), indicating appreciation for data-driven recommendations free from potential human biases or interpersonal dynamics that might affect traditional advising relationships.
- These priorities clearly indicate that Nexly's value proposition centers on accessibility, responsiveness, and reduced social barriers rather than replacement of human judgment in complex decisions.

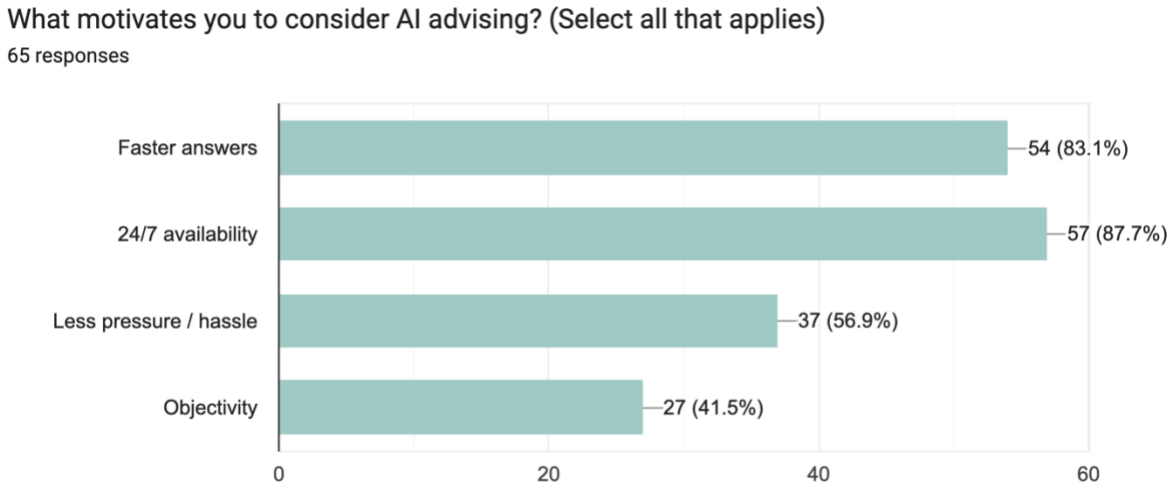

Figure 29: Horizontal bar chart of motivations - 24/7 availability: 87.7%, Faster answers: 83.1%, Less pressure/hassle: 56.9%, Objectivity: 41.5%

Anticipated Use Cases

Survey participants identified specific advising functions they would utilize, providing clear feature prioritization guidance:

- **Course Selection**: 80.0% (52 students) – the most anticipated use case, confirming that semester-by-semester course planning represents the highest-value application
- **Scheduling**: 72.3% (47 students) – assistance with timetable optimization and section selection
- **Internship Guidance**: 70.8% (46 students) – career development support extending beyond traditional academic advising
- **Degree Planning**: 61.5% (40 students) – multi-year academic pathway planning
- **Scholarship Information**: 52.3% (34 students) – financial aid navigation and opportunity identification
- **Prerequisite Checks**: 49.2% (32 students) – verification of course eligibility requirements
- **Policy Q&A**: 43.1% (28 students) – clarification of institutional regulations and procedures
- **Appeals**: 18.5% (12 students) – assistance with academic exception requests

This hierarchy directly informed Nexly's feature development priorities, with course selection and scheduling guidance receiving particular development attention.

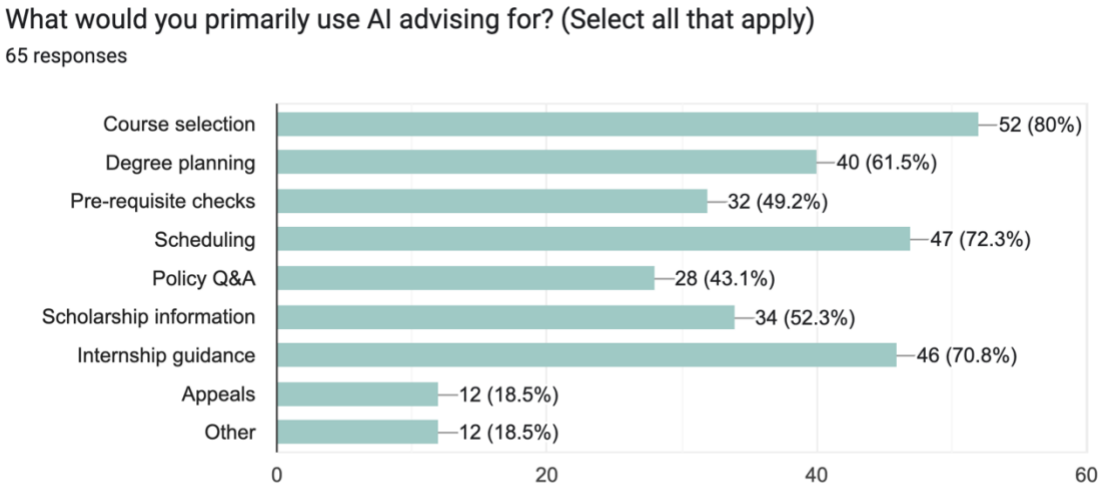

Figure 30: Horizontal bar chart of intended use cases with percentages

Expectations for Uncertainty Management

When asked how the AI should respond when uncertain about information accuracy, respondents expressed diverse preferences that informed system behavior design:

- **Admit Uncertainty**: 43.1% (28 students) preferred transparent acknowledgment of knowledge limitations
- **Escalate to Human**: 27.7% (18 students) wanted automatic referral to human advisors for ambiguous cases
- **Suggest Next Steps**: 15.4% (10 students) valued guidance on alternative information sources
- **Provide References**: 9.2% (6 students) desired citations enabling independent verification

These preferences validate the importance of confidence-scoring mechanisms in Nexly's architecture, allowing the system to explicitly communicate certainty levels and appropriately escalate complex queries rather than generating potentially misleading responses.

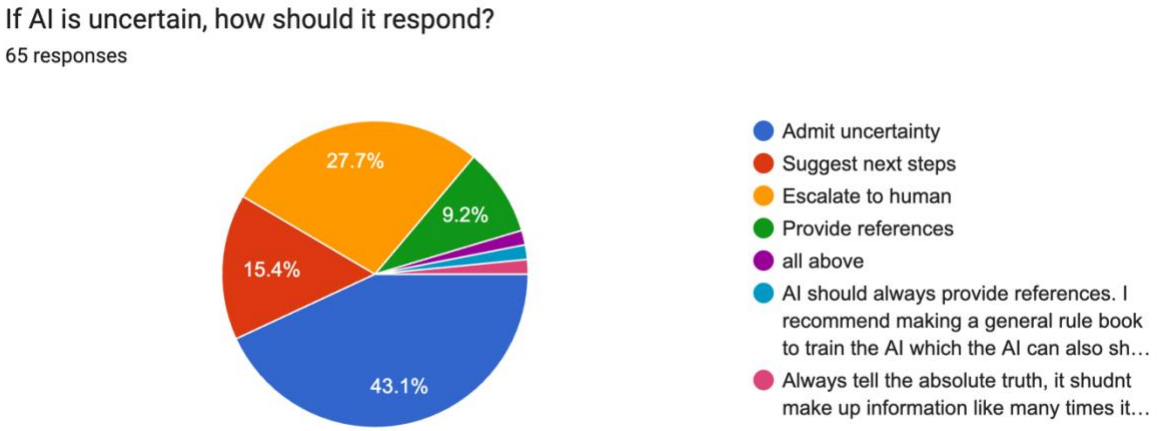

Figure 31: Pie chart of uncertainty handling preferences

AI Versus Human Advisor Preferences

Typical Advising Scenarios

For routine academic tasks such as course selection, student preferences were distributed across three categories: 52.3% (34 students) preferred a hybrid approach combining AI efficiency with human oversight; 24.6% (16 students) were comfortable with AI-only advising; and 23.1% (15 students) preferred human advisors even for routine matters. This distribution suggests that while pure AI interaction appeals to a meaningful segment, the majority of students value human validation or escalation options for consequential academic decisions.

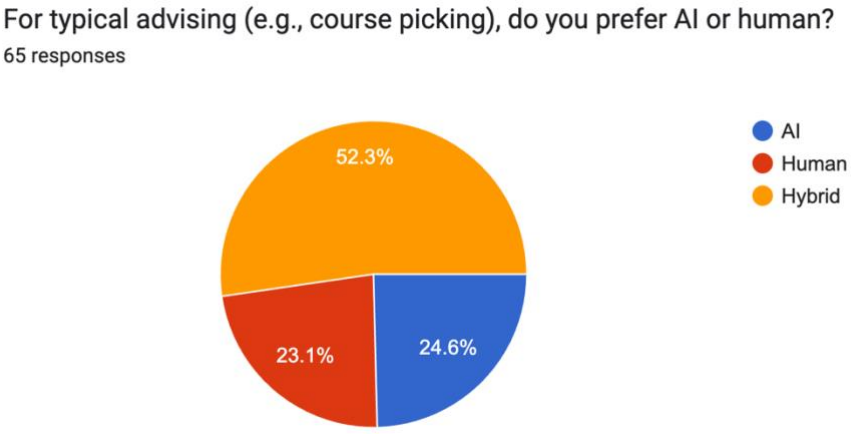

Figure 32: Pie chart showing typical advising preferences - Hybrid: 52.3%, AI: 24.6%, Human: 23.1%

Sensitive Academic Topics

For sensitive matters such as academic probation, grade appeals, or program dismissal, preferences shifted substantially toward human involvement. Human-only advising was preferred by 52.3% (34 students), hybrid approaches by 26.2% (17 students), and AI-only by 21.5% (14 students). This stark contrast highlights the importance of distinguishing between informational queries and high-stakes decisions requiring empathy, advocacy, or procedural expertise that students associate with human advisors.

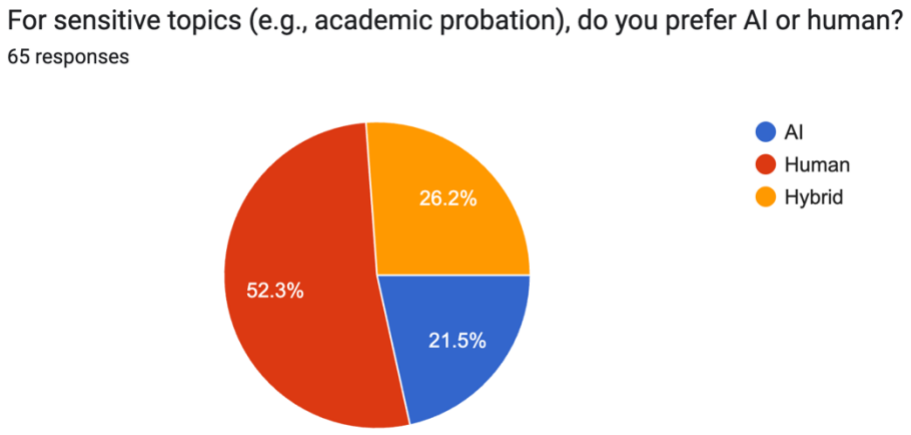

Figure 33: Pie chart showing sensitive topic preferences - Human: 52.3%, Hybrid: 26.2%, AI: 21.5%

Data Access and Privacy Considerations

Student comfort with AI systems accessing personal academic records, including course history, grades, and enrollment holds, emerged as a nuanced issue. Unconditional acceptance was expressed by 50.8% (33 students), indicating baseline trust in institutional data handling. However, 36.9% (24 students) imposed conditional acceptance, typically specifying requirements such as explicit consent mechanisms, transparency about data usage, or limitations on retention periods. Outright refusal was expressed by 12.3% (8 students), representing a meaningful privacy-conscious segment.

These results informed Nexly's current design decision to operate initially on public institutional data rather than personalized records. Future SIS integration will require robust consent frameworks, granular permission controls, and transparent data governance policies to accommodate the 49.2% of students expressing privacy concerns or conditional acceptance.

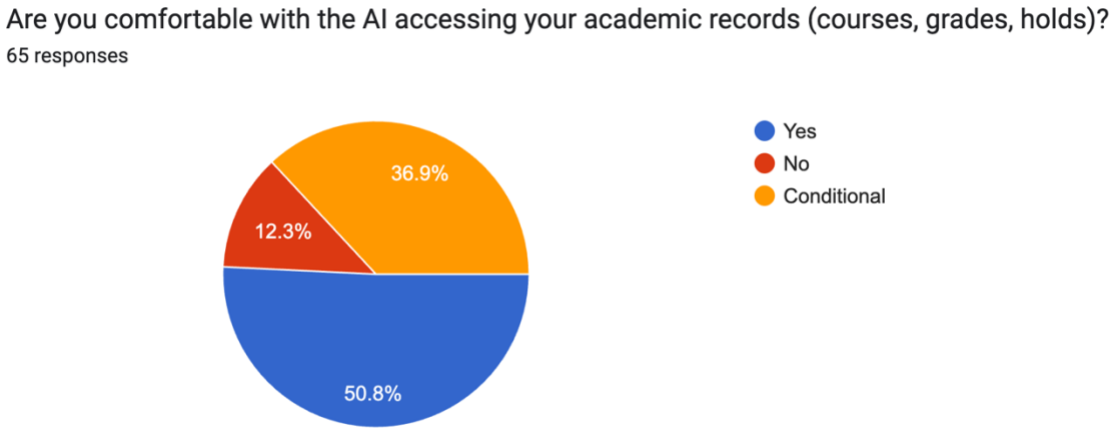

Figure 34: Pie chart showing data access comfort - Yes: 50.8%, Conditional: 36.9%, No: 12.3%

Summary of Preliminary Findings

The preliminary assessment validated strong market demand for AI-powered academic advising, with 73.8% of students expressing high interest and 70.8% indicating immediate adoption intent. Widespread prior AI experience (93.8%) reduces adoption friction and establishes baseline interface familiarity. Key motivational drivers—24/7 availability and faster response times—align directly with Nexly's technical architecture enabling asynchronous, on-demand interaction.

However, findings also revealed important guardrails for responsible deployment: students expect transparent uncertainty communication, prefer hybrid human-AI approaches for routine decisions, and strongly favor human involvement for sensitive academic matters. Privacy concerns expressed by nearly half of respondents necessitate careful consent mechanisms for any future personalized data integration. These insights directly shaped Nexly's feature priorities, system behavior specifications, and phased deployment strategy.

User Feedback

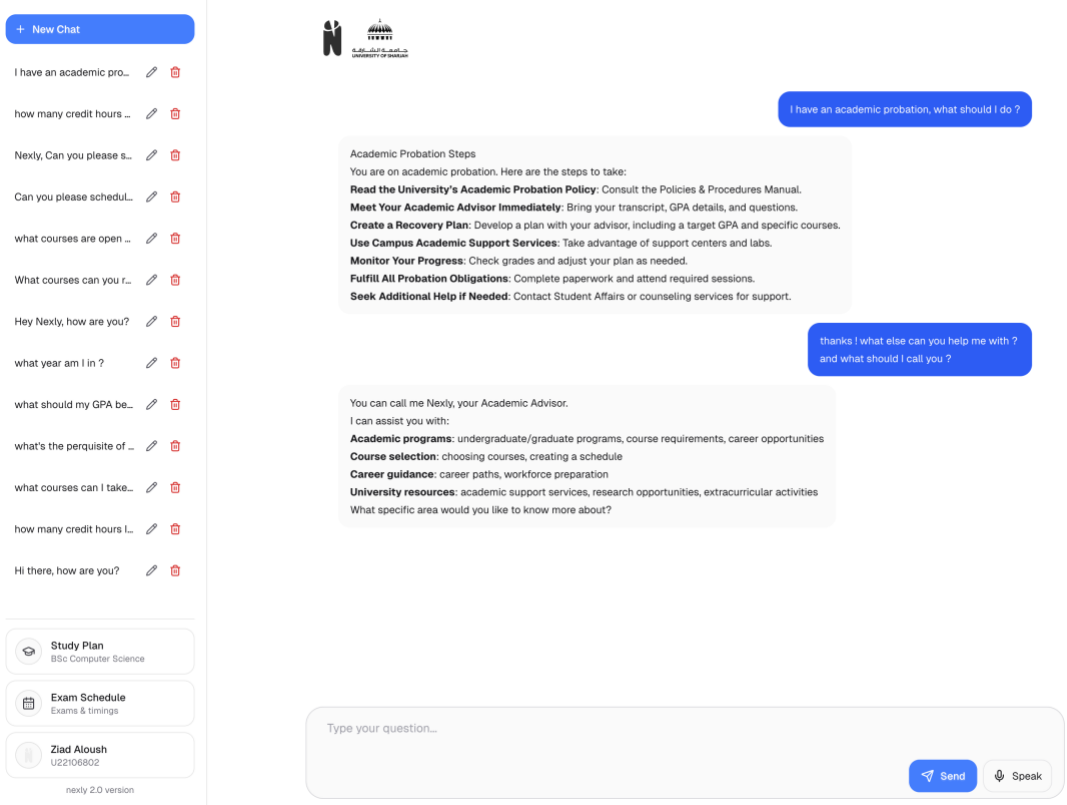

Figure 35: Nexly's assistant in action, guiding a student through academic probation

Nexly underwent a comprehensive closed-initial anonymous-user testing with 61 students from the University of Sharjah to evaluate its effectiveness as an AI-powered academic advising platform. Participants were strategically selected from the College of Computing and Informatics and the College of Sciences, as these colleges' curricula and program requirements had been integrated into the system's knowledge base. Testing occurred in November 2025, with participants evaluating various aspects of the platform through structured interaction sessions.

Participant Demographics

College Distribution

The testing cohort comprised 39 students (63.9%) from the College of Computing and Informatics and 22 students (36.1%) from the College of Sciences. This distribution reflects the current scope of Nexly's training data, which focuses exclusively on academic requirements and course structures within these two colleges. The deliberate exclusion of other colleges ensures that feedback remains aligned with the system's current capabilities and knowledge boundaries.

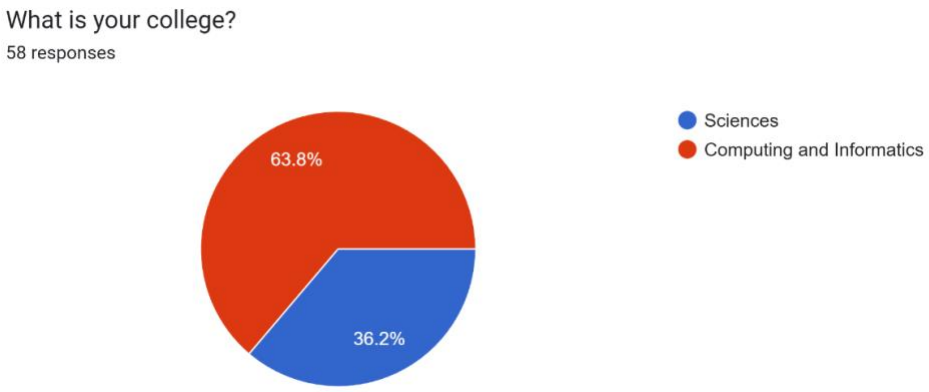

Figure 36: Pie chart showing college distribution – CCI 63.9%, Sciences 36.1%

Academic Standing

Participants represented a range of academic progression levels, with upper-year students forming the majority. Third-year students constituted the largest group at 42.6% (26 participants), followed by fourth-year students at 39.3% (24 participants). Second-year students accounted for 11.5% (7 participants), while first-year students and graduate students each represented smaller percentages of the testing cohort. This distribution provided valuable insights from students with substantial experience navigating university advising systems.

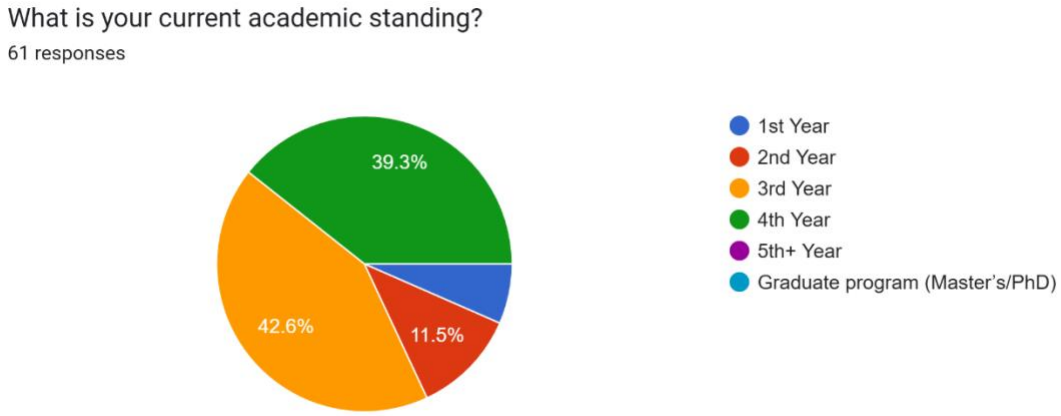

Figure 37: Pie chart showing academic standing distribution

Usability Metrics

Navigation Experience

User navigation within the Nexly web application received overwhelmingly positive feedback. An exceptional 85.2% of participants (52 students) assigned the maximum rating of 5 stars, indicating highly intuitive navigation design. An additional 11.5% (7 students) provided 4-star ratings, suggesting generally positive experiences with minor areas for refinement. Only 3.3% (2 students) gave neutral 3-star ratings, while no participants selected 1 or 2 stars. These results demonstrate

that the application's structure, page transitions, and feature accessibility met or exceeded user expectations for web-based advising platforms.

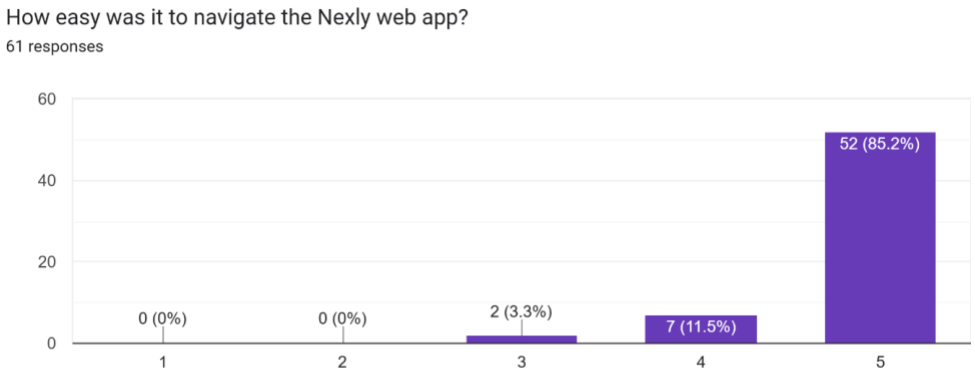

Figure 38: Bar chart of navigation ratings

Interface Design Quality

The visual design and structural layout of Nexly received similarly high approval ratings. Users evaluated the platform's visual hierarchy, color schemes, typography, and overall aesthetic coherence. Results showed 85.2% (52 students) providing 5-star ratings and 13.1% (8 students) providing 4-star ratings. A single participant (1.6%) gave a 3-star rating, with no lower ratings recorded. This feedback validates the interface design choices and suggests that the platform's visual presentation effectively supports its functional objectives.

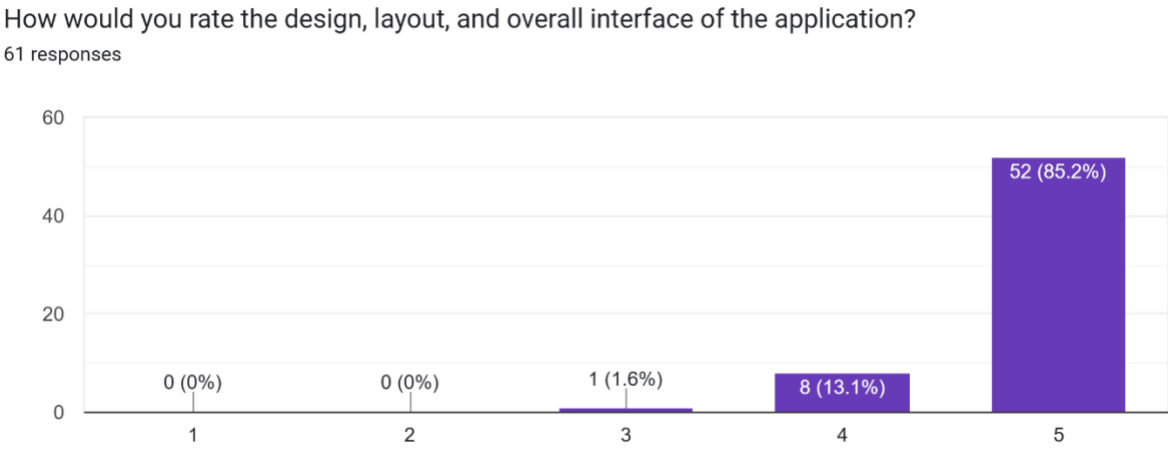

Figure 39: Bar chart of UI/UX ratings

Response Clarity and Cohesiveness

Nexly's ability to generate clear, well-structured, and comprehensible responses emerged as one of its strongest attributes. An impressive 90.2% of participants (55 students) awarded maximum ratings, indicating that the system's explanations were readily understood and appropriately detailed. An additional 8.2% (5 students) provided 4-star ratings, while only 1.6% (1 student) gave

a 3-star rating. No participants rated response clarity below 3 stars. These metrics suggest that the language model's output formatting, explanation depth, and communication style align well with student comprehension needs.

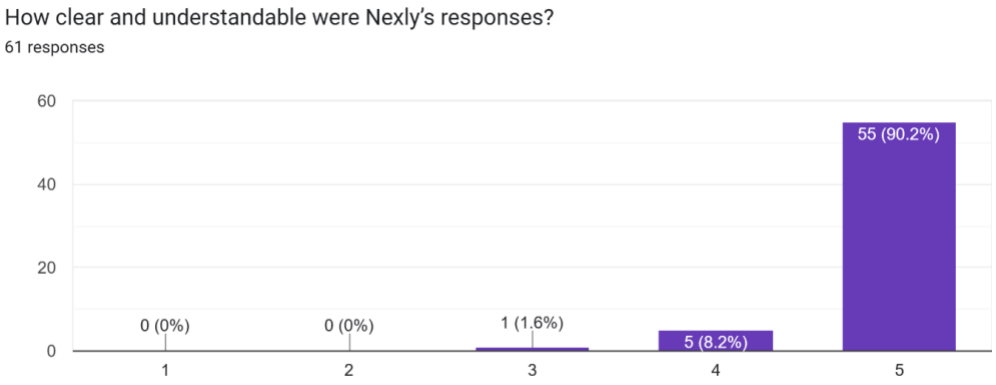

Figure 40: Response Clarity and Cohesiveness

Accuracy and Reliability

Relevancy and Accuracy of Academic Guidance

Trust in AI-generated academic advice is paramount for adoption. Testing revealed strong confidence in Nexly's recommendations, with 85.2% (52 students) providing 5-star accuracy ratings and 13.1% (8 students) giving 4-star ratings. Only one participant (1.6%) assigned a 3-star rating, with no lower ratings recorded. This distribution indicates that Nexly's retrieval-augmented generation architecture successfully delivers contextually appropriate and academically sound guidance that aligns with institutional requirements.

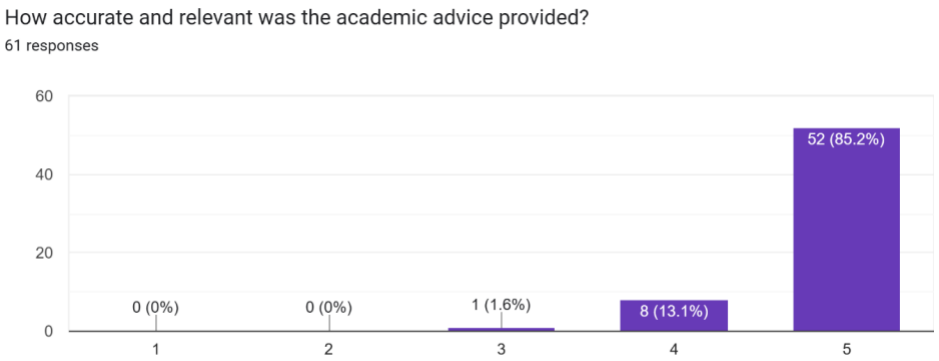

Figure 41: Bar chart of accuracy ratings

User Confidence in Following AI Recommendations

Beyond accuracy, the willingness of students to act upon AI-generated advice serves as a critical validation metric. Results demonstrated exceptionally high confidence levels, with 90.2% (55 students) rating their confidence at 5 stars and 8.2% (5 students) at 4 stars. Only 1.6% (1 student)

expressed moderate confidence with a 3-star rating. These findings suggest that Nexly has achieved a level of trustworthiness sufficient to influence academic decision-making, a crucial threshold for practical deployment.

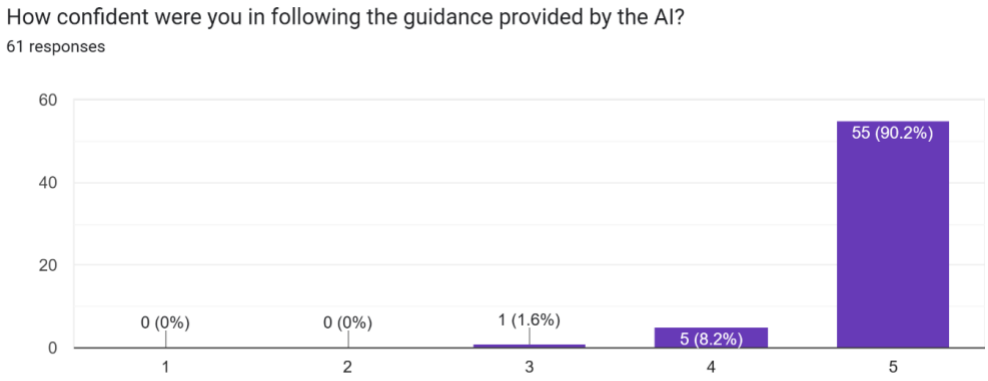

Figure 42: Bar chart of user confidence ratings

Efficiency and Performance

Speed Advantage Over Traditional Advising

One of Nexly's primary value propositions is accelerated access to academic information. Testing validated this advantage decisively, with 77% of participants reporting that Nexly significantly expedited their ability to obtain answers compared to traditional advising methods. An additional 18% indicated that Nexly provided somewhat faster access to information. A small neutral response (5%) was recorded, with no participants reporting slower or equivalent performance. These results confirm that asynchronous, AI-powered advising substantially reduces information retrieval time.

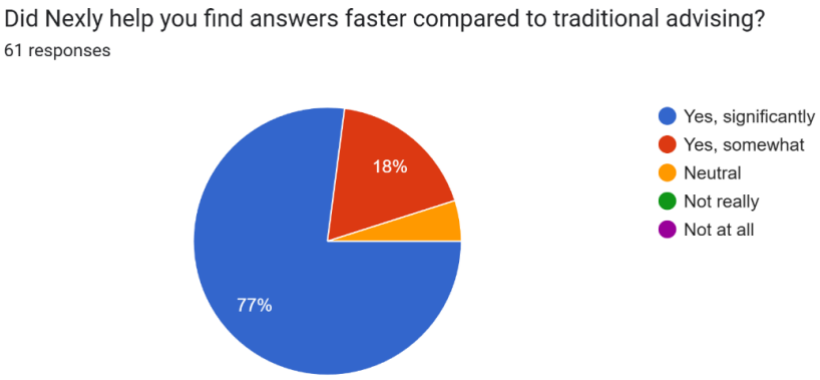

Figure 43: Pie chart showing speed comparison - Yes significantly 77%, Yes somewhat 18%, Neutral 5%

Feature Evaluation

Participants identified which platform features provided the greatest value during their testing sessions. Multiple selections were permitted, revealing clear usage patterns:

- **Course Selection Guidance**: 95.1% (58 students) – the most valued feature, confirming that semester planning assistance addresses a critical student need

- **User Profile Dashboard**: 85.2% (52 students) – centralized academic information display proved highly useful
- **Degree Requirement Explanation**: 80.3% (49 students) – clarification of program-specific requirements was frequently accessed
- **Responsiveness/UI**: 73.8% (45 students) – interface quality directly contributed to positive user experience
- **Quick Q&A Chatbot**: 68.9% (42 students) – conversational interaction for immediate clarification was well-utilized
- **Study Plan Suggestions**: 59.0% (36 students) – multi-semester planning was valuable but used more selectively
- **Graduation Planning**: 41.0% (25 students) – long-term planning features served specific user segments

These prioritizations provide clear direction for feature development and resource allocation in future iterations.

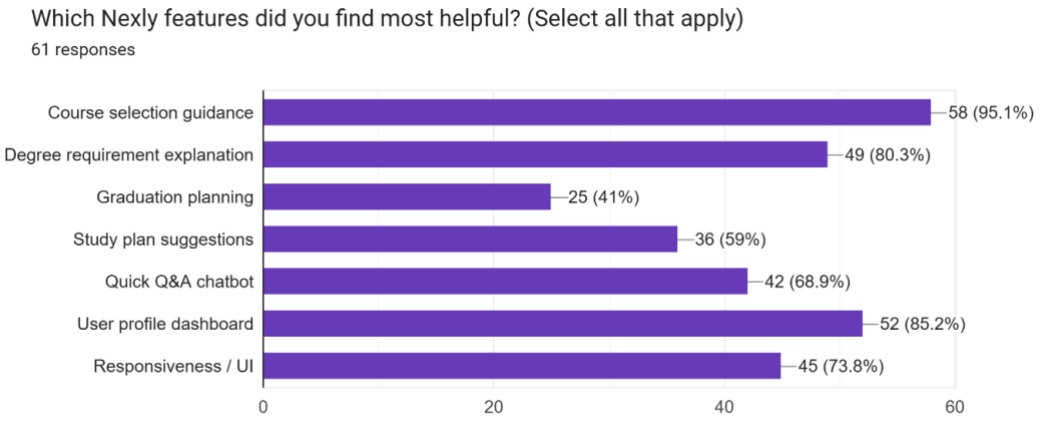

Figure 44: Horizontal bar chart of feature helpfulness percentage

Overall Satisfaction and Recommendation Likelihood

Aggregate User Satisfaction

Overall platform satisfaction served as a composite metric reflecting usability, accuracy, performance, and utility. Results indicated exceptional approval, with 90.2% (55 students) providing 5-star ratings and 6.6% (4 students) giving 4-star ratings. Only 3.3% (2 students) assigned 3-star ratings, with no lower evaluations recorded. This distribution demonstrates that Nexly successfully integrates its component features into a cohesive, valuable user experience.

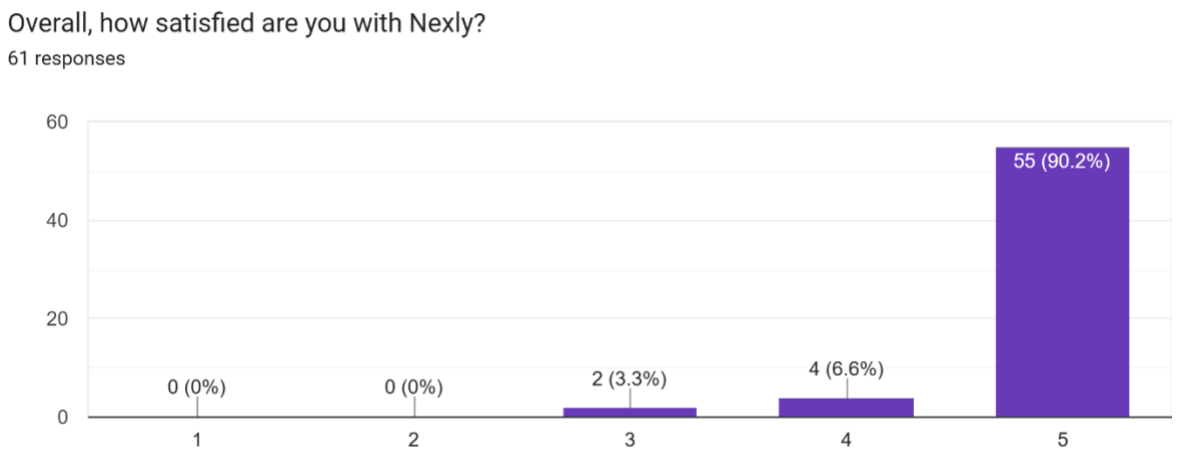

Figure 45: Bar chart of overall satisfaction ratings

Recommendation Propensity

The willingness of users to recommend Nexly to peers serves as a critical indicator of perceived value and adoption potential. Results showed 85.2% (52 students) definitively recommending the platform, with an additional 13.1% (8 students) providing standard positive recommendations. Only 1.6% (1 student) responded neutrally, with no negative responses recorded. These metrics suggest strong word-of-mouth potential and indicate readiness for broader institutional deployment.

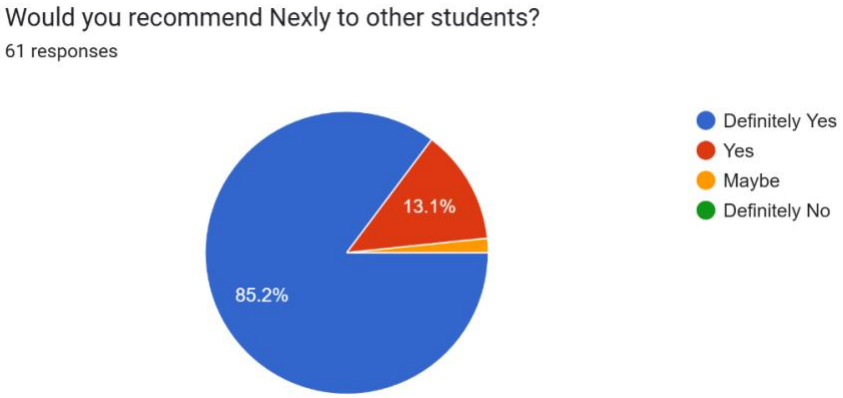

Figure 46: Pie chart - Definitely Yes 85.2%, Yes 13.1%, Maybe 1.6%

Limitations and Future Work

Current Limitiations

Restricted Handbook Coverage

During development, curriculum data from only two colleges, Computing and Informatics, and Sciences, was fully processed and integrated into the system's knowledge base. Additionally, while

one student handbook was prepared for fine-tuning purposes, this dataset was never deployed due to the computational overhead of running fine-tuned models in the local testing environment. Consequently, certain handbook-specific policies, academic regulations, and program-specific nuances were not evaluated during user trials, limiting the breadth of academic scenarios the system can accurately address.

Absence of Student Information System Integration

Nexly currently operates exclusively on publicly available institutional data. The system does not interface with the University of Sharjah's Student Information System (SIS), precluding access to individualized student records such as cumulative GPA, completed credit hours, course enrollment history, and academic holds. This limitation prevents the implementation of personalized features including automated degree audits, real-time course eligibility verification, and individualized academic pathway recommendations based on actual transcript data.

Monolingual Interface and Processing

The user interface and the underlying RAG model operate solely in English, and the LLM does respond in Arabic, but the quality might be compromised. While this serves a huge portion of the student body, it excludes Arabic-speaking students who may prefer or require advising support in their native language. This linguistic constraint limits accessibility and does not fully reflect the multilingual environment characteristic of the University of Sharjah and the broader UAE higher education landscape.

Static Data Dependencies

Although Nexly provides study plans and interprets academic policies, it cannot access real-time course availability data, section enrollment capacities, or live scheduling updates. The system relies on static institutional data rather than dynamic academic feeds, which may result in outdated recommendations during high-activity periods such as registration windows, add/drop deadlines, and semester transitions. This gap reduces the system's utility for time-sensitive advising decisions.

Preliminary Agentic Capabilities

While Nexly's architecture incorporates an agentic AI framework designed to enable autonomous task execution, current agent functionalities remain limited. Advanced capabilities such as automated form completion, appointment scheduling with human advisors, and multi-step procedural task execution are not yet implemented. The system's "agent actions" are conceptual rather than operationally integrated with university administrative systems.

Future Development Priorities

Student Information System API Integration

A primary objective for future development is establishing a secure, authenticated connection to the university's SIS via an officially sanctioned API. This integration would enable transformative personalized advising features including real-time degree audit generation, prerequisite satisfaction verification, dynamic GPA calculation, and automated enrollment recommendations. Implementation will require institutional approvals, data security compliance, and robust authentication mechanisms to protect sensitive student records.

Arabic Language Support

Expanding Nexly into a fully bilingual platform represents a critical accessibility enhancement. Future iterations will implement a complete Arabic user interface alongside Arabic-capable language model responses. This development will require Arabic natural language processing capabilities, culturally appropriate interface design, and validation by Arabic-speaking users. Successful implementation will substantially broaden the platform's reach and inclusivity within the student population.

Real-Time Academic Data Feeds

Future versions aim to integrate live data streams from university administrative systems, enabling features such as up-to-the-minute course seat availability, last-minute schedule modifications, real-time waitlist status, and instant registration alerts. This capability would transform Nexly from an informational tool into a dynamic, always-current advising assistant that reflects the live state of academic operations.

Proactive Notification System

To enhance user engagement and reduce missed deadlines, future iterations will implement personalized push notifications for critical academic milestones including registration opening dates, add/drop period deadlines, examination schedules, financial aid deadlines, and policy updates affecting specific student cohorts. This proactive communication layer would shift the platform from reactive query-response interactions to anticipatory guidance delivery.

Native Mobile Application Development

While the current Nexly web application includes mobile-responsive design, a dedicated native mobile application for iOS and Android platforms is planned. A purpose-built mobile application would provide offline data access, native push notification support, biometric authentication, deeper device integration, and optimized performance on mobile networks. This development would significantly improve accessibility and user engagement, particularly for students who primarily access university services via smartphones.

Expanded College Coverage

Future phases will systematically incorporate curriculum data, academic regulations, and program structures from all University of Sharjah colleges beyond Computing and Informatics and Sciences. This expansion will require coordinated data collection efforts, validation protocols with academic departments, and incremental knowledge base construction to ensure comprehensive institutional coverage and equitable service delivery across all student populations.

Chapter 5: Conclusion and Final Thoughts

Institutional AI Governance and Legalization

The deployment of artificial intelligence systems in higher education institutions extends beyond technical implementation to encompass critical questions of governance, ethics, and regulatory compliance. As Nexly represents an institutionally integrated AI solution handling student data and providing academic guidance, its development and potential deployment must be considered within the broader framework of AI governance structures emerging across the educational sector and the United Arab Emirates specifically.

The Emerging Landscape of AI Governance in Higher Education

Higher education institutions worldwide are grappling with the challenge of establishing comprehensive governance frameworks for artificial intelligence. Recent research reveals significant gaps in institutional preparedness, with EDUCAUSE's 2024 AI Landscape Study documenting that the majority of higher education institutions lack robust AI-related policies and guidelines. This policy vacuum creates uncertainty around questions of algorithmic accountability, data stewardship, transparency obligations, and the appropriate balance between innovation and oversight.

Multiple frameworks have emerged to address these governance challenges. Anthology's 2024 AI Policy Framework establishes seven foundational principles for higher education AI deployment: fairness, reliability, human oversight, transparency and explainability, privacy and security, value alignment, and accountability. These principles align with international standards including the NIST AI Risk Management Framework, the European Union AI Act, and OECD AI Principles, creating a converging global consensus on responsible AI governance. The recently published Higher Education Act for AI, or HEAT-AI framework, provides operational guidance for universities implementing AI systems, emphasizing the need for continuous evaluation, stakeholder engagement, and adaptive policy structures that can evolve alongside rapidly changing AI capabilities.

Chan's comprehensive AI Ecological Education Policy Framework proposes a three-dimensional governance model encompassing pedagogical, governance, and operational considerations. This framework acknowledges that effective AI governance requires addressing not merely technical specifications but the complex interplay between educational mission, institutional values, stakeholder concerns, and regulatory obligations. For student-facing AI systems like Nexly, this translates to considerations around academic integrity, equitable access, transparency about system capabilities and limitations, mechanisms for human oversight and intervention, and clear accountability structures when automated guidance proves insufficient or incorrect.

UAE Regulatory Framework and Strategic Positioning

The United Arab Emirates has established itself as a pioneer in artificial intelligence governance, becoming the first nation to appoint a Minister of State for Artificial Intelligence in 2017 and subsequently developing one of the world's most comprehensive national AI strategies. The UAE National Strategy for Artificial Intelligence 2031 articulates an ambitious vision to position the Emirates as a global AI leader by systematically integrating artificial intelligence across key sectors including healthcare, transportation, and critically for Nexly's context, education. This

strategic framework creates an enabling environment for innovative AI applications while simultaneously establishing expectations for responsible development and deployment.

In July 2024, the UAE launched the UAE Charter for the Development and Use of Artificial Intelligence, establishing twelve guiding principles for AI development: strengthening human-machine collaboration, safety assurance, algorithmic bias mitigation, data privacy protection, transparency, human oversight, governance and accountability, technological excellence, human commitment, peaceful coexistence with AI, inclusive access, and compliance with applicable legal frameworks. These principles establish clear expectations for AI systems operating within UAE jurisdiction, requiring developers and deploying institutions to demonstrate adherence to ethical standards while fostering innovation.

The establishment of regulatory bodies further structures the AI governance landscape. Law Number 3 of 2024 formally created the Artificial Intelligence and Advanced Technology Council in Abu Dhabi, tasked with regulating AI projects, research, and investments within the emirate. The UAE Council for Artificial Intelligence, operating at the federal level, oversees AI integration across government departments and the education sector specifically, proposing policies to create an AI-friendly ecosystem while ensuring ethical deployment. These institutional structures signal the UAE government's recognition that AI systems require active oversight rather than operating within an unregulated environment.

For educational AI applications, the implications are profound. The UAE government has mandated artificial intelligence as a subject across all grade levels from kindergarten through Grade 12 beginning in 2025, demonstrating commitment to AI literacy as a foundational educational competency. This policy context suggests receptiveness to AI-enhanced educational services while simultaneously establishing expectations that such systems will be developed responsibly, deployed transparently, and subjected to appropriate institutional oversight. Academic institutions deploying AI systems like Nexly must therefore ensure alignment with emerging national frameworks while establishing internal governance structures appropriate to their specific institutional context and student population.

Governance Considerations for Nexly Deployment

Translating these broad governance principles and regulatory frameworks into operational guidelines for Nexly's deployment requires addressing several specific institutional considerations. First, the University of Sharjah must establish clear policies governing student data utilized by the system. While Nexly currently operates exclusively on public institutional documents, future integration with the Student Information System would involve access to individually identifiable student records including academic transcripts, enrollment history, and potentially demographic information. Such integration necessitates robust data protection protocols, explicit student consent mechanisms, clear data retention and deletion policies, and stringent access controls ensuring student information remains confidential and is utilized exclusively for legitimate educational purposes.

Second, institutional governance structures must address questions of accountability and oversight. The agentic architecture enabling Nexly to autonomously retrieve information, formulate responses, and potentially execute actions requires clear delineation of responsibility when system outputs prove inaccurate or inappropriate. Establishing oversight committees comprising academic advisors, technology specialists, student representatives, and institutional leadership creates mechanisms for continuous monitoring, evaluation, and refinement of system performance. Regular audits examining response accuracy, bias detection, and adherence to institutional policies

ensure Nexly operates within acceptable parameters while identifying areas requiring intervention or improvement.

Third, transparency obligations require clear communication to students about Nexly's capabilities, limitations, and appropriate use cases. Students must understand when they are interacting with an AI system rather than a human advisor, recognize the distinction between routine informational queries appropriate for automated handling and complex academic decisions requiring human judgment, and know how to escalate concerns or request human intervention when automated guidance proves insufficient. Interface design decisions, explicit system disclaimers, and comprehensive user education materials collectively fulfill these transparency obligations while managing student expectations appropriately.

Fourth, equity considerations demand ensuring all students have equal access to AI-enhanced advising services regardless of technological literacy, language proficiency, disability status, or socioeconomic background. Future bilingual capabilities addressing Arabic-speaking students, accessibility features supporting students with disabilities, and alternative channels for students lacking consistent internet connectivity or appropriate devices ensure Nexly enhances rather than exacerbates existing educational inequalities. Universal design principles embedded from initial development through continuous refinement help ensure the system serves the entire student population rather than creating a two-tiered advising structure.

Finally, institutional AI governance requires establishing procedures for continuous evaluation and adaptive policy refinement. The rapid evolution of large language model capabilities, changing regulatory requirements, shifting student needs, and emerging best practices necessitate treating AI governance as an iterative process rather than a one-time policy declaration. Regular stakeholder consultations, systematic collection of user feedback, ongoing assessment of system performance metrics, and periodic review of governance policies ensure Nexly's deployment remains aligned with institutional values, regulatory obligations, and student interests as circumstances evolve.

Why Nexly Matters

Nexly represents more than a technological demonstration or academic exercise. The system embodies a response to fundamental challenges confronting contemporary higher education while pointing toward transformative possibilities for how universities support student success. Understanding why Nexly matters requires examining both the immediate practical problems it addresses and the broader implications of AI-enhanced educational support systems for the future of academic advising, institutional efficiency, and student experience.

Addressing Critical Gaps in Academic Support Infrastructure

Higher education institutions worldwide face persistent challenges in providing adequate academic advising to growing and increasingly diverse student populations. Research consistently demonstrates the critical importance of quality academic advising for student retention, timely degree completion, and overall satisfaction with the university experience. However, traditional advising models struggle to scale effectively as enrollments increase while budgetary constraints limit advisor hiring. The resulting advisor-to-student ratios often leave students unable to access guidance when needed, with appointment waiting times stretching to weeks during peak registration periods and routine informational queries consuming advisor time that could be devoted to complex cases requiring human judgment and empathy.

Nexly directly addresses this capacity challenge by automating responses to routine informational inquiries that constitute the majority of advising interactions. Questions about prerequisite requirements, course offering patterns, program requirements, academic policies, and procedural timelines represent well-defined information retrieval tasks ideally suited for AI systems. By handling these routine queries instantly and accurately, Nexly preserves human advisor capacity for the complex, nuanced advising interactions that genuinely require professional expertise: helping students navigate academic difficulty, exploring alternative degree pathways, addressing personal circumstances affecting academic progress, and providing the mentorship and encouragement essential to student persistence. This capacity augmentation enables institutions to provide better service to all students without proportionally scaling advising staff, addressing resource constraints while improving support quality.

The system's twenty-four-hour availability addresses temporal accessibility barriers that traditional advising hours create for many students. Working students, those with family responsibilities, international students in different time zones, and evening program participants often cannot access advising during standard business hours. Nexly's persistent availability ensures all students can obtain academic information when convenient for their schedules rather than coordinating around advisor availability. This accessibility enhancement particularly benefits students from backgrounds historically underserved by higher education, reducing friction in navigating academic systems and potentially improving retention among populations that institutions seek to support more effectively.

Demonstrating Responsible AI Integration in Education

Beyond immediate functional benefits, Nexly matters as a case study in thoughtful artificial intelligence integration within higher education contexts. The system's development reflects careful attention to responsible AI principles rather than pursuing technological sophistication as an end goal. The architecture's transparency regarding information sources, explicit acknowledgment of limitations, and integration pathways preserving human oversight demonstrate that effective AI systems can augment rather than displace human expertise while maintaining accountability and institutional control.

This approach addresses legitimate faculty and staff concerns about artificial intelligence's role in education. Rather than presenting AI as a wholesale replacement for human advisors or expertise, Nexly occupies a carefully delimited domain handling well-defined informational tasks while escalating complex queries requiring judgment, interpretation, or emotional intelligence to human professionals. This human-in-the-loop architecture respects professional expertise while acknowledging AI's comparative advantage in information retrieval, consistency, and scalability. The model provides a replicable template for other educational AI applications, demonstrating how institutions can harness AI capabilities while maintaining educational quality and professional standards.

The development process itself offers lessons for educational technology initiatives broadly. Beginning with junior-phase prototyping, iterating through multiple platform approaches, conducting extensive user testing, and refining the system based on empirical feedback rather than theoretical assumptions resulted in a solution genuinely addressing user needs rather than imposing predetermined technological solutions. This user-centered development methodology, combined with transparent performance evaluation and honest acknowledgment of current limitations, models a mature approach to educational technology that the sector increasingly requires as AI capabilities advance and deployment pressures intensify.

Advancing Institutional Digital Transformation

Nexly represents a concrete step in the University of Sharjah's broader digital transformation journey, demonstrating institutional capacity to develop rather than merely purchase technology solutions. The project showcases student and faculty capability to design, implement, and evaluate sophisticated AI systems addressing real institutional challenges. This internal development capacity matters both practically, enabling customized solutions fitting specific institutional contexts, and culturally, building organizational competence and confidence in emerging technologies that will increasingly shape higher education's operational and pedagogical landscape.

The system's modular architecture and extensible design create foundations for expanded functionality addressing additional institutional needs. The retrieval-augmented generation framework established for academic advising could be adapted to support course selection, career counseling, research opportunity discovery, or student service navigation. The multi-agent routing system could orchestrate specialized AI assistants for different functional domains while maintaining consistent user experience. The knowledge integration pipeline processing institutional documents could expand to continuously updated handbooks, policy changes, and evolving program requirements. Rather than a standalone application, Nexly represents infrastructure supporting various AI-enhanced services as institutional needs and technological capabilities evolve.

This infrastructure development aligns with broader trends in higher education technology. Leading institutions increasingly recognize that generic commercial solutions cannot address their unique institutional contexts, student populations, and academic structures. Institution-specific AI systems trained on local data, reflecting institutional policies, and customized to local workflows provide advantages that off-the-shelf products cannot match. Nexly demonstrates this principle while establishing technical capabilities and organizational processes enabling future custom development initiatives.

Preparing Students for an AI-Augmented Future

Perhaps most fundamentally, Nexly matters because it provides students with experience interacting with AI systems in academically meaningful contexts while still in protected educational environments. Students graduating into increasingly AI-saturated professional landscapes must develop competencies around evaluating AI output, recognizing system capabilities and limitations, understanding when to trust automated guidance versus seeking human expertise, and effectively collaborating with AI systems as work tools. Nexly offers a low-stakes environment for developing these critical literacies while pursuing genuine academic goals rather than artificial training scenarios.

The transparency around Nexly's architecture, capabilities, and functioning creates learning opportunities beyond its immediate advising function. Students understanding how the system retrieves information, formulates responses, and acknowledges limitations develop mental models of AI systems generalizable to other contexts. This understanding helps counter both naive over-trust in AI infallibility and uninformed rejection of AI utility, instead fostering appropriately calibrated expectations and skillful system usage. These capabilities represent crucial preparation for professional environments where AI systems will increasingly mediate information access, decision support, and task execution.

Moreover, for computer science students specifically, Nexly demonstrates that meaningful AI applications can be developed at university scale using accessible technologies and reasonable resources. The project's existence as a senior capstone deliverable rather than a commercial product or heavily-resourced institutional initiative shows students that they can create substantive AI systems addressing real problems rather than merely consuming AI services developed elsewhere. This empowerment matters for developing the next generation of AI practitioners who will shape how these technologies are designed, deployed, and governed throughout their careers.

Conclusion

The development and evaluation of Nexly represents a comprehensive journey from conceptual vision to functional prototype, demonstrating both the possibilities and complexities of integrating artificial intelligence into higher education support systems. This project has traversed technical challenges, navigated user experience considerations, confronted real-world deployment constraints, and ultimately delivered a system that addresses genuine institutional needs while establishing foundations for continued evolution and enhancement.

The technical achievements merit recognition while maintaining appropriate perspective on their significance. The system successfully implements a sophisticated retrieval-augmented generation architecture, integrating semantic search, vector databases, large language models, and multi-agent coordination into a coherent platform delivering accurate, context-aware academic guidance. The 96% response correctness rate and strong user satisfaction metrics validate fundamental architectural decisions while demonstrating practical viability for institutional deployment. The modern web application built using Next.js provides responsive, accessible interaction that students find intuitive and helpful, reducing friction in accessing academic information and enabling self-service resolution of common advising queries.

Yet technical sophistication alone does not constitute project success. The careful attention to user needs throughout development, reflected in extensive testing protocols and systematic incorporation of feedback, ensures Nexly addresses real rather than imagined problems. The honest assessment of limitations including current handbook coverage restrictions, absence of Student Information System integration, monolingual operation, static data dependencies, and preliminary agentic capabilities demonstrates intellectual maturity and research integrity. These acknowledged constraints do not diminish project value but rather frame realistic expectations while charting pathways for future enhancement.

The governance and regulatory analysis situates Nexly within broader contexts that any institutional AI deployment must navigate. The alignment between project approach and emerging higher education AI frameworks validates the fundamental design philosophy of augmenting rather than replacing human expertise while maintaining accountability, transparency, and institutional oversight. The careful consideration of UAE regulatory requirements, particularly the National AI Strategy 2031 and the UAE Charter for AI development, demonstrates awareness that technological capability must be matched with regulatory compliance and ethical responsibility. Future deployment success depends not merely on technical robustness but on institutional governance structures, policy frameworks, and stakeholder engagement ensuring responsible AI integration.

The broader significance of this work extends beyond the University of Sharjah's immediate advising challenges. Nexly demonstrates that meaningful AI applications can emerge from academic environments rather than solely from commercial vendors or research laboratories. The project provides a replicable model for other institutions seeking to develop customized AI

solutions addressing their unique contexts while maintaining control over functionality, data handling, and ethical standards. The technical documentation, architectural patterns, and lessons learned represent contributions to collective knowledge that accelerate the field's maturation and enable more informed decision-making by educational technology leaders.

Looking forward, several development trajectories promise substantial value enhancement. The human element remains central to this project's ultimate success or failure. While artificial intelligence capabilities continue advancing at remarkable pace, the human judgment, empathy, contextual understanding, and relationship-building that characterize exceptional academic advising remain beyond current AI systems. Nexly succeeds not by replicating these human capabilities but by automating routine tasks that free human advisors to focus on interactions where their distinctive expertise matters most. The system's value lies not in replacing advisors but in amplifying their impact, enabling each professional to serve more students more effectively by handling informational queries that would otherwise consume their limited time and attention.

This recognition of AI's proper role within educational contexts reflects broader wisdom about technology's place in fundamentally human endeavors. Education remains at its core a profoundly human enterprise built on relationships, inspiration, challenge, support, and growth. Technology serves education most effectively when it removes barriers, reduces friction, expands access, and enables more meaningful human connection rather than attempting to automate the human elements that make education transformative. Nexly embodies this philosophy through architecture that preserves human oversight, acknowledges limitations, enables escalation to human advisors, and focuses AI capabilities on well-defined informational tasks where automation provides clear value without compromising educational quality.

The development team's growth throughout this project perhaps represents outcomes as significant as the system itself. The technical skills acquired spanning modern web development, cloud infrastructure, vector databases, language model integration, and full-stack architecture provide capabilities applicable far beyond this specific project. The research competencies developed including literature review, user testing methodology, quantitative evaluation, and technical writing prepare team members for future research contributions whether in academic or industry settings. The collaborative skills honed through distributed development, code review, integration challenges, and shared decision-making model professional software development practices essential for career success.

Most fundamentally, the project demonstrates that students can engage meaningfully with cutting-edge technologies to solve real institutional problems rather than merely completing academic exercises. This empowerment matters for developing confident, capable practitioners who understand that they can shape technology's trajectory rather than passively accepting others' implementations. The experience of identifying problems, designing solutions, confronting implementation challenges, evaluating outcomes, and communicating results builds professional identity and self-efficacy that will serve team members throughout their careers regardless of specific technical domains or roles they ultimately pursue.

As artificial intelligence capabilities continue advancing and educational institutions increasingly integrate these technologies into core functions, projects like Nexly provide essential learning experiences about both possibilities and responsibilities. The technical achievements demonstrate what current AI systems can accomplish when thoughtfully architected and carefully implemented. The limitations and future work acknowledge that substantial challenges remain before AI-enhanced advising fully realizes its potential. The governance analysis recognizes that technical capability must be matched with institutional oversight, regulatory compliance, and ethical

responsibility. The broader significance discussion situates this work within larger contexts of educational access, institutional transformation, and professional preparation.

In the final analysis, Nexly matters not because it perfectly solves academic advising challenges or represents the ultimate expression of educational AI but because it demonstrates thoughtful, responsible progress toward better supporting student success. The system provides genuine value to students through improved information access while respecting human advisors' essential role in complex academic guidance. It establishes technical infrastructure enabling future enhancement while maintaining manageable complexity. It models governance approaches balancing innovation with accountability. It contributes knowledge to research and practice communities working to understand AI's appropriate role in education. And it prepares students for futures where effective AI collaboration represents essential professional competence.

The journey from initial concept to functional prototype has been challenging, enlightening, and ultimately rewarding. The lessons learned, capabilities developed, and system delivered represent substantial accomplishments worthy of recognition while establishing foundations for continued evolution. As Nexly transitions from academic project to potential institutional service, the principles guiding its development remain relevant: center user needs, maintain technical rigor, acknowledge limitations, preserve human judgment, ensure institutional oversight, and approach innovation with both enthusiasm and responsibility. These principles will serve the system well as it grows and matures, just as they have guided this development phase to successful completion.

The future of higher education will undoubtedly involve increasingly sophisticated artificial intelligence systems augmenting traditional educational practices. Projects like Nexly help shape that future by demonstrating what works, identifying what doesn't, and modeling how institutions can integrate AI capabilities while maintaining educational values and serving student interests. The work continues, challenges remain, and opportunities abound. But this project has moved the conversation forward, contributed to collective understanding, and delivered practical value to the University of Sharjah community. That represents success by any reasonable measure, and a foundation upon which future work can build toward even greater impact.

The End

Sat 22nd Nov, 2025

References

- Anthology. (2024). *AI policy framework*. https://www.anthology.com/news/new-ai-policy-framework-from-anthology-empowers-higher-education-to-balance-the-risks-and
- Chan, C. K. Y. (2023). A comprehensive AI policy education framework for university teaching and learning. *International Journal of Educational Technology in Higher Education, 20*(38). https://doi.org/10.1186/s41239-023-00408-3
- EDUCAUSE. (2024). *2024 EDUCAUSE action plan: AI policies and guidelines*. https://www.educause.edu/research/2024/2024-educause-action-plan-ai-policies-and-guidelines
- Garant Business Consultancy. (2025). *AI in the UAE in 2025: Regulation, real-world applications, and strategic business opportunities*. https://garant.ae/en/insights/ai-in-the-uae-in-2025-regulation-real-world-applications-and-strategic-business-opportunities
- Global Legal Insights. (2025). *AI, machine learning & big data laws 2025: UAE*. https://www.globallegalinsights.com/practice-areas/ai-machine-learning-and-big-data-laws-and-regulations/uae/
- Humble, N. (2025). Higher education AI policies—A document analysis of university guidelines. *European Journal of Education.* https://doi.org/10.1111/ejed.70214
- Jiang, S., et al. (2025). Exploring the effectiveness of institutional policies and regulations for generative AI usage in higher education. *Higher Education Quarterly.* https://doi.org/10.1111/hequ.70054
- Latham & Watkins. (2024). *AI in the UAE: Understanding the regulatory landscape and key authorities*. https://www.lw.com/en/insights/ai-in-the-uae-understanding-the-regulatory-landscape-and-key-authorities
- Temper, M., Tjoa, S., & David, L. (2025). Higher Education Act for AI (HEAT-AI): A framework to regulate the usage of AI in higher education institutions. *Frontiers in Education, 10.* https://doi.org/10.3389/feduc.2025.1505370
- UAE Government. (2024). *Artificial intelligence in government policies*. https://u.ae/en/about-the-uae/digital-uae/digital-technology/artificial-intelligence/artificial-intelligence-in-government-policies
- UAE Government. (2024). *UAE's international stance on artificial intelligence policy*. https://uaelegislation.gov.ae/en/policy/details/uae-s-international-stance-on-artificial-intelligence-policy
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Küttler, H., Lampe, A., Wu, Y., Fisch, A., Science, A., & Riedel, S. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems.*
- Okonkwo, C. W., & Ade-Ibijola, A. (2021). Chatbots applications in education: A systematic review. *Computers & Education: Artificial Intelligence, 2*, 100033.
- Stone, P., Kaminka, G. A., Kraus, S., & Rosenschein, J. S. (2010). Ad hoc autonomous agent teams: Collaboration without pre-coordination. *Proceedings of the National Conference on Artificial Intelligence (AAAI).*
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). *MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers* (arXiv:2002.10957). https://arxiv.org/abs/2002.10957

- Wollny, S., et al. (2021). Are we there yet? A systematic literature review on chatbots in education. *Frontiers in Artificial Intelligence, 4.* https://doi.org/10.3389/frai.2021.654924
- Denoyelles, A., Brown, T., Seilhamer, R., & Chen, B. (2023). The evolving landscape of students' mobile learning practices in higher education. *EDUCAUSE Review.* https://er.educause.edu/articles/2023/1/the-evolving-landscape-of-students-mobile-learning-practices-in-higher-education
- Perera, P. (2022, January 4). Visual explanation and comparison of CSR, SSR, SSG and ISR. *DEV Community.* https://dev.to/pahanperera/visual-explanation-and-comparison-of-csr-ssr-ssg-and-isr-34ea
- Prismic. (n.d.). *Client-side rendering (CSR) vs. server-side rendering (SSR)*. https://prismic.io/blog/client-side-vs-server-side-rendering
- Yang, A., Goh, C. H., & Lim, J. Y. (2023). The research into dark mode: A systematic review using a two-stage approach and the S-O-R framework. *Social Space: International Journal of Social Sciences and Humanities, 23*(4).
- Apple Developer Documentation. (n.d.). *Buttons — Human Interface Guidelines*. https://developer.apple.com/design/human-interface-guidelines/buttons
- FAISS. (n.d.). *FAISS: A library for efficient similarity search*. Facebook AI Research. https://github.com/facebookresearch/faiss
- Groq. (n.d.). *Groq API documentation*. https://console.groq.com/docs
- Next.js. (n.d.). *Next.js documentation*. Vercel. https://nextjs.org/docs
- PostgreSQL. (n.d.). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/
- shadcn. (n.d.). *Components – shadcn/ui*. https://ui.shadcn.com/docs/components
- Supabase. (n.d.). *Supabase documentation*. https://supabase.com/docs
- Tailwind CSS. (n.d.). *Tailwind CSS documentation*. https://tailwindcss.com/docs
- Vercel. (n.d.). Geist Sans & Geist Mono. https://vercel.com/font
- Whisper. (n.d.). *Whisper speech recognition model*. OpenAI. https://github.com/openai/whisper
- University of Sharjah. (2024). *Undergraduate catalog 2024–2025*. University of Sharjah, Registrar's Office.
- University of Sharjah. (2024). *Student handbook* (2024–2025). University of Sharjah, Student Affairs.
- University of Sharjah. (2024). *College of Computing & Informatics study plans* (2024–2025). Academic Advising Unit.
- University of Sharjah. (2024). *Exam regulations and academic policies*. Registrar's Office.
- Caddy Server. (n.d.). *Caddy documentation*. https://caddyserver.com/docs
- Docker. (n.d.). *Docker documentation*. https://docs.docker.com/
- LangChain / LangGraph. (n.d.). *LangChain documentation*. https://python.langchain.com/
- OpenAI. (n.d.). *OpenAI Embeddings models*. https://platform.openai.com/docs/guides/embeddings

List of Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
API	Application Programming Interface
CCI	College of Computing & Informatics
CSR	Client-Side Rendering
DB	Database
DIM	Dimensionality (e.g., 384-dim embeddings)
DOCX	Microsoft Word Open XML Document
DOM	Document Object Model
Docker	Containerization Platform
FAISS	Facebook AI Similarity Search
Groq	Groq Inference API
HIG	Human Interface Guidelines
HTML	HyperText Markup Language
ISR	Incremental Static Regeneration
kNN	k-Nearest Neighbors
LangChain	LLM Orchestration Framework
LangGraph	Agentic Workflow Graph Framework
LLM	Large Language Model
MiniLM	Mini Language Model (all-MiniLM-L6-v2)
NLP	Natural Language Processing
PDF	Portable Document Format
PostgreSQL	Relational Database Management System
RAG	Retrieval-Augmented Generation
RLS	Row-Level Security
SIS	Student Information System
SSG	Static Site Generation
SSR	Server-Side Rendering
Supabase	Backend-as-a-Service Platform
Tailwind CSS	Utility-First CSS Framework
UI	User Interface
UoS	University of Sharjah
UX	User Experience
Whisper	Whisper Speech Recognition Model

Table 11: Table of Abbreviations for the report